操作系统原理

Operating Systems Principles

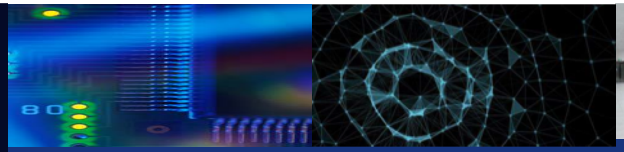
陈鹏飞
计算机学院

第六讲 — 进程调度

——处理器调度

概述

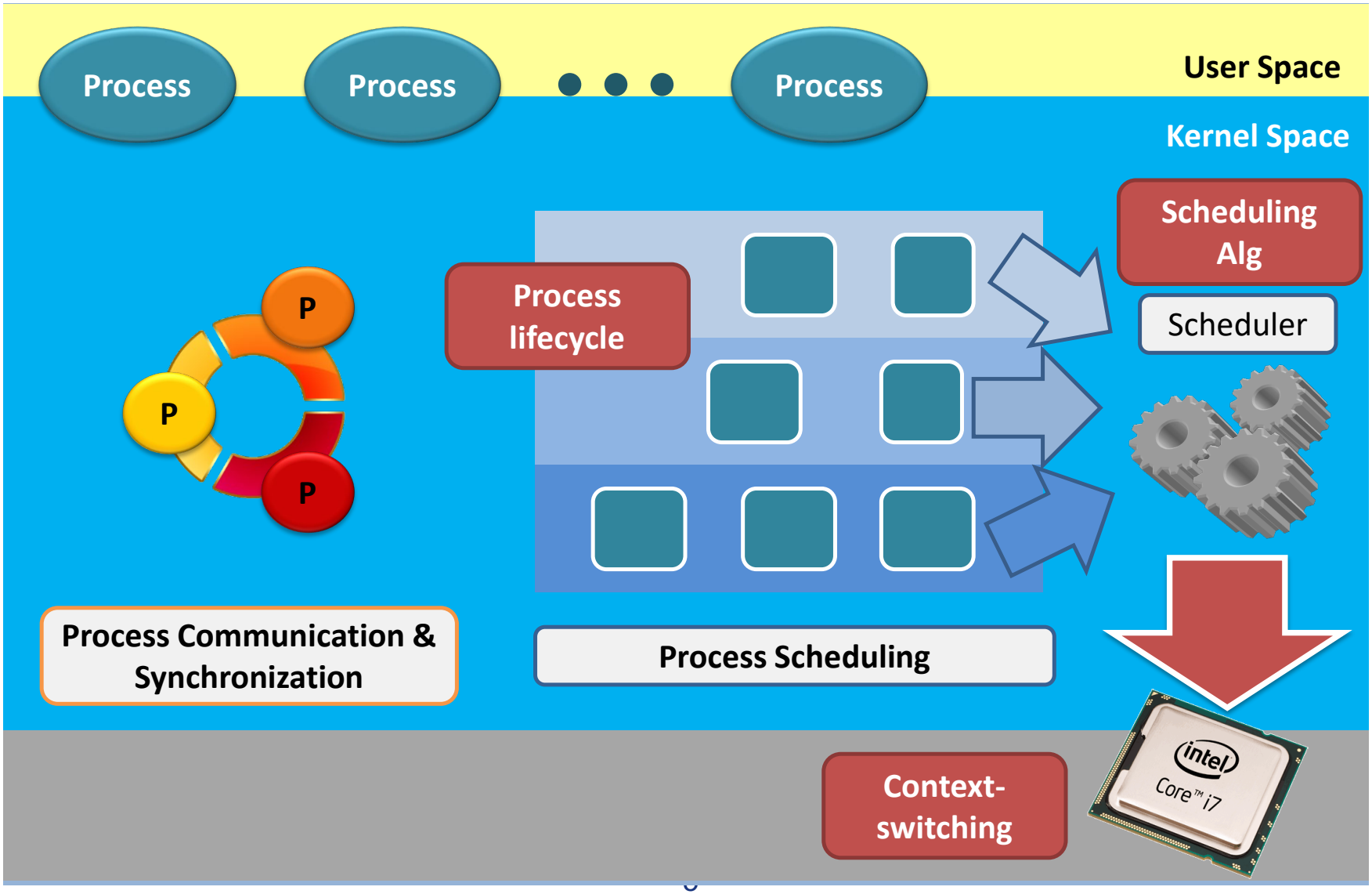
- 基本概念
- 调度标准
- 调度算法
- 线程调度
- 多处理器调度
- 实时CPU调度
- 操作系统示例
- 算法评估



目标

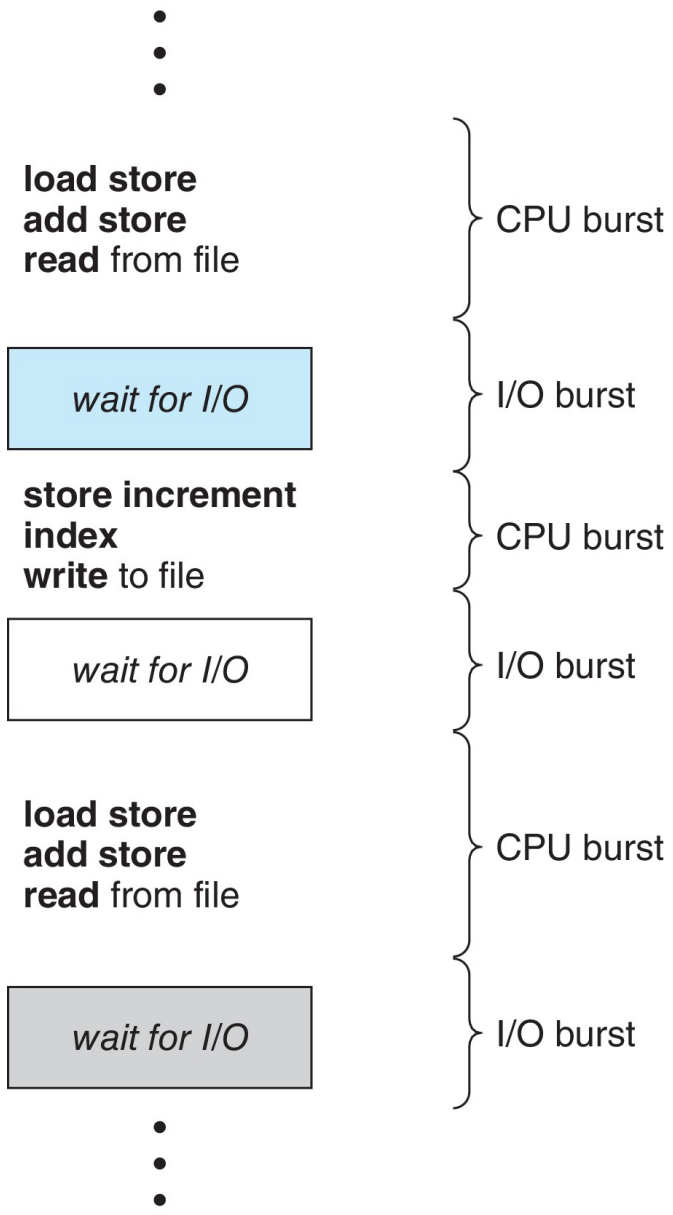
- 了解长程、中程、短程调度和区别；
- 描述各种CPU调度算法
- 根据调度标准评估CPU调度算法
- 解释与多处理器和多核调度相关的问题
- 描述各种实时调度算法
- 描述Windows、Linux和Solaris操作系统中使用的调度算法；
- 应用建模和仿真评估CPU调度算法
- 掌握传统UNIX系统中的调度思想；

处理器调度


Process
Process
Process
User Space
Kernel Space
Scheduling Alg
Scheduler
Process lifecycle
P
P
P
Process Communication & Synchronization
Process Scheduling
Context-switching
intel
Core™ i7

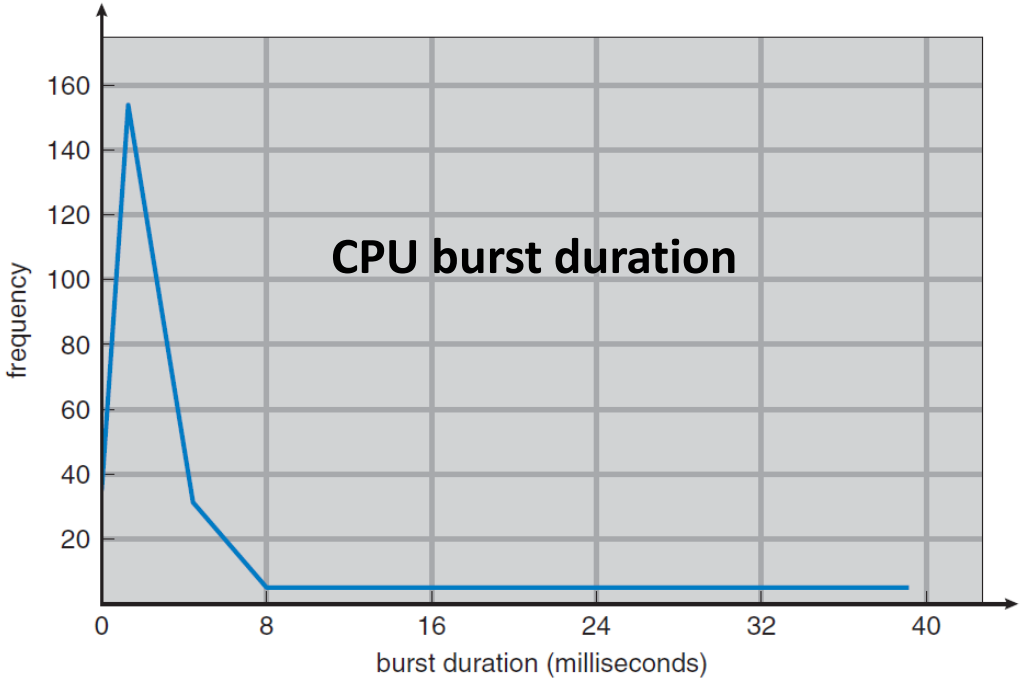
处理器调度

- 通过多道程序设计获得的最大CPU利用率
- CPU－I/O执行周期－进程执行由CPU执行周期和I/O等待周期组成
- CPU执行，然后是I/O执行
- CPU执行分布是主要关注的问题



为什么需要调度

- 进程执行包括了CPU执行和I/O执行；
- CPU执行时间与频率之间的关系形成指数或者超指数分布；
- 从分布上看，有大量短CPU执行和少量长CPU执行；
- I/O密集型程序通常是大量短CPU执行，CPU密集型程序可能只有少量长CPU执行。



为什么需要调度

Question. How to improve CPU utilization (CPU is much faster than I/O)?

Question. How to improve system responsiveness (interactive applications)?

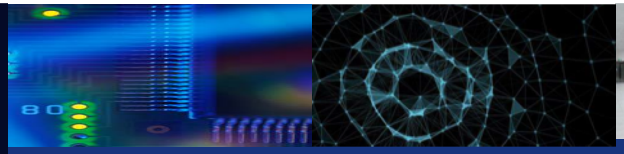
Multiprogramming

Multitasking

Demo

A system may contain many processes which are at different states (ready for running, waiting for I/O)

Scheduling is required because the number of computing resource – the CPU – is **limited**.

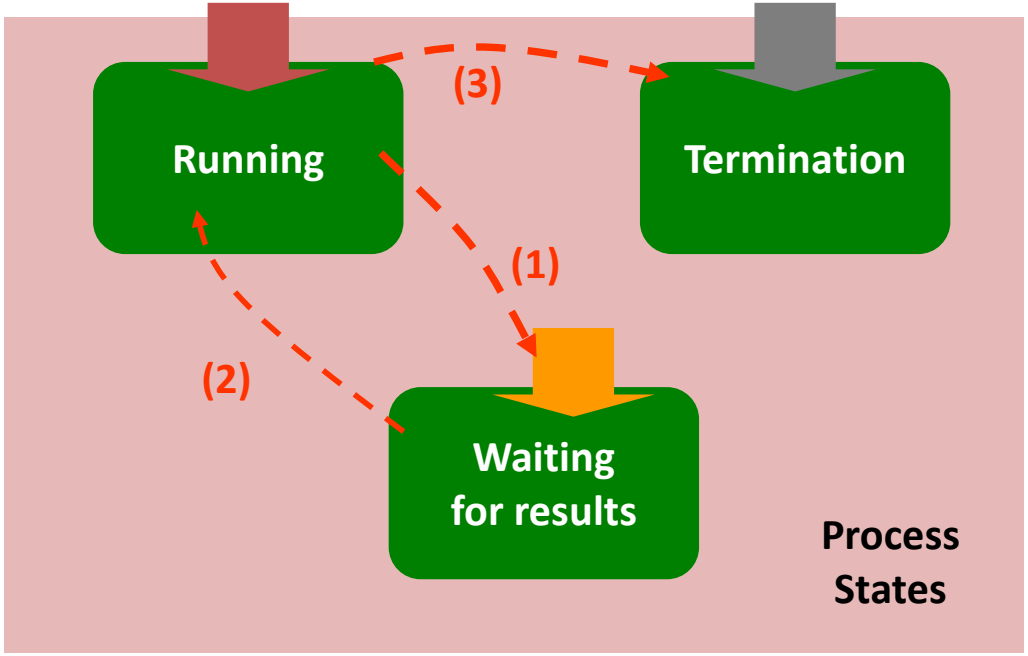


进程的生命周期

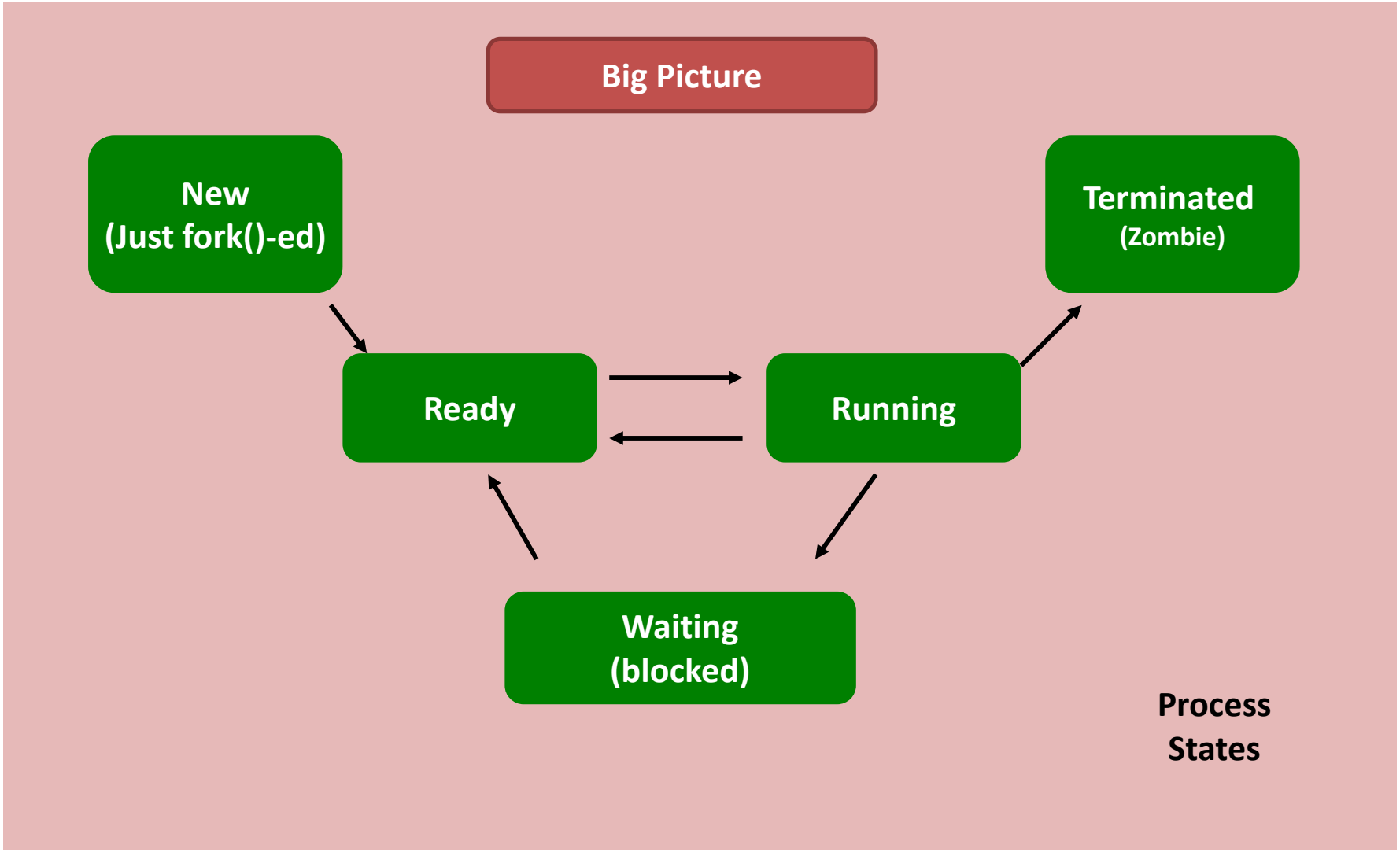
站在用户的角度看进程

- This is how a fresh programmer looks at a process' life cycle.

```
int main(void) {
    int x = 1;
    getchar();
    return x;
}
```



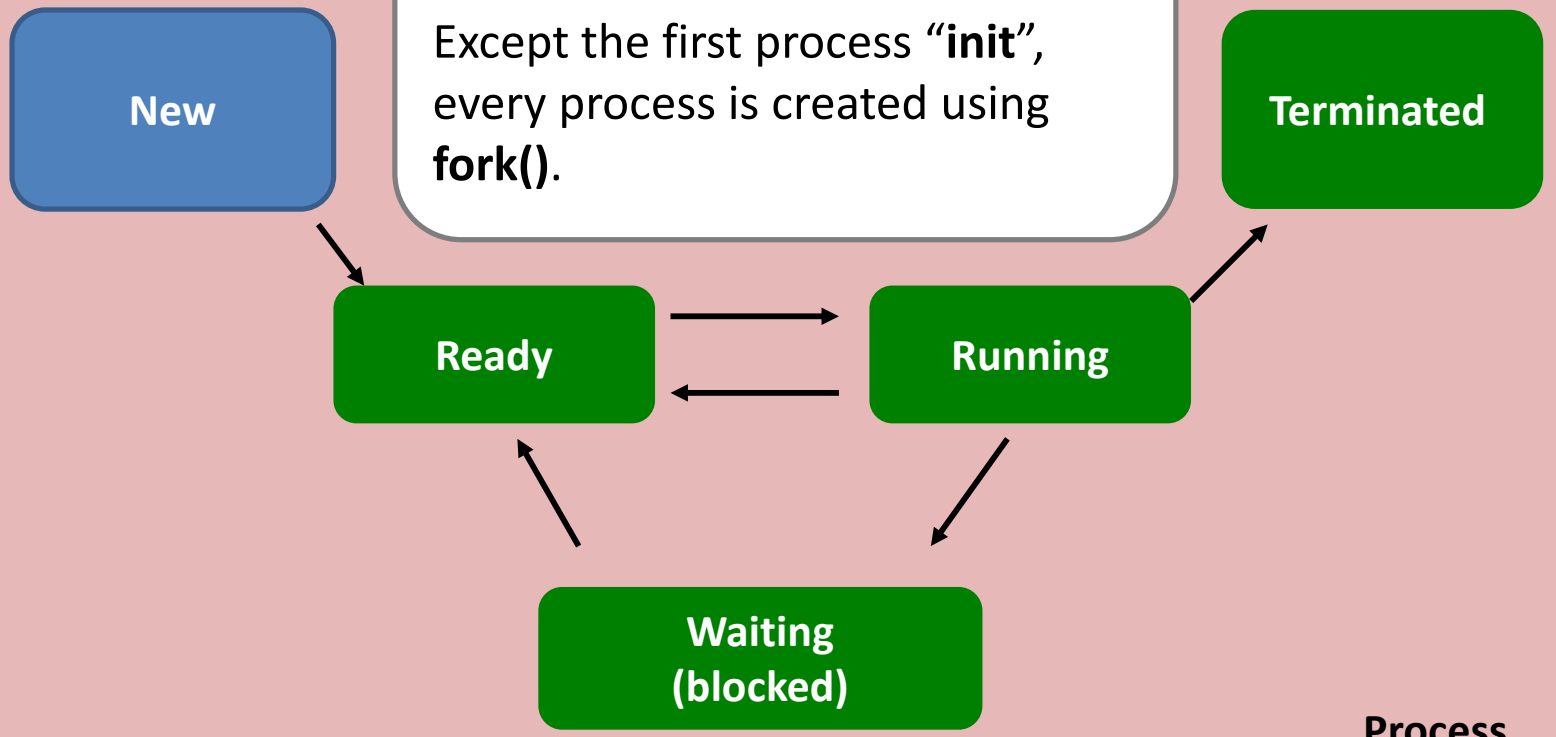
站在内核的角度看进程


Big Picture
New
(Just fork()-ed)
Terminated
(Zombie)
Ready
Running
Waiting
(blocked)
Process
States

站在内核的角度看进程

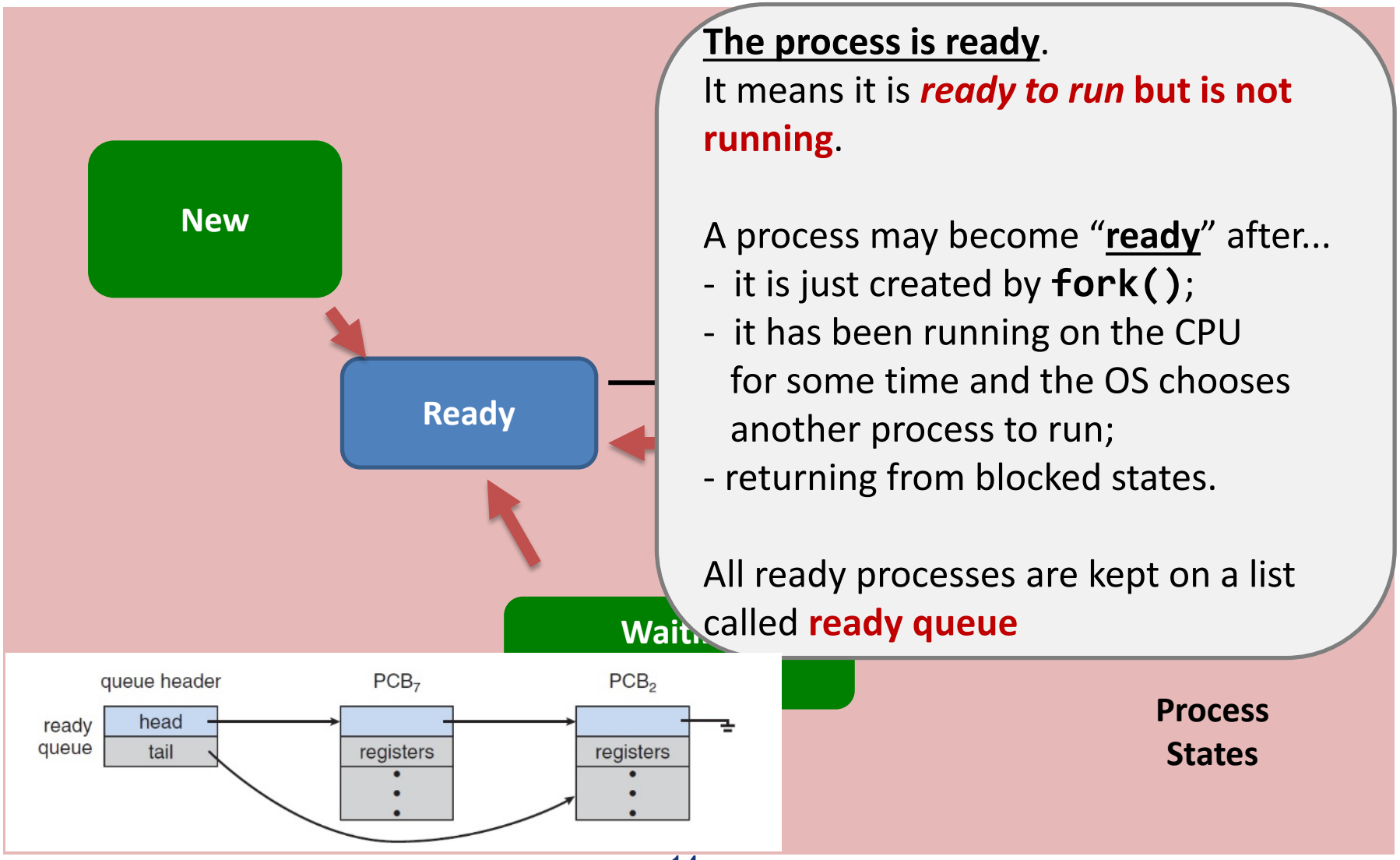
The birth of a process.

Except the first process "**init**", every process is created using **fork()**.



Process States

站在内核的角度看进程

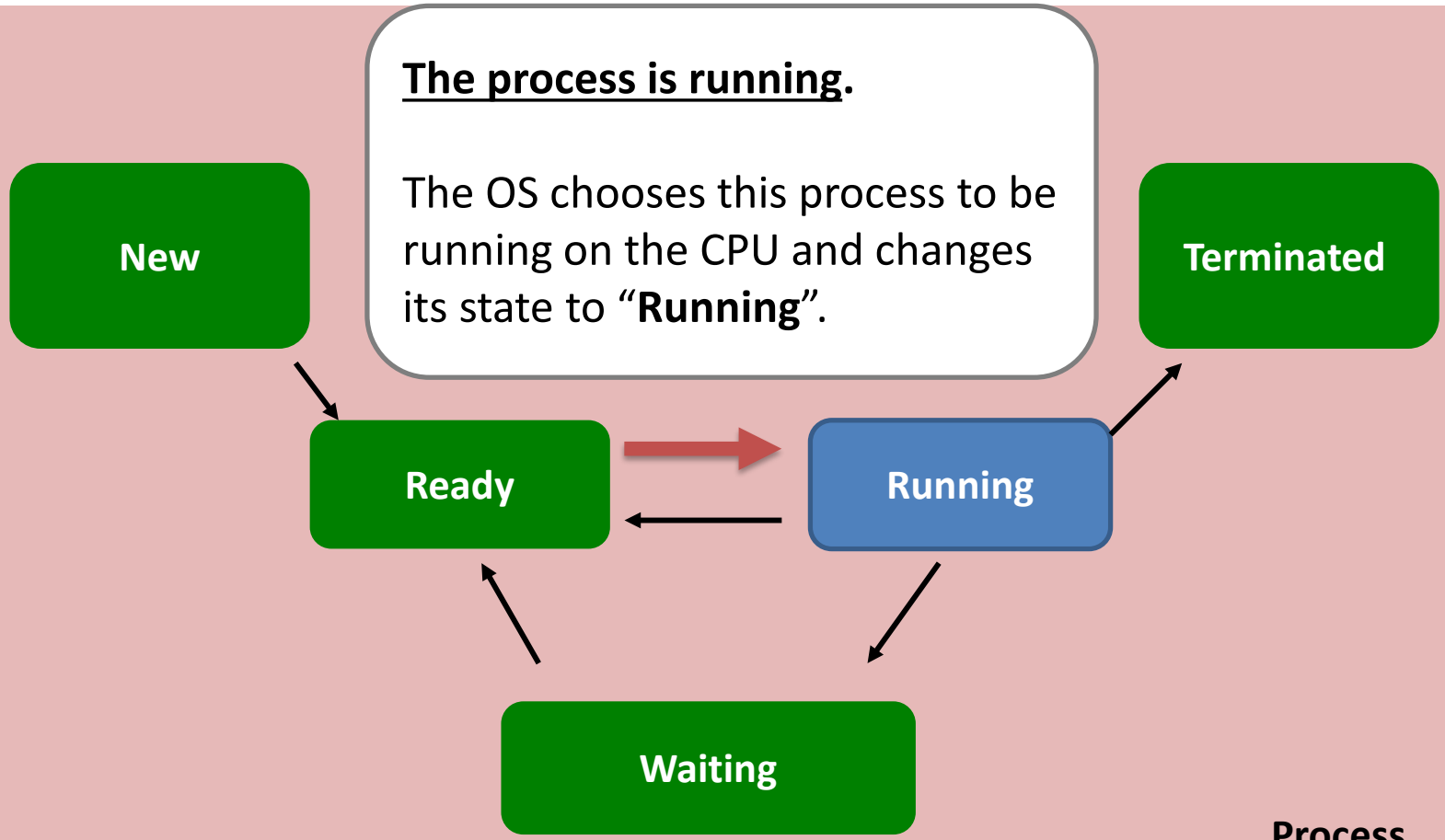


The process is ready.
It means it is ***ready to run*** **but is not running**.

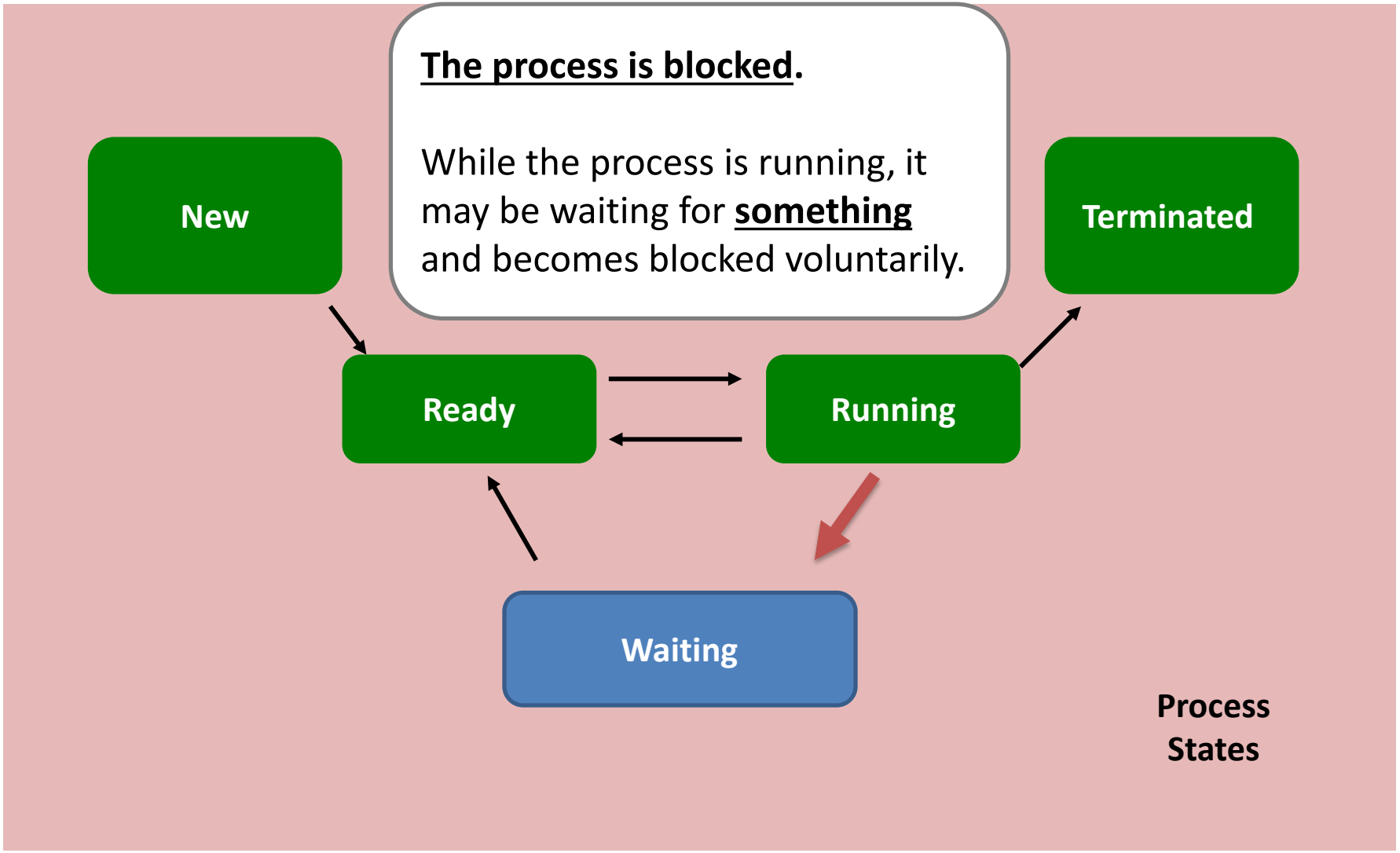
A process may become "**ready**" after...
- it is just created by **`fork()`**;
- it has been running on the CPU for some time and the OS chooses another process to run;
- returning from blocked states.

All ready processes are kept on a list called **ready queue**

站在内核的角度看进程


The process is running.
The OS chooses this process to be running on the CPU and changes its state to "Running".
New
Ready
Running
Terminated
Waiting
Process States

站在内核的角度看进程

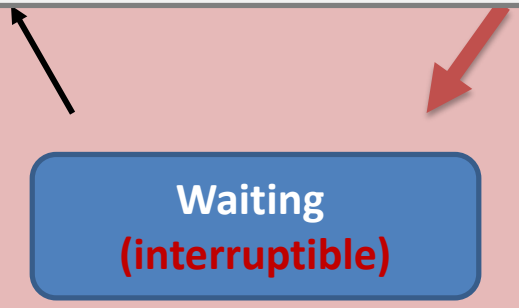

The process is blocked.
While the process is running, it may be waiting for something and becomes blocked voluntarily.
New
Ready
Running
Terminated
Waiting
Process States

站在内核的角度看进程

Example. **Reading a file**.

Sometimes, the process has to wait for the response from the device and, therefore, it is **blocked**.

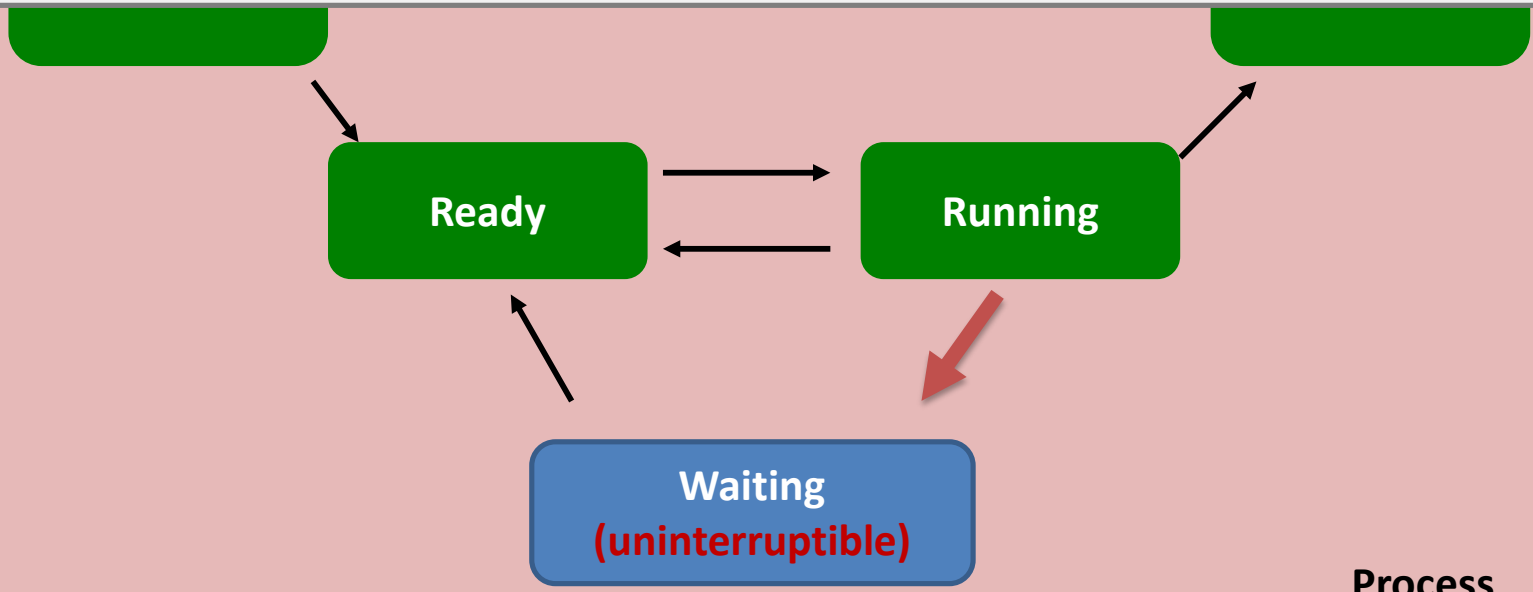
Nevertheless, this blocking state is **interruptible**. E.g., "**Ctrl + C**" can get the process out of the waiting state (but goes to termination state instead).



Process States

站在内核的角度看进程

Sometimes, a process needs to wait for a resource but it doesn't want to be disturbed while it is waiting. In other words, **the process wants that resource very much**. Then, the process status is set to the **uninterruptible** status.

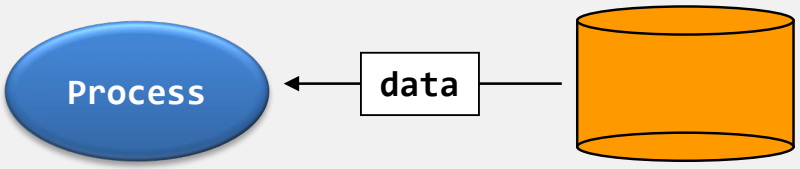


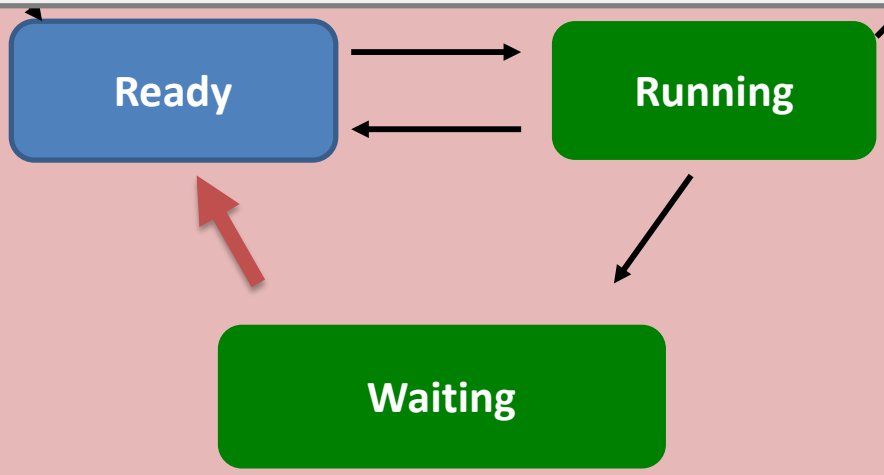
Process States

站在内核的角度看进程

Return back to ready.

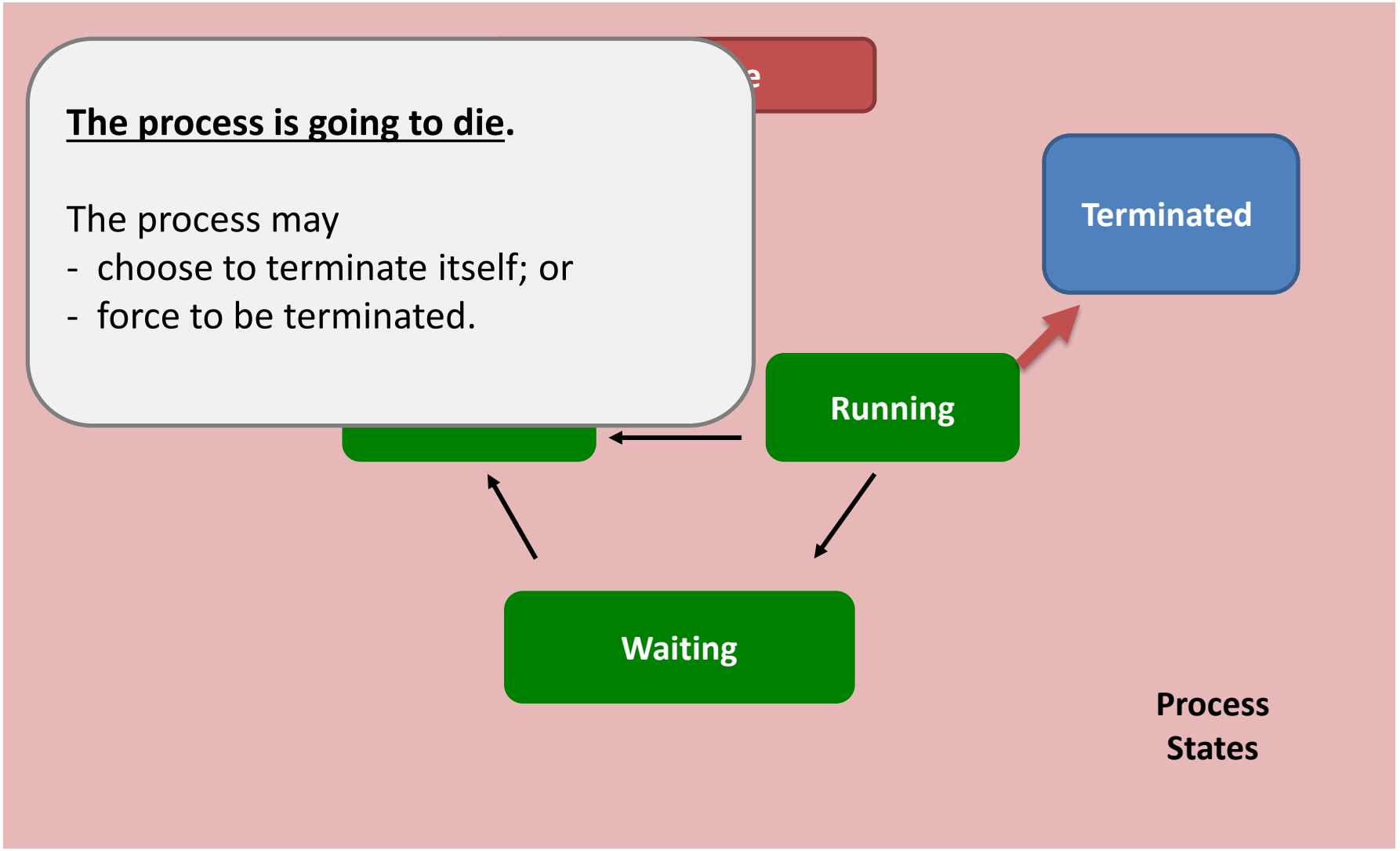
When response arrives, the status of the process changes back to **Ready**. from any one of the blocked states.





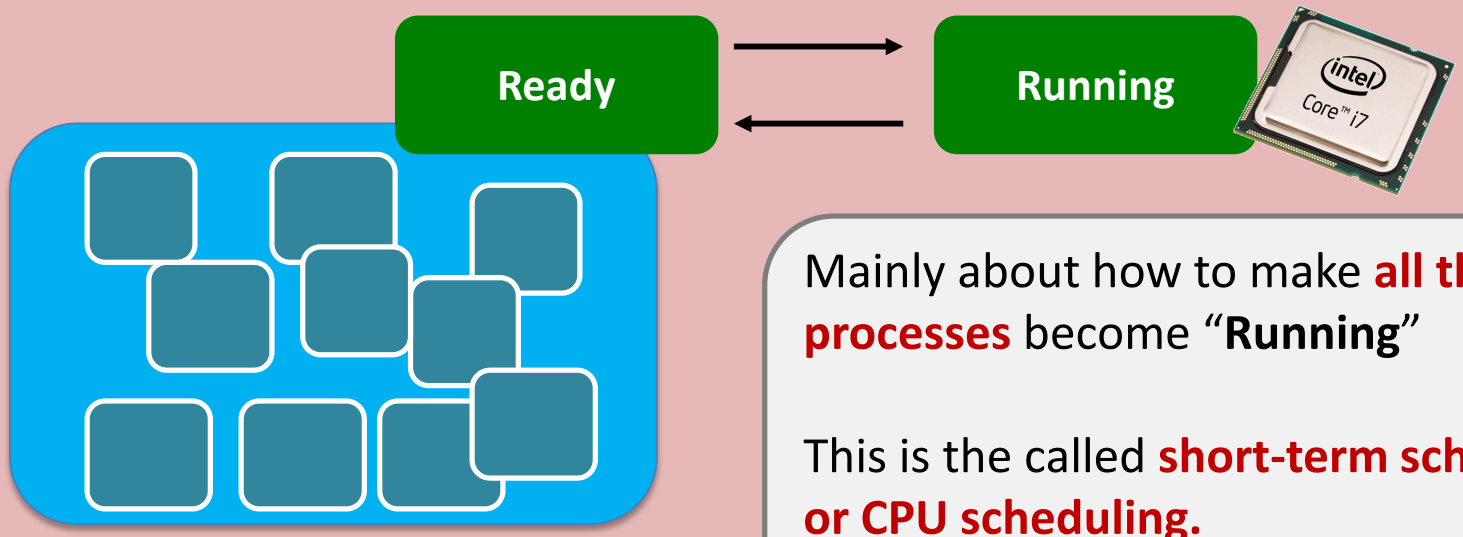
Process States

站在内核的角度看进程


The process is going to die.
The process may
- choose to terminate itself; or
- force to be terminated.
Terminated
Running
Waiting
Process States

站在内核的角度看进程

So, what is process scheduling?



Mainly about how to make **all the ready processes** become "**Running**"

This is the called **short-term scheduling or CPU scheduling.**

调度的类型

当CPU空闲时，操作系统应从就绪队列中选择一个进程来执行。进程选择**短期调度程序**或CPU调度程序。

Long-term scheduling	The decision to add to the pool of processes to be executed
Medium-term scheduling	The decision to add to the number of processes that are partially or fully in main memory
Short-term scheduling	The decision as to which available process will be executed by the processor
I/O scheduling	The decision as to which process's pending I/O request shall be handled by an available I/O device

这些调度的名称表明了执行这些功能的相对时间比例

处理器调度

- 处理器调度的目的是满足系统目标（响应时间、吞吐率、处理器效率）的方式，把进程分配到一个或者多个处理器上执行；
- 分为三个独立的功能：（调度的名字表明了执行这些功能的相对时间）

进程状态

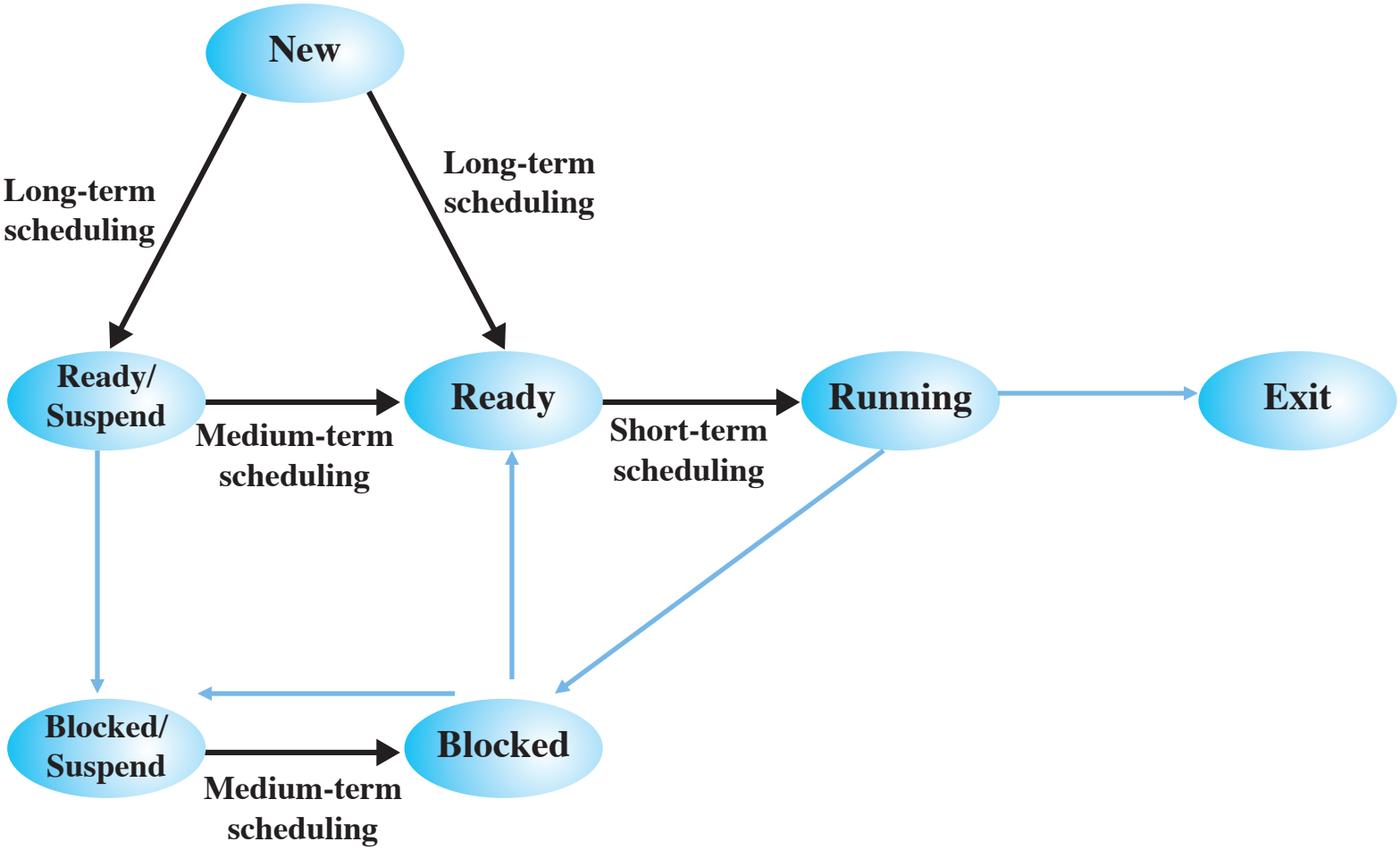


Figure 9.1 Scheduling and Process State Transitions

调度层次

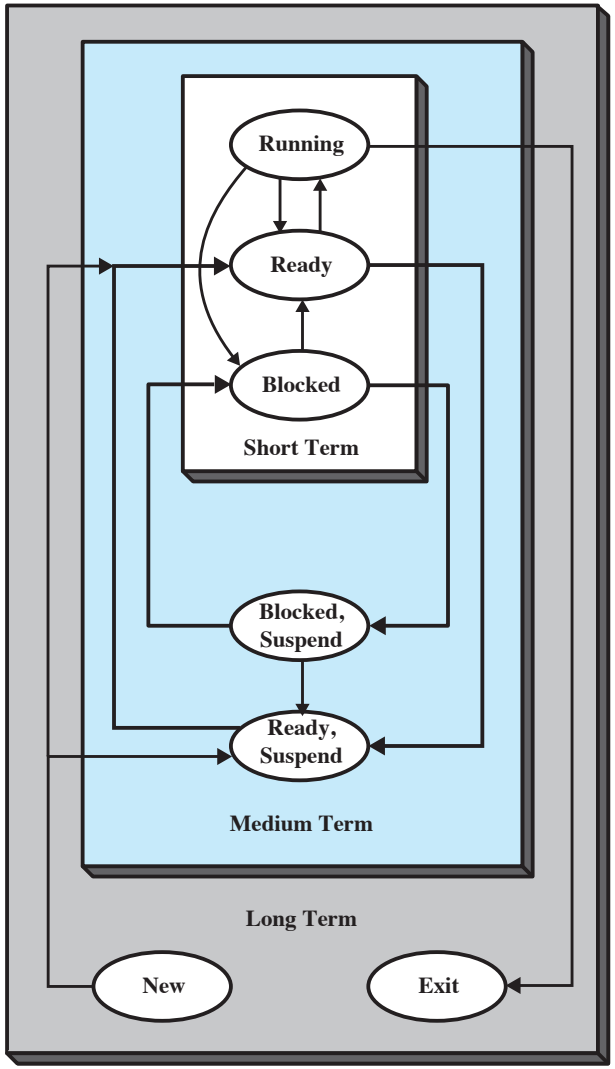


Figure 9.2 Levels of Scheduling

调度队列

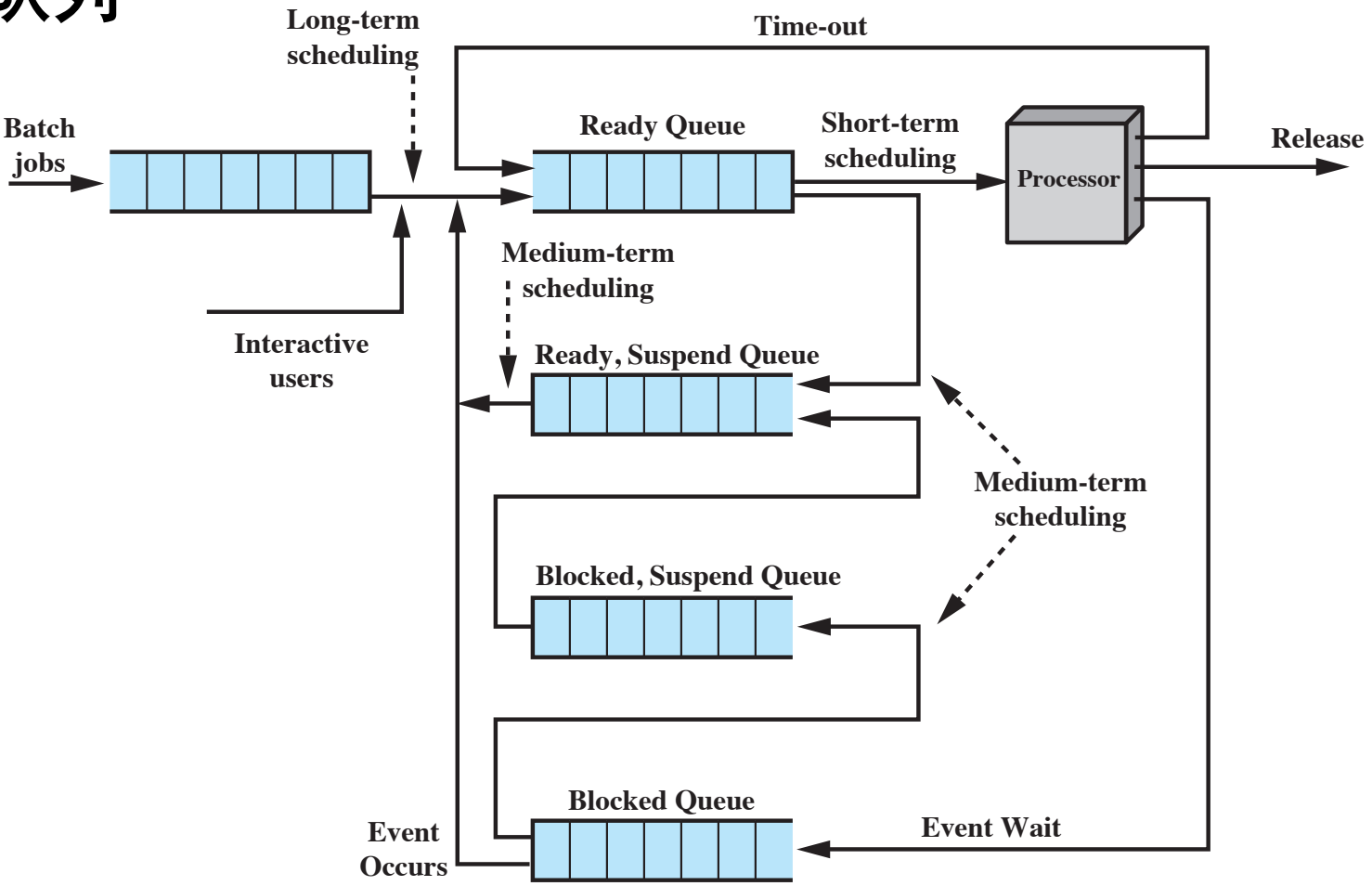
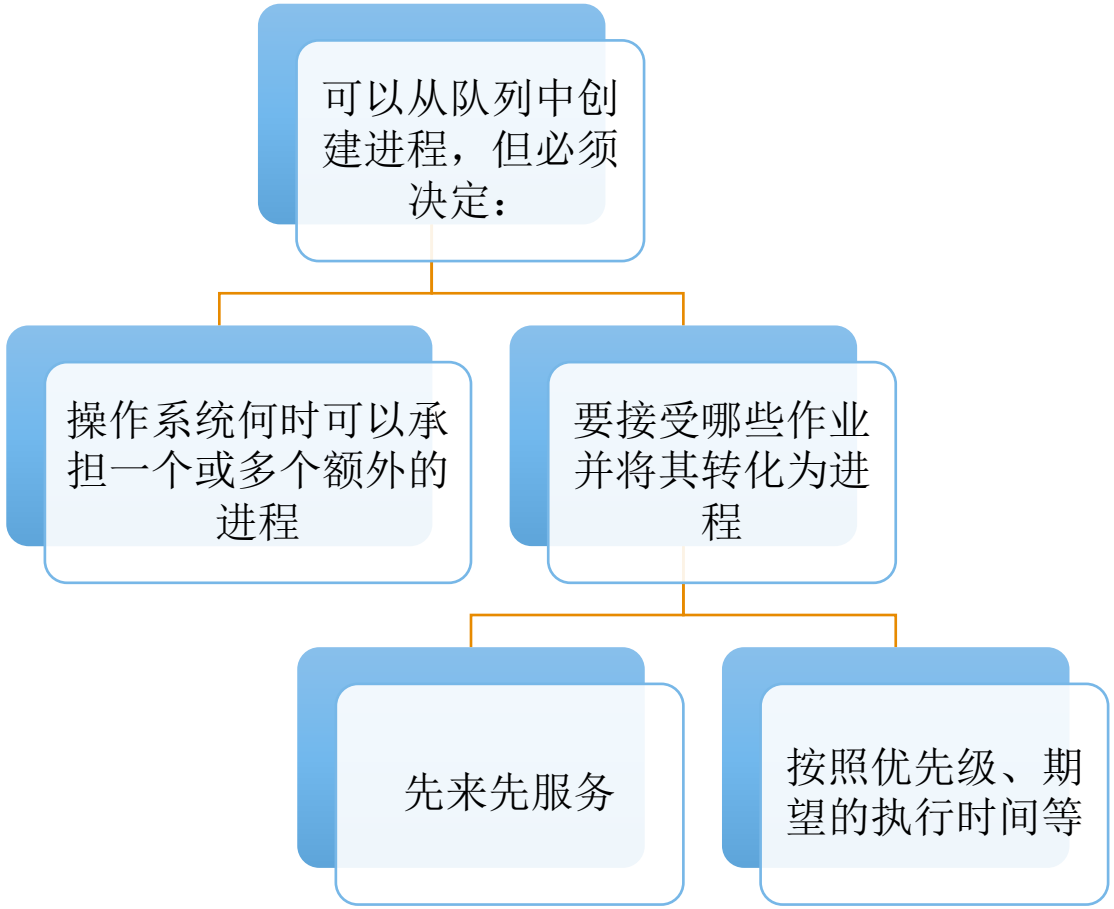


Figure 9.3 Queuing Diagram for Scheduling

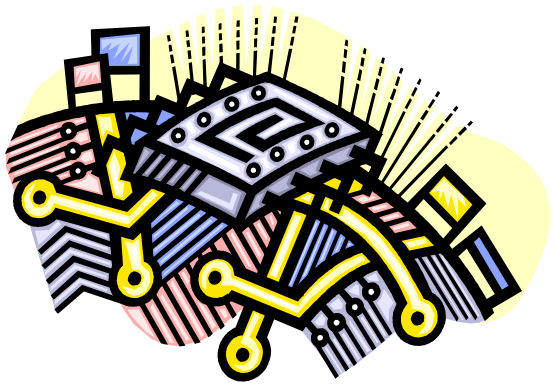
长程调度

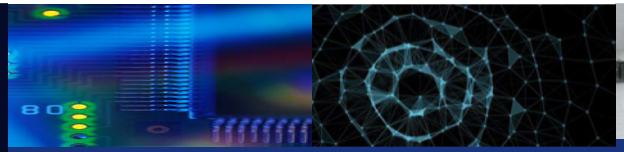
- 确定哪一个程序进入系统中处理；
- 控制了系统的并发度；
  - 创建的进程越多，每个进程的执行时间的百分比越小；
  - 长程调度程序可能会限制系统的并发度，为了给当前的进程提供满意的服务；



中程调度

- 中程调度是交换功能的一部分，
- 典型的情况下，换入（swapping-in)决定取决于管理系统并发度的需求。
  - 存储管理是一个问题，换入决策将考虑换出进程的存储需求；





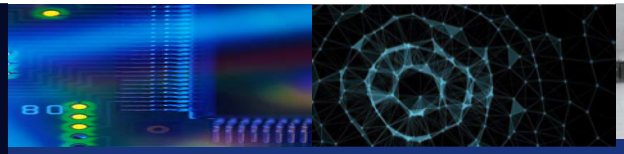
短程调度

- 也称为分派程序（dispatcher）
- 执行频率较高

  根据频繁程度：长程<中程<短程；
- 精确决定下次执行哪一个进程；
- 导致当前进程阻塞或抢占当前运行进程的事件发生时，调用短程调度程序；

事件包括：

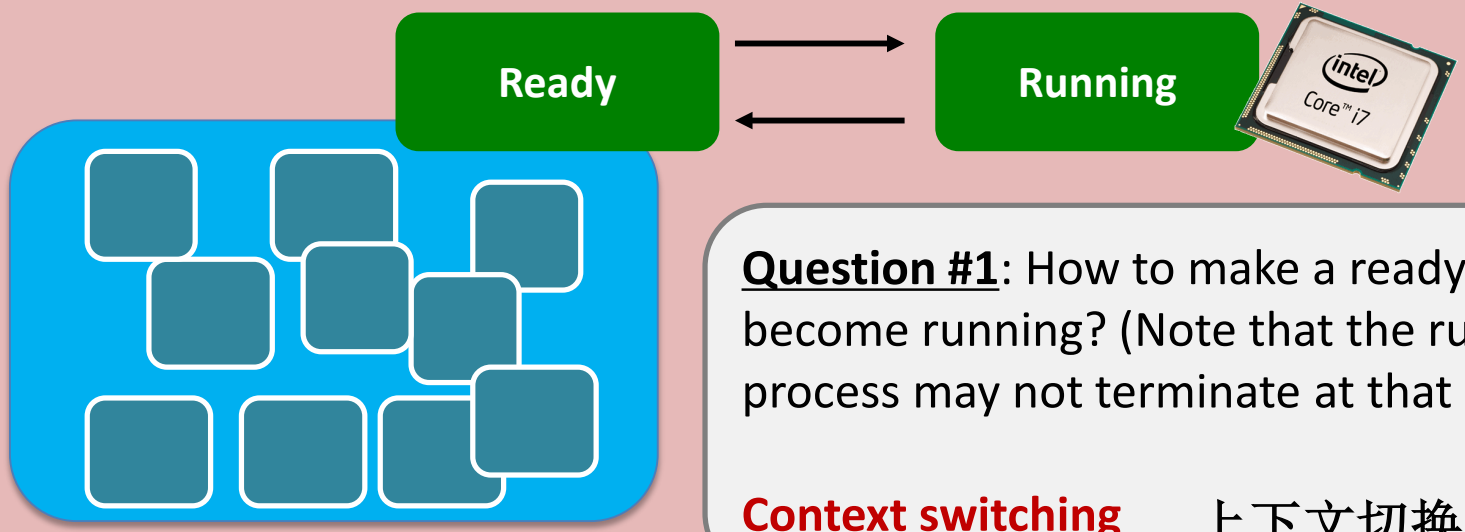
- 时钟中断；
- I/O 中断；
- 操作系统调用
- 信号 (e.g., 信号量)



触发调度的事件

A new process is created.	When "**fork()**" is invoked and returns successfully. Then, whether the parent or the child is scheduled is up to the scheduler's decision.
An existing process is terminated.	**The CPU is freed**. The scheduler should choose another process to run.
A process waits for I/O.	**The CPU is freed**. The scheduler should choose another process to run.
A process finishes waiting for I/O.	The interrupt handling routine **makes a scheduling request**, if necessary.

主要的问题



Question #1: How to make a ready process become running? (Note that the running process may not terminate at that time)

Context switching 上下文切换

Question #2: How to decide which process should be running?

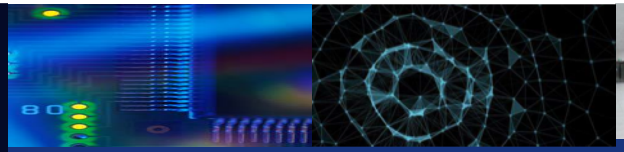
Scheduling criteria & scheduling algorithms 调度标准和算法

Question #3: How to design scheduling in a real/specific system? 实时调度

Multiprocessor system, real-time system, algorithm eveluation

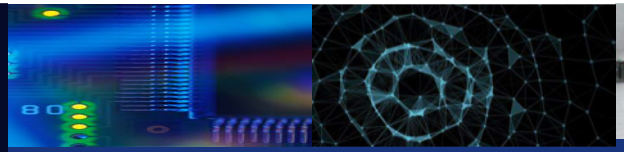
CPU调度程序

- CPU调度器从就绪队列中的进程中进行选择，并将CPU核心分配给其中一个进程
  - 队列可以以各种方式排序
- CPU调度决策可能在以下情况下发生：
  1. 从运行状态切换到等待状态（I/O请求或wait（）调用）
  2. 从运行状态切换到就绪状态（出现中断）
  3. 从等待切换到就绪 （ I/O完成）
  4. 终止
- 对于情况1和4，在调度方面没有选择。必须选择一个新进程（如果就绪队列中存在一个）来执行。
- 然而，对于情况2和3，有其他选择。



抢占式和非抢占式调度

- 当调度仅在情况1和4下发生时，调度方案是非抢占的；
- 否则，是抢占式调度；
- 在非抢占式调度下，一旦CPU分配给进程，进程将保持CPU，直到通过终止或切换到等待状态释放CPU为止；
- 几乎所有现代操作系统，包括Windows、MacOS、Linux和UNIX，都使用抢占式调度算法；

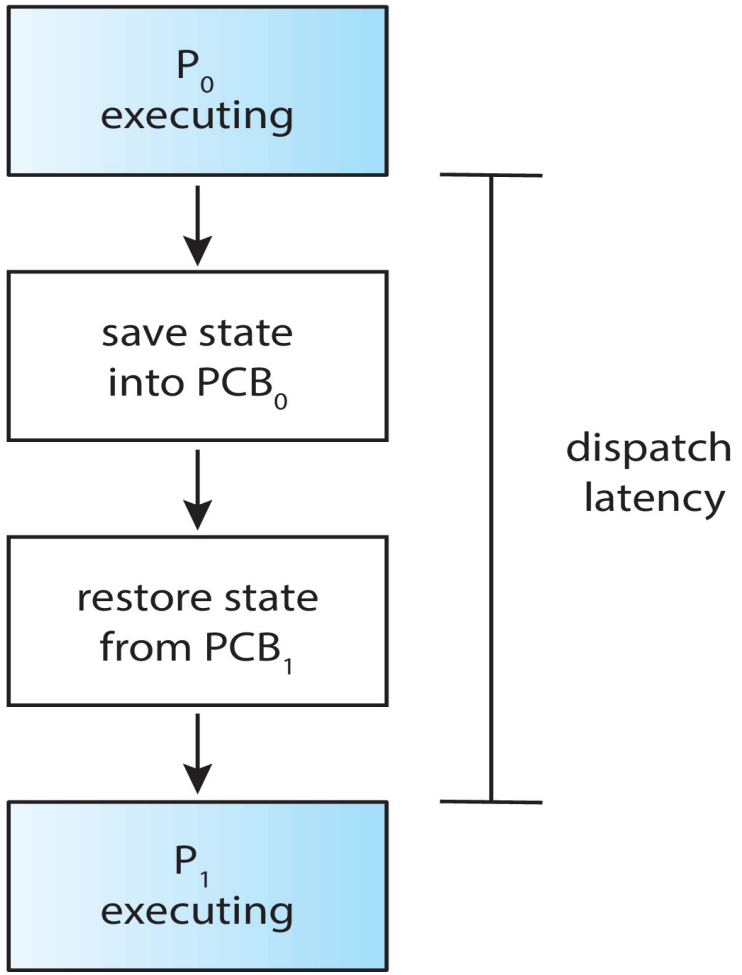


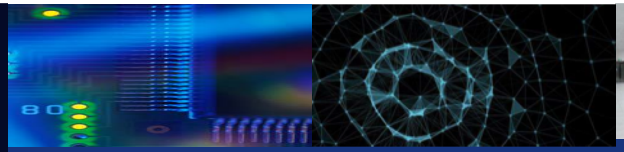
抢占式和非抢占式调度

- 当数据在多个进程之间共享时，抢占式调度可能导致竞争条件；
- 考虑两个共享数据的进程的情况。当一个进程更新数据时，它被抢占，以便第二个进程可以运行。然后，第二个进程可能读取处于不一致状态的数据；
- 第6章将详细探讨这个问题；
- 抢占也影响操作系统内核的设计，在上下文切换之前，等待系统调用的完成，或者等待I/O阻塞的发生。确保内核简单，但是难以应用于实时系统中。

调度程序

- 调度程序模块向CPU调度程序选择的进程，提供CPU控制；这包括：
  - 切换上下文；
  - 切换到用户模式；
  - 跳转到用户程序中的正确位置以重新启动该程序；
- Dispatch latency－调度器停止一个进程并启动另一个运行进程所需的时间





调度准则

- How to choose which algorithm to use in a particular situation?

Types

Preemptive

Nonpreemptive

Application

Multiprocessor

Real-time sys

Algorithm Properties

CPU utilization **Throughput**

Turnaround time **Waiting time**

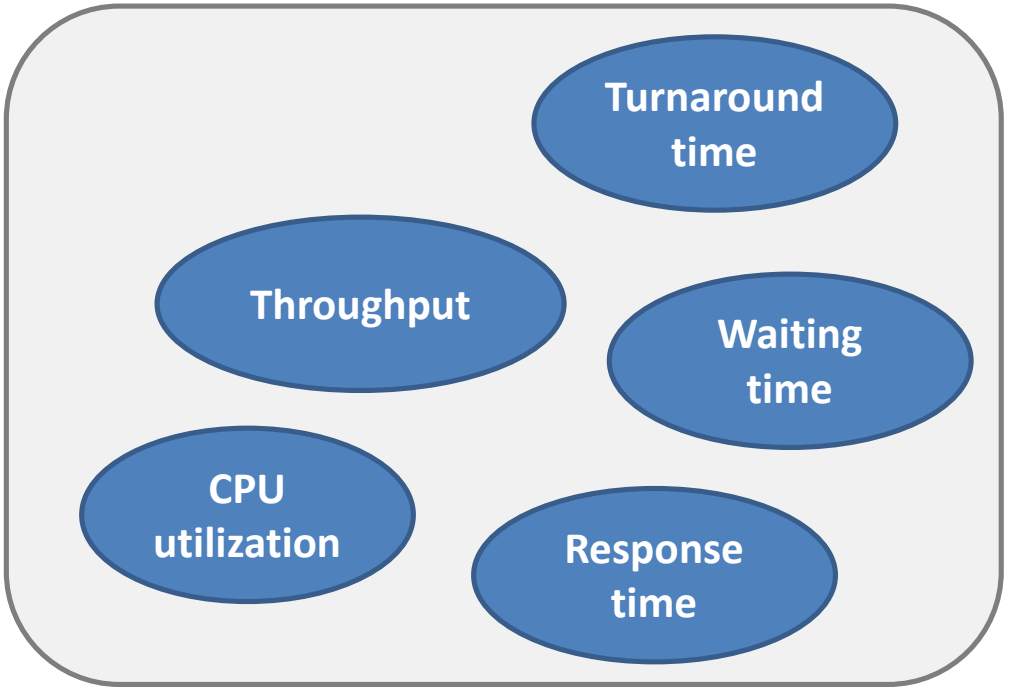
Response time

Application requirements and algorithm properties may vary significantly

调度准则

In algorithm design:

What factors/performance measures should be carefully considered?



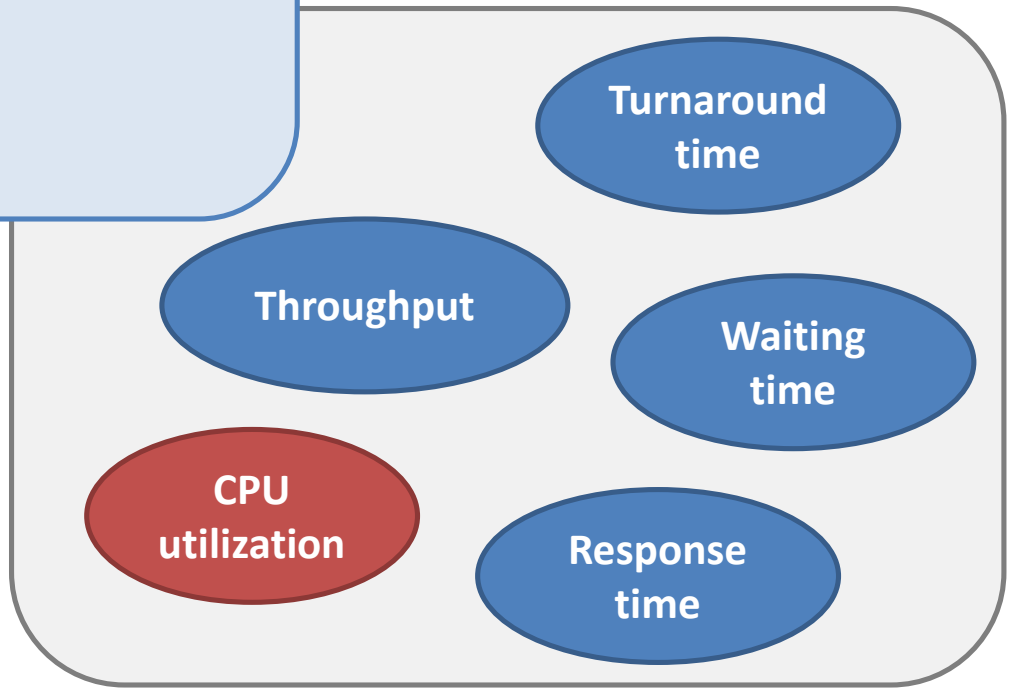
调度准则

CPU utilization. CPU的利用率

We want to keep CPU as busy as possible.

Theoretically, can range from 0-100%, but in real system, range from 40%-90%

The higher the better

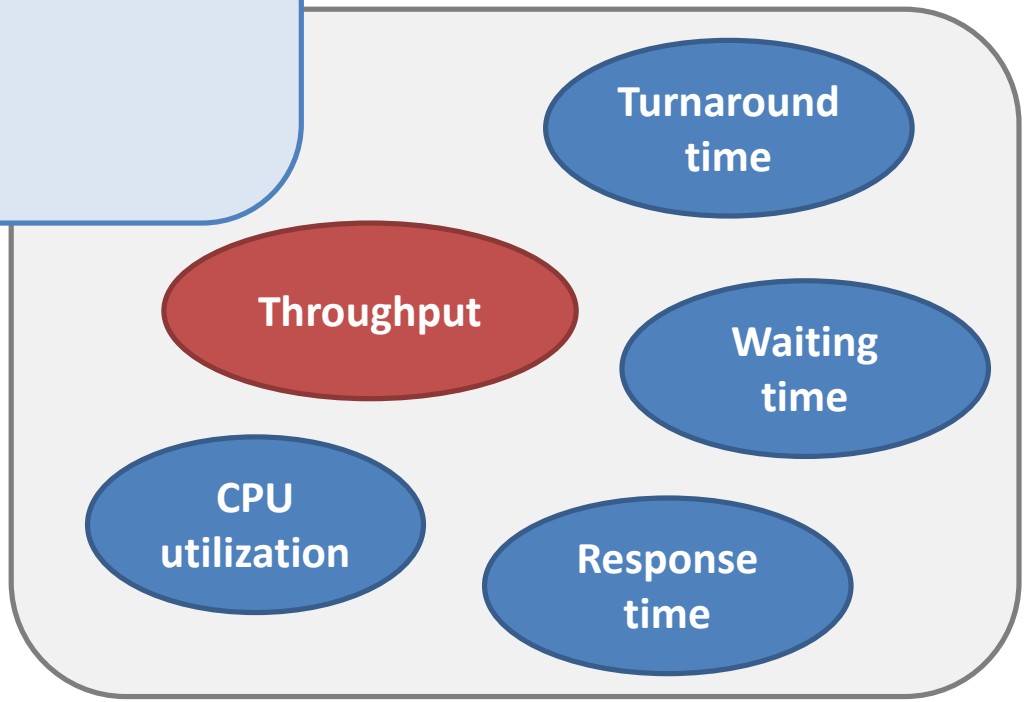


调度准则

Throughput. **吞吐量**

Number of processes that are completed per time unit

The higher the better

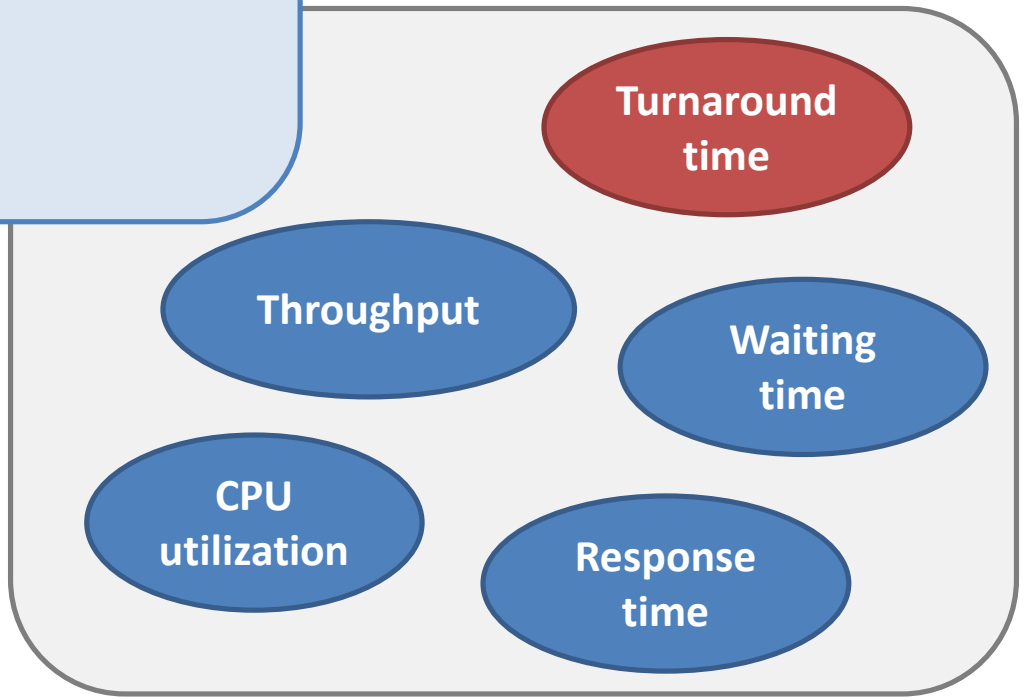


调度准则

Turnaround time. 周转时间

Time to execute the process: interval from the time of submission to the time of completion (total running time + waiting time+ doing I/O)

The lower the better

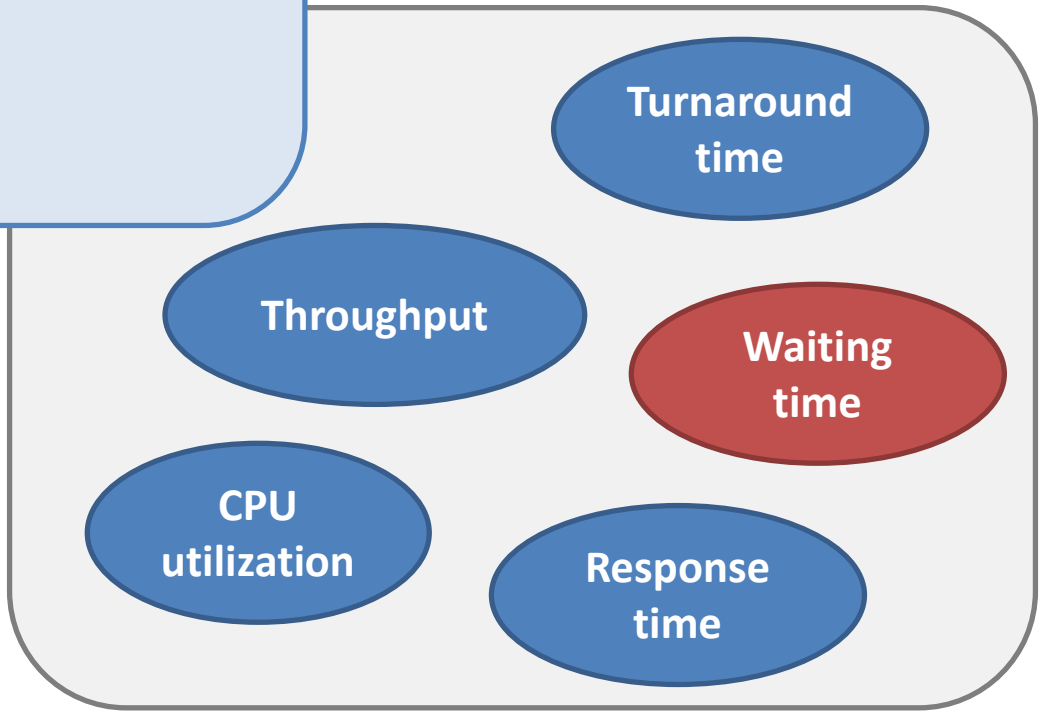


调度准则

Waiting time. 等待时间

The time spent waiting in the ready queue

The lower the better

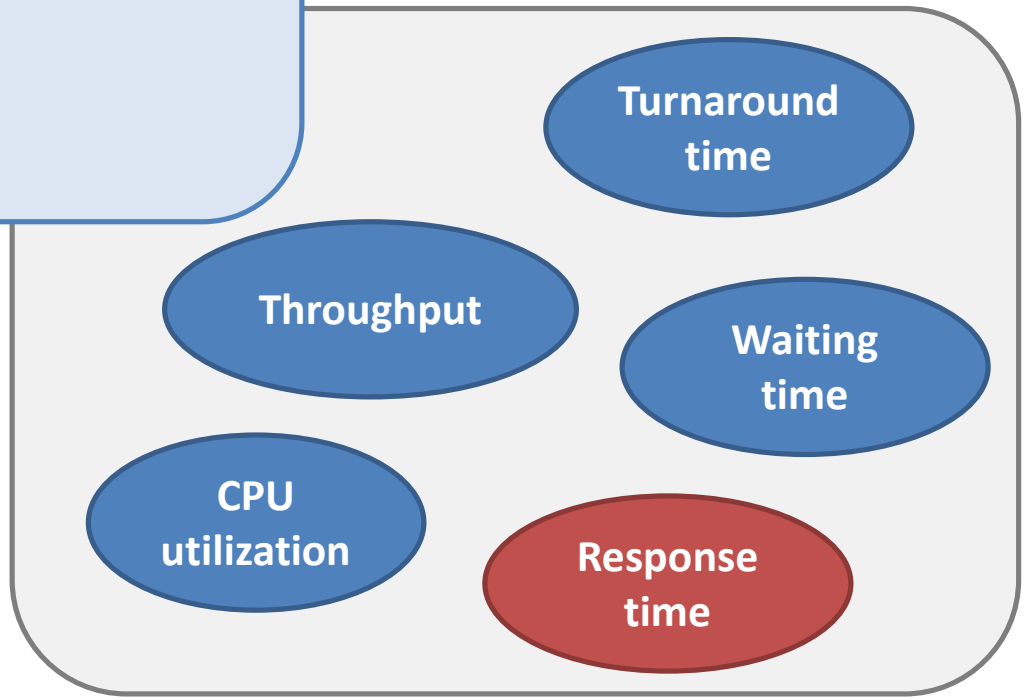


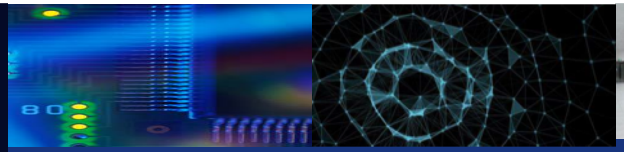
调度准则

Response time. **响应时间**

The time from the submission of a request until the first response is produced (useful measure for interactive systems)

The lower the better





优化原则

- 最大CPU利用率
- 最大吞吐量
- 最小周转时间
- 最小等待时间
- 最小响应时间
- 大多数情况下，优化的是平均值；有些情况下优化的是最小值或最大值，也可以最小化方差；

挑战问题

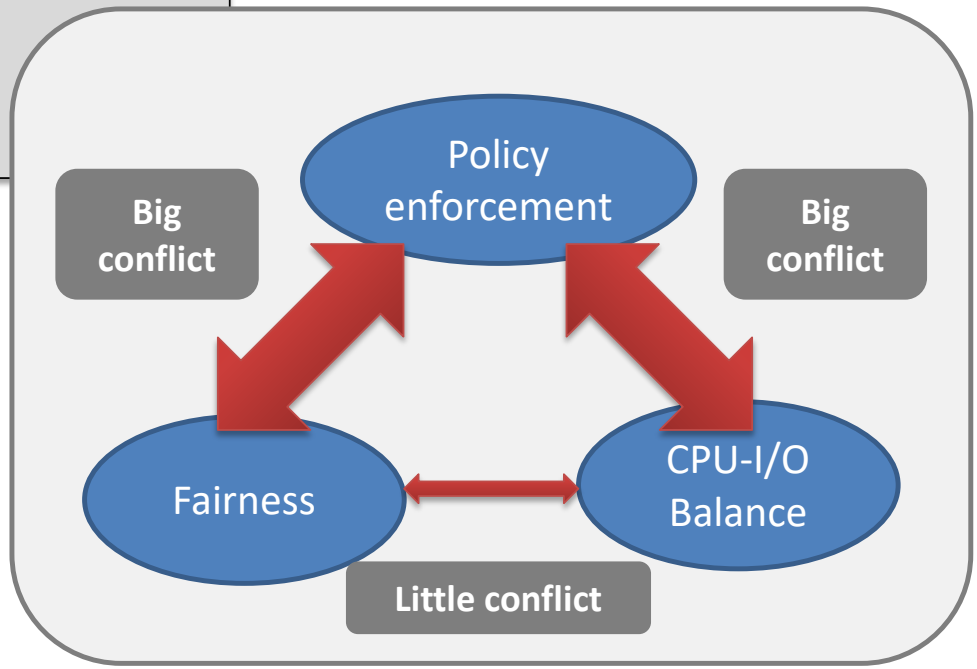
Question:

Can we optimize all the above measures simultaneously?

Usually can not!

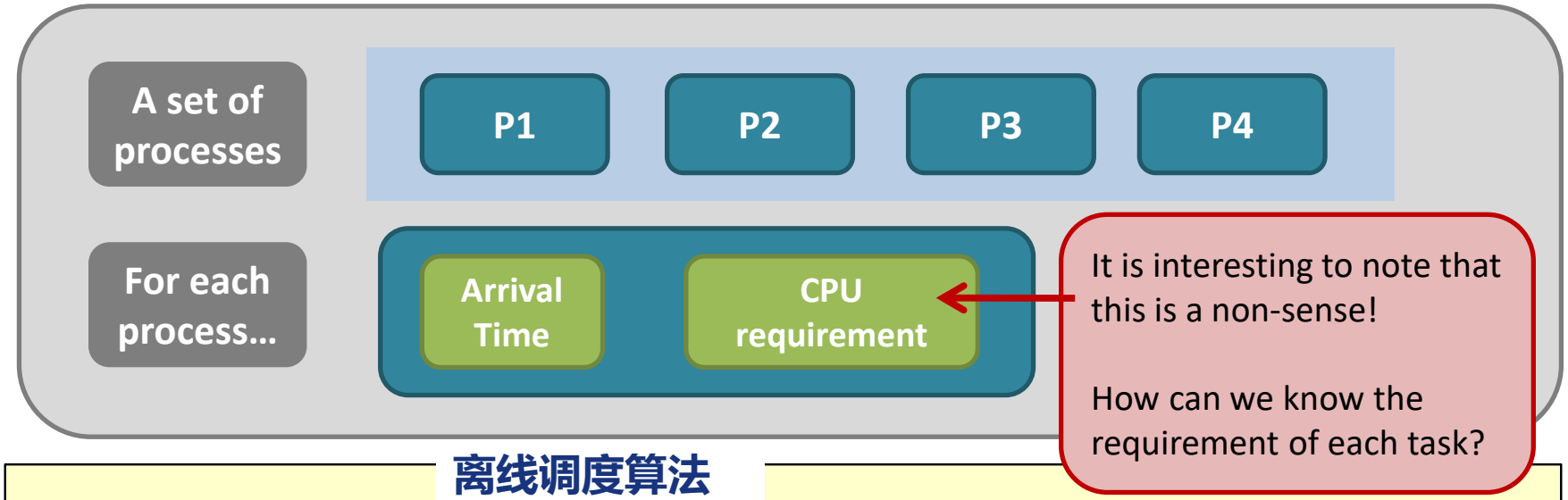
Common goal

Design Tradeoff



调度算法

算法的输入



离线调度算法

Online VS Offline

An **offline scheduling algorithm** assumes that you know all the processes submitted to the system before hand. But, an **online scheduling algorithm** does not have such an assumption.

在线调度算法

Yet, every real scheduler has to work in an "**online scenario**". So, we have to think in an "online" way…

调度算法

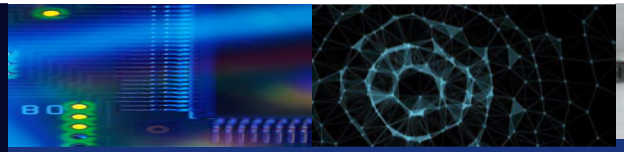
算法的输出

Scheduling order

Individual & average turnaround time

Individual & average waiting time

Number of context switching



调度算法

Algorithms	Preemptive?	Target System
First-come, first-served or First-in, First-out (FIFO)	No.	Out-of-date
Shortest-job-first (SJF)	Can be both.	Out-of-date
Round-robin (RR)	Yes.	Modern
Priority scheduling	Yes.	Modern
Priority scheduling with multiple queues.	The real implementation!	

Demo

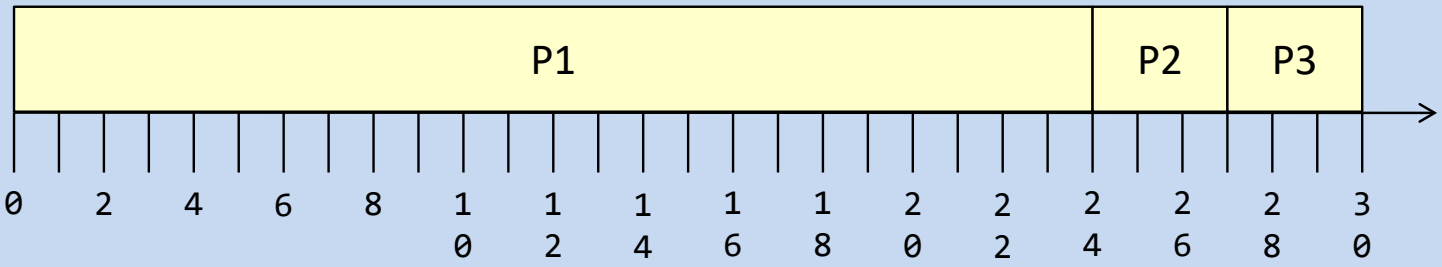
调度算法之FCFS-先到先服务调度

先请求CPU的进程首先分配到CPU，通过FIFO队列容易实现；

Output

Gantt Chart

No preemption



Waiting time: P1 = 0; P2 = 23; P3 = 25;

Average waiting time = (0+23+25)/3 = 16;

Turnaround time: P1 = 24; P2 = 26; P3 = 28;

Average turnaround time = (24+26+28)/3 = 26;

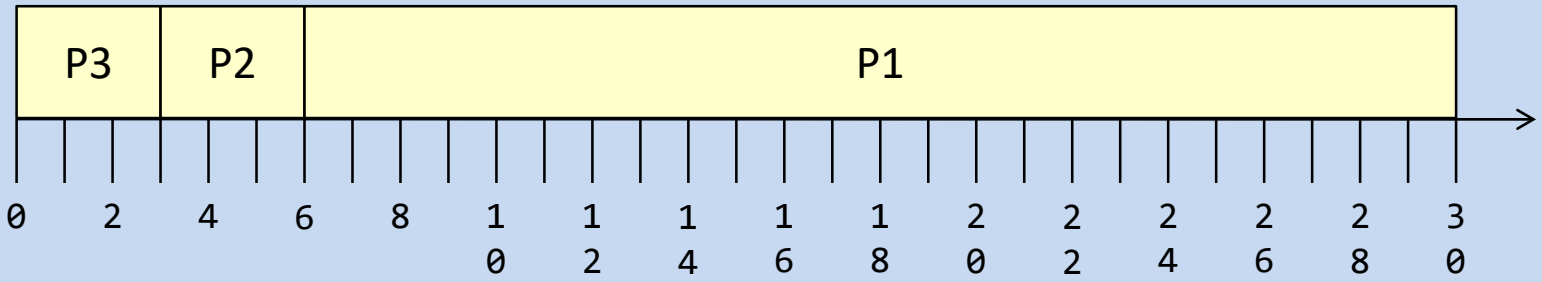
Task	Arrival Time	CPU Req.
P1	0	24
P2	1	3
P3	2	3

Input

调度算法之FCFS-先到先服务调度

Output

Gantt Chart



Waiting time: P1 = 4; P2 = 2; P3 = 0;

Average waiting time = (4+2+0)/3 = 2;
(which is 16 in the previous case)

Turnaround time: P1 = 28; P2 = 5; P3 = 3;

Average turnaround time = (28+5+3)/3 = 12;
(which is 26 in the previous case)

Task	Arrival Time	CPU Req.
P3	0	3
P2	1	3
P1	2	24

Input order changed

调度算法之FCFS-先到先服务调度

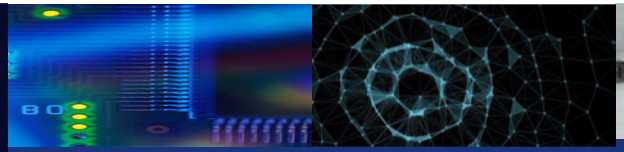
FCFS调度器总结：

—— FCFS是非抢占的，进程得到CPU后会一直占用CPU直到终止或者请求I/O；

—— FCFS调度器对输入敏感,即到达的顺序；

—— FCFS调度器的等待时间过长，特别是在动态情况下FCFS的调度性能较差(出现护航效应)；

—— 是否可以做一些改变？

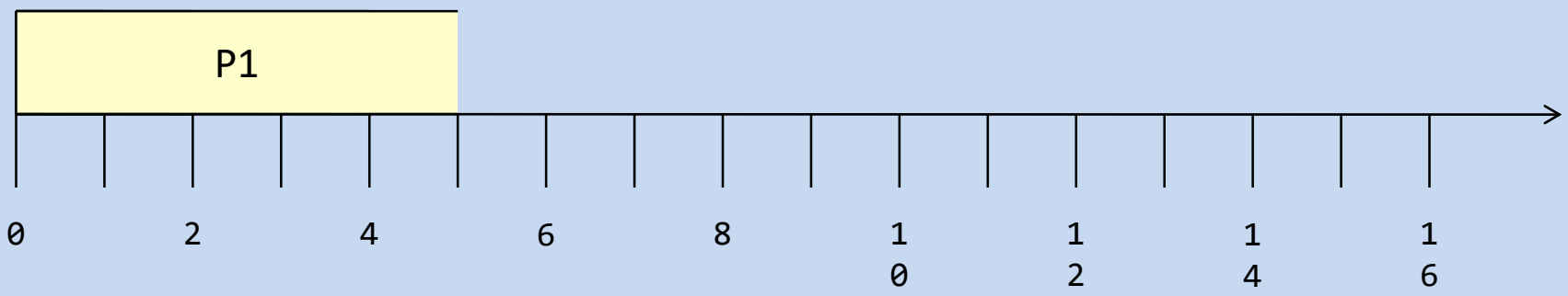


调度算法之SJF-最短作业优先调度

Algorithms	Preemptive?	Target System
First-come, first-served or First-in, First-out (FIFO)	No.	Out-of-date
Shortest-job-first (SJF)	Can be both.	Out-of-date
Round-robin (RR)	Yes.	Modern
Priority scheduling	Yes.	Modern
Priority scheduling with multiple queues.	The real implementation!	

非抢占的SJF

当一个作业的优先级比当前进程的优先级高时，不抢占当前进程，而是允许其继续执行



Time = 5

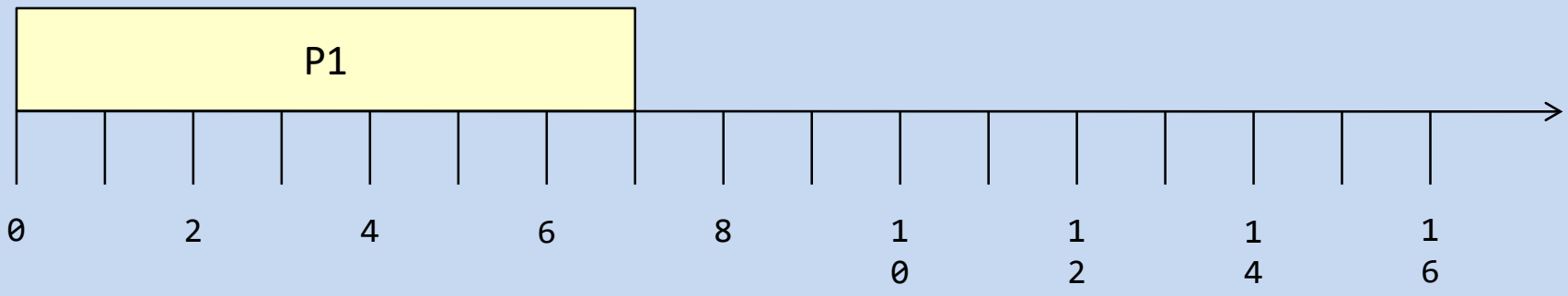
Not allow preemption

Set of processes

P1 P2 P3 P4

Task	Arrival Time	CPU Req.
P1	0	7
P2	2	4
P3	4	1
P4	5	4

非抢占的SJF



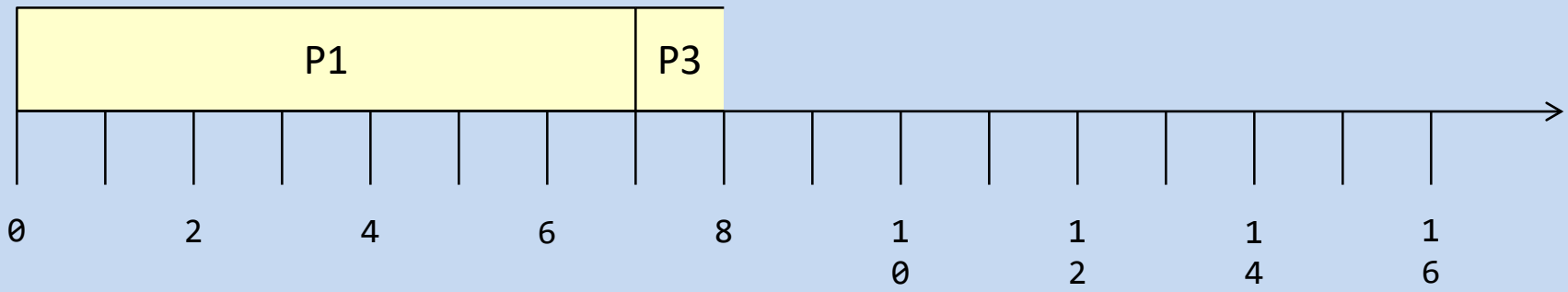
Time = 7

Set of processes

P1 P2 P3 P4

Task	Arrival Time	CPU Req.
P1	0	7
P2	2	4
P3	4	1
P4	5	4

非抢占的SJF



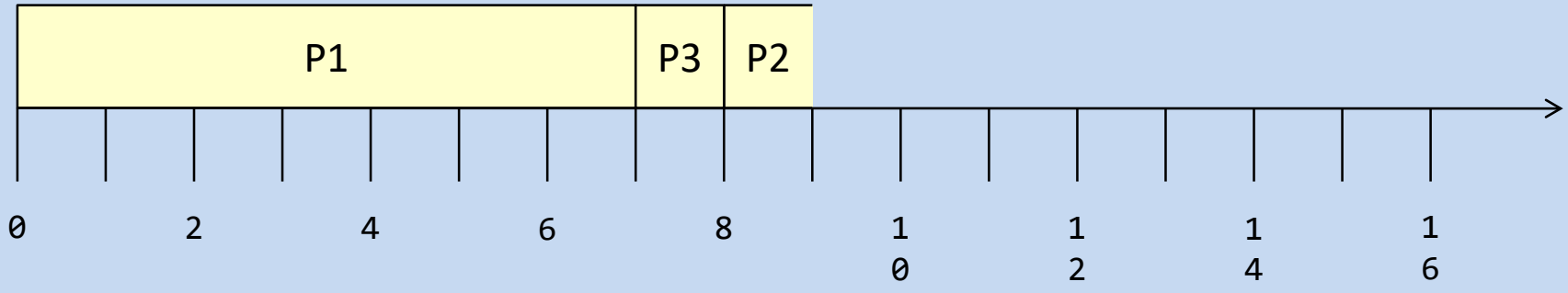
Time = 7

Set of processes

Task	Arrival Time	CPU Req.
P1	0	7
P2	2	4
P3	4	1
P4	5	4

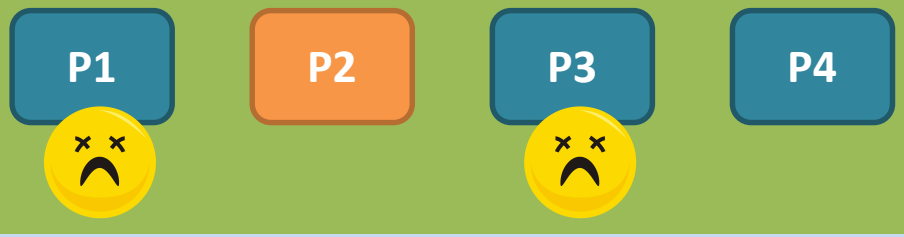
非抢占的SJF

In this example, we use **FIFO** to break the tie.



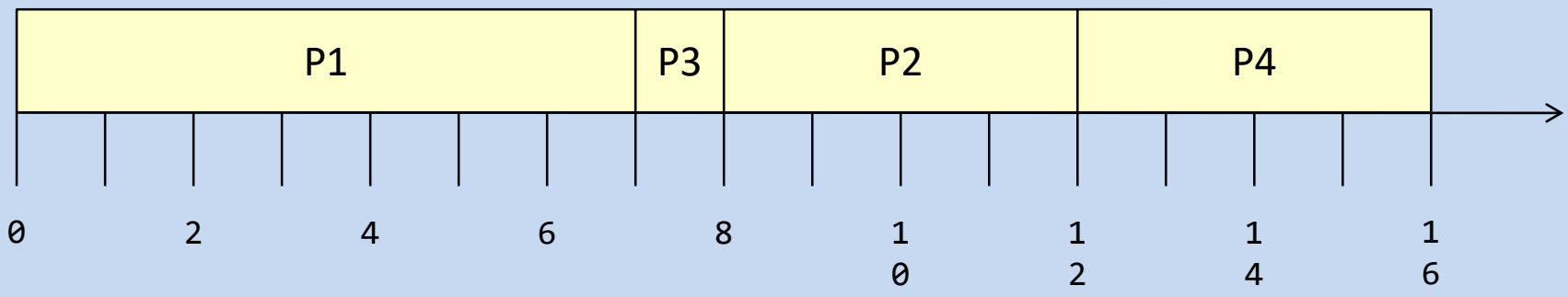
Time = 8

Set of processes



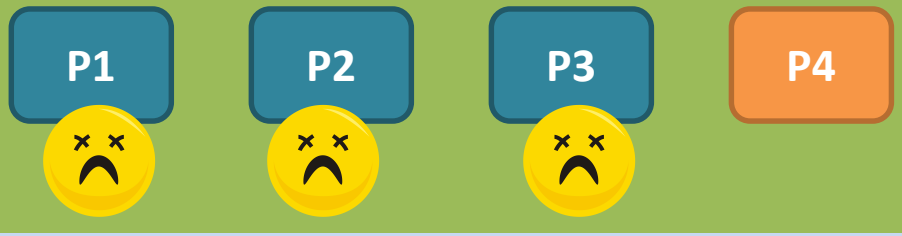
Task	Arrival Time	CPU Req.
P1	0	7
P2	2	4
P3	4	1
P4	5	4

非抢占的SJF



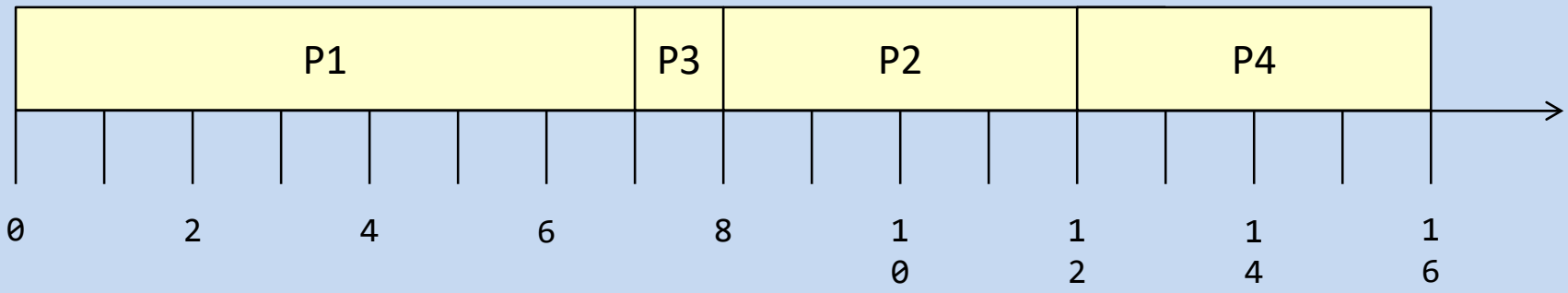
Time = 16

Set of processes



Task	Arrival Time	CPU Req.
P1	0	7
P2	2	4
P3	4	1
P4	5	4

非抢占的SJF



Waiting time:

P1 = 0; P2 = 6; P3 = 3; P4 = 7;

Average = (0 + 6 + 3 + 7) / 4 = 4.

Turnaround time:

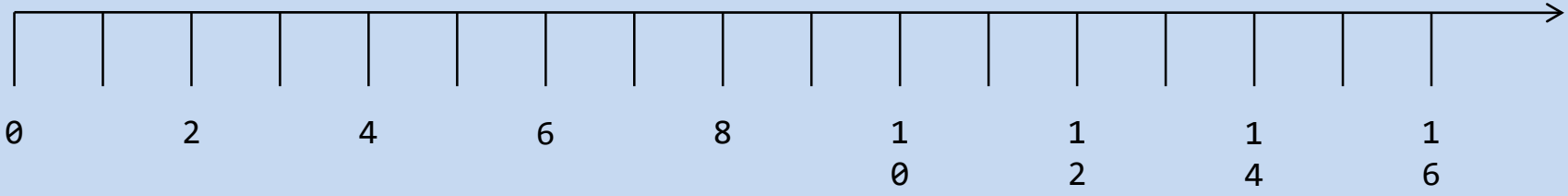
P1 = 7; P2 = 10; P3 = 4; P4 = 11;

Average = (7 + 10 + 4 + 11) / 4 = 8.

Task	Arrival Time	CPU Req.
P1	0	7
P2	2	4
P3	4	1
P4	5	4

抢占的SJF

当一个作业的优先级比当前进程的优先级高时，抢占当前进程，



Rules for preemptive scheduling
(for this example only)

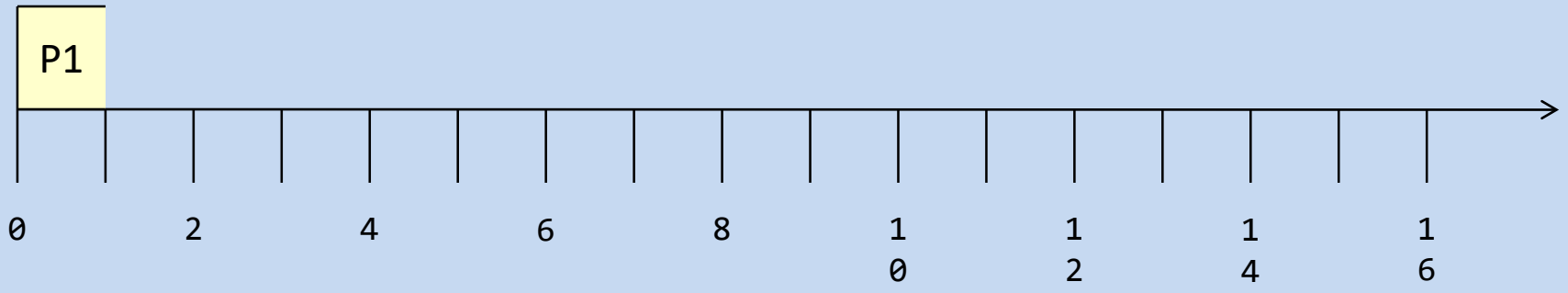
-Preemption happens when a new process arrives at the system.

-Then, the scheduler steps in and selects the next task based on **their remaining CPU requirements**.

Shortest-remaining-time-first

Task	Arrival Time	CPU Req. Initial & Remain	
P1	0	7	7
P2	2	4	4
P3	4	1	1
P4	5	4	4

抢占的SJF



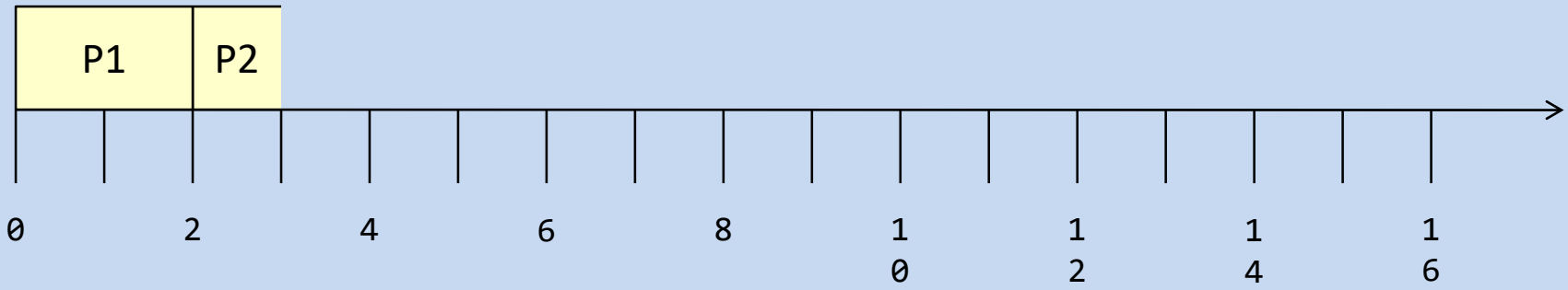
Time = 0

Set of processes

P1

Task	Arrival Time	CPU Req. Initial & Remain	
P1	0	7	7
P2	2	4	4
P3	4	1	1
P4	5	4	4

抢占的SJF



Time = 2

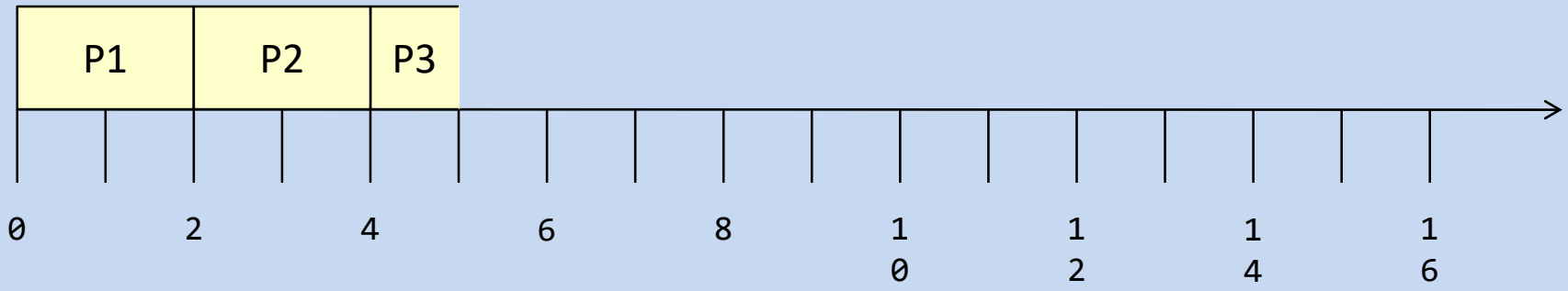
P2 is selected!

Set of processes

P1 P2

Task	Arrival Time	CPU Req. Initial & Remain	
P1	0	7	5
P2	2	4	4
P3	4	1	1
P4	5	4	4

抢占的SJF



Time = 4

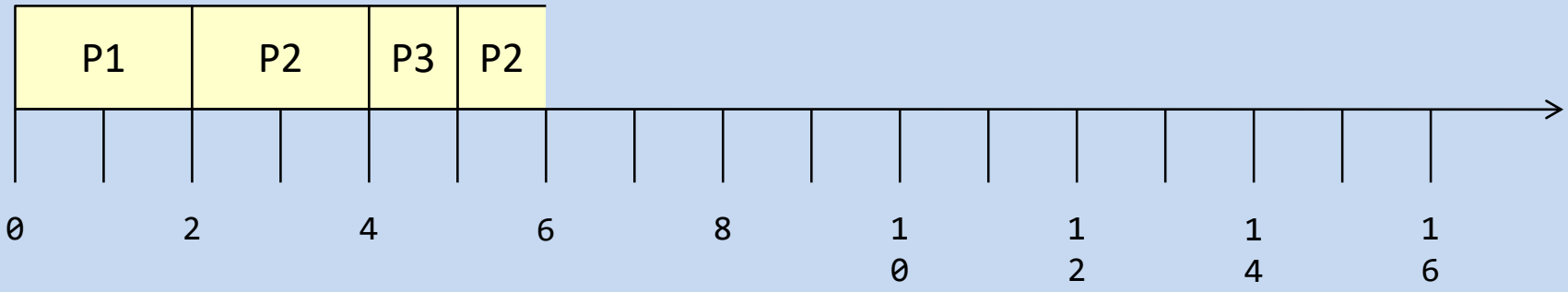
P3 is selected!

Set of processes

P1 P2 P3

Task	Arrival Time	CPU Req. Initial & Remain	
P1	0	7	5
P2	2	4	2
P3	4	1	1
P4	5	4	4

抢占的SJF



Time = 5

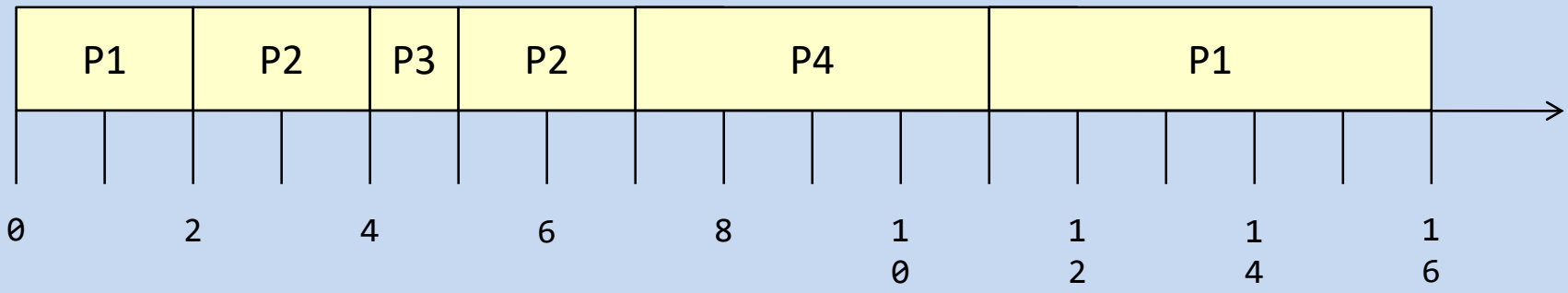
P2 is selected!

Set of processes

P1 P2 P3 P4

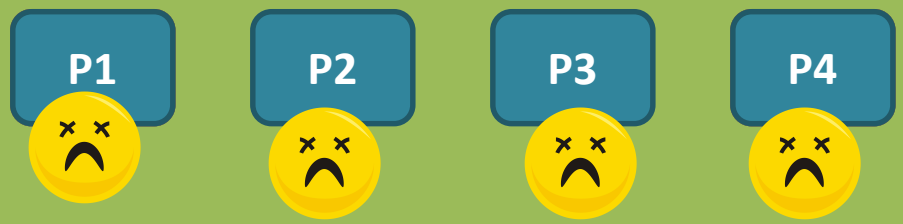
Task	Arrival Time	CPU Req. Initial & Remain	
P1	0	7	5
P2	2	4	2
P3	4	1	0
P4	5	4	4

抢占的SJF



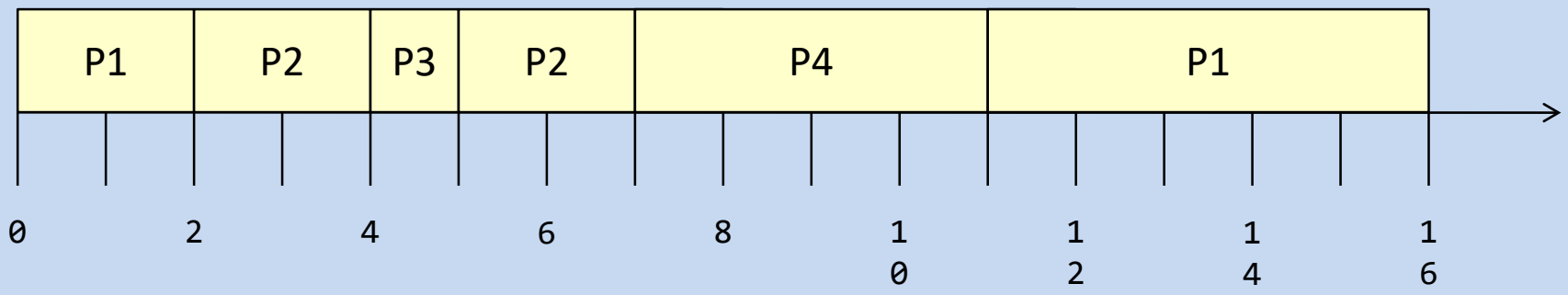
Time = 16

Set of processes



Task	Arrival Time	CPU Req. Initial & Remain	
P1	0	7	0
P2	2	4	0
P3	4	1	0
P4	5	4	0

抢占的SJF



Waiting time:

P1 = 9; P2 = 1; P3 = 0; P4 = 2;

Average = (9 + 1 + 0 + 2) / 4 = 3.

Turnaround time:

P1 = 16; P2 = 5; P3 = 1; P4 = 6;

Average = (16 + 5 + 1 + 6) / 4 = 7.

Task	Arrival Time	CPU Req. Initial & Remain	
P1	0	7	0
P2	2	4	0
P3	4	1	0
P4	5	4	0

调度算法之SJF-最短作业优先调度

	Non-preemptive SJF	Preemptive SJF
Average waiting time	4	**3 (smallest)**
Average turnaround time	8	**7 (smallest)**
# of context switching	**3 (smallest)**	5

平均等待时间和平均周转时间降低，但是代价是增加了上下文切换的成本；

Task	Arrival Time	CPU Req.
P1	0	7
P2	2	4
P3	4	1
P4	5	4

调度算法之SJF-最短作业优先调度

	Non-preemptive SJF	Preemptive SJF
Average waiting time	4	3 (smallest)
Average turnaround time	8	7 (smallest)
# of context switching	3 (smallest)	5

可以证明SJF调度算法是最优的，因为SJF的平均等待时间最小；但是，如何知道下次CPU执行的长度？

Task	Arrival Time	CPU Req.
P1	0	7
P2	2	4
P3	4	1
P4	5	4

调度算法之SJF-最短作业优先调度

Challenge: How to know the length of the next CPU request?

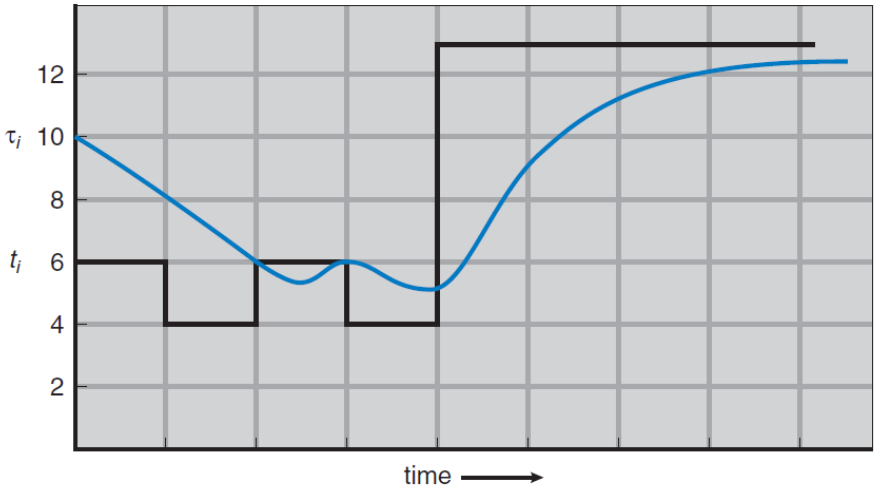
Solution: Prediction (by expecting that the next CPU burst will be similar in length to the previous ones)

General approach
exponential average

$$\tau_{n+1} = \alpha\, t_n + (1 - \alpha)\tau_n.$$

Predicted value

Most recent information



CPU burst (t_i)	6	4	6	4	13	13	13	...
"guess" (τ_i)	10	8	6	6	5	9	11	12	...

调度算法之RR-轮转调度

- RR专门为分时系统设计，类似FCFS但是增加了抢占以切换进程。
- RR是抢占式调度的；
- 每个进程给定一个较小的时间单位称为时间片，时间片用完后，CPU选择另外一个进程调度执行；
- CPU调度程序循环整个就绪队列，一个一个执行；

调度算法之RR-轮转调度

- 每个进程获得一个小的CPU时间单位（时间量和时间片），通常为10-100毫秒。此时间过后，进程将被抢占并添加到就绪队列的末尾。
- 如果就绪队列中有n个进程，且时间片为q，则每个进程一次最多以q个时间单位的块获取1/n的CPU时间。没有进程等待超过（n-1）个时间单位。
- 计时器中断每个时间片以安排下一个进程
- 性能
  - q如果非常大， 变成先进先出（FCFS）；
  - q如果非常小，导致大量的上下文切换，相对于上下文切换，q必须很大，否则开销太高

调度算法之RR–轮转调度

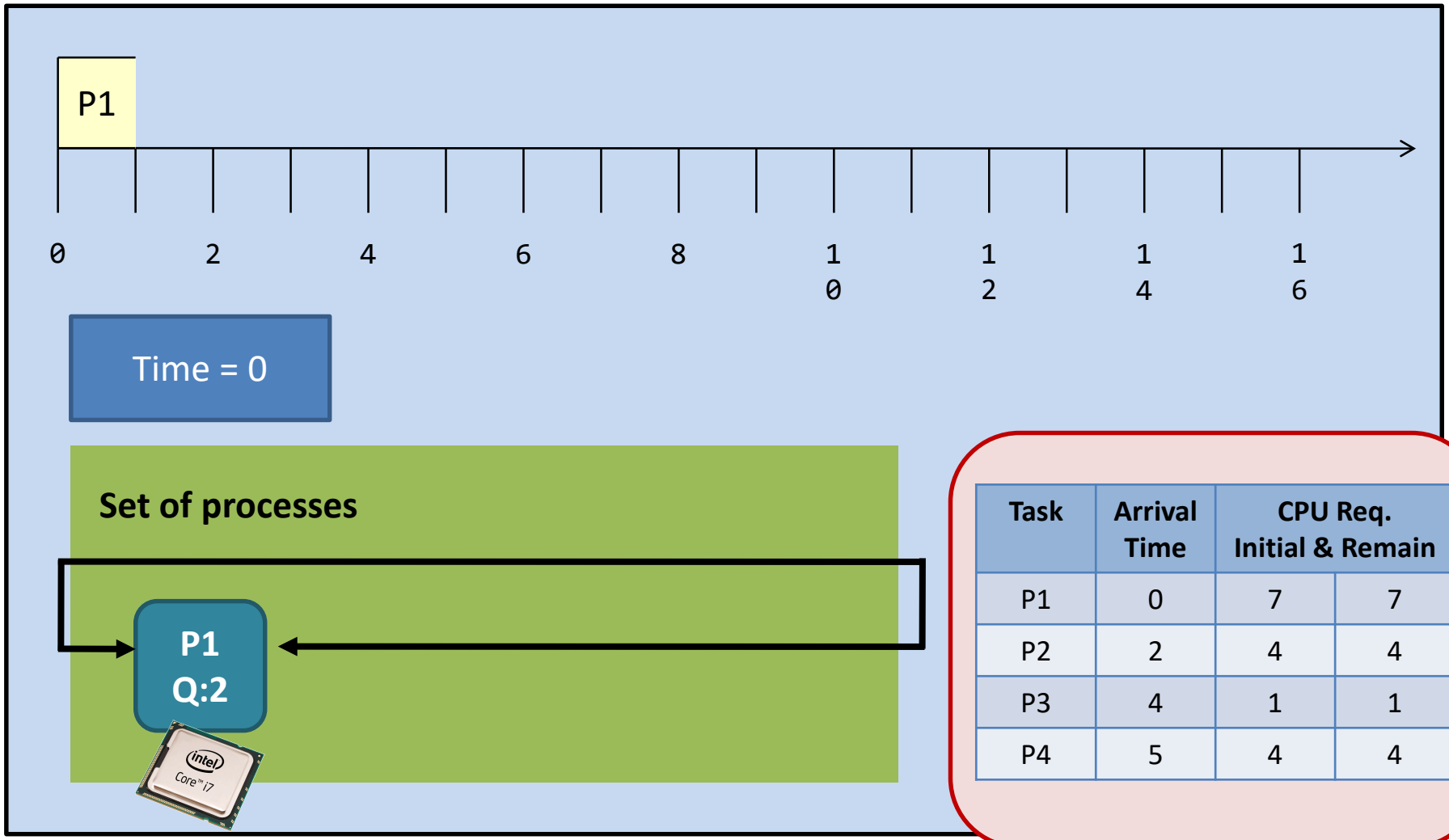
Rules for Round-Robin
(for this example only)

-The quantum of every process is fixed and is 2 units.

-The process queue is sorted according the processes' arrival time, in an ascending order.
(This rule allows us to break tie.)

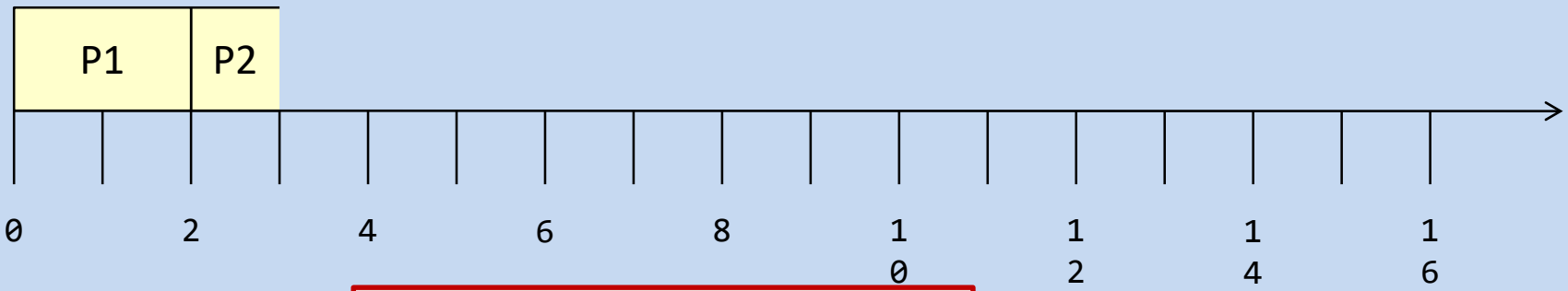
Task	Arrival Time	CPU Req. Initial & Remain	
P1	0	7	7
P2	2	4	4
P3	4	1	1
P4	5	4	4

调度算法之RR-轮转调度



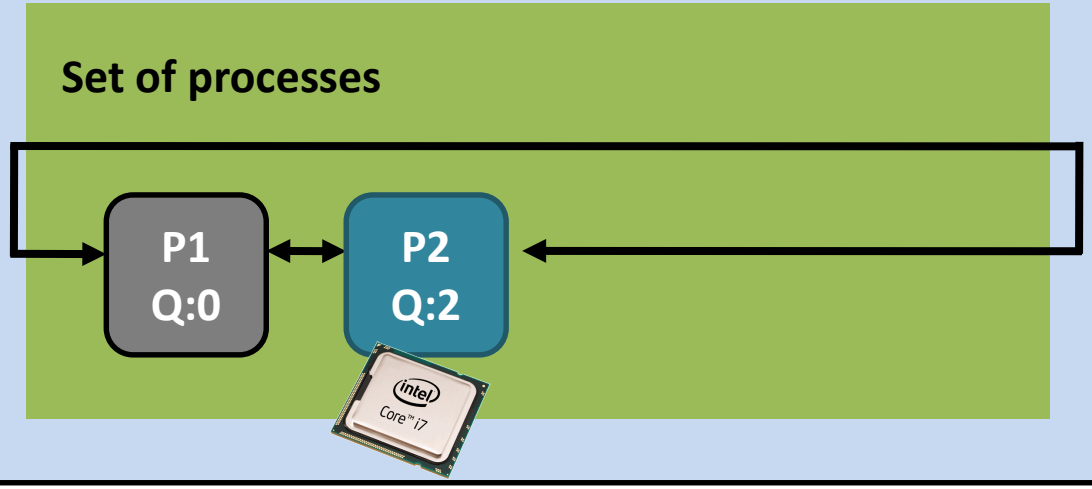
Task	Arrival Time	CPU Req. Initial & Remain	
P1	0	7	7
P2	2	4	4
P3	4	1	1
P4	5	4	4

调度算法之RR-轮转调度



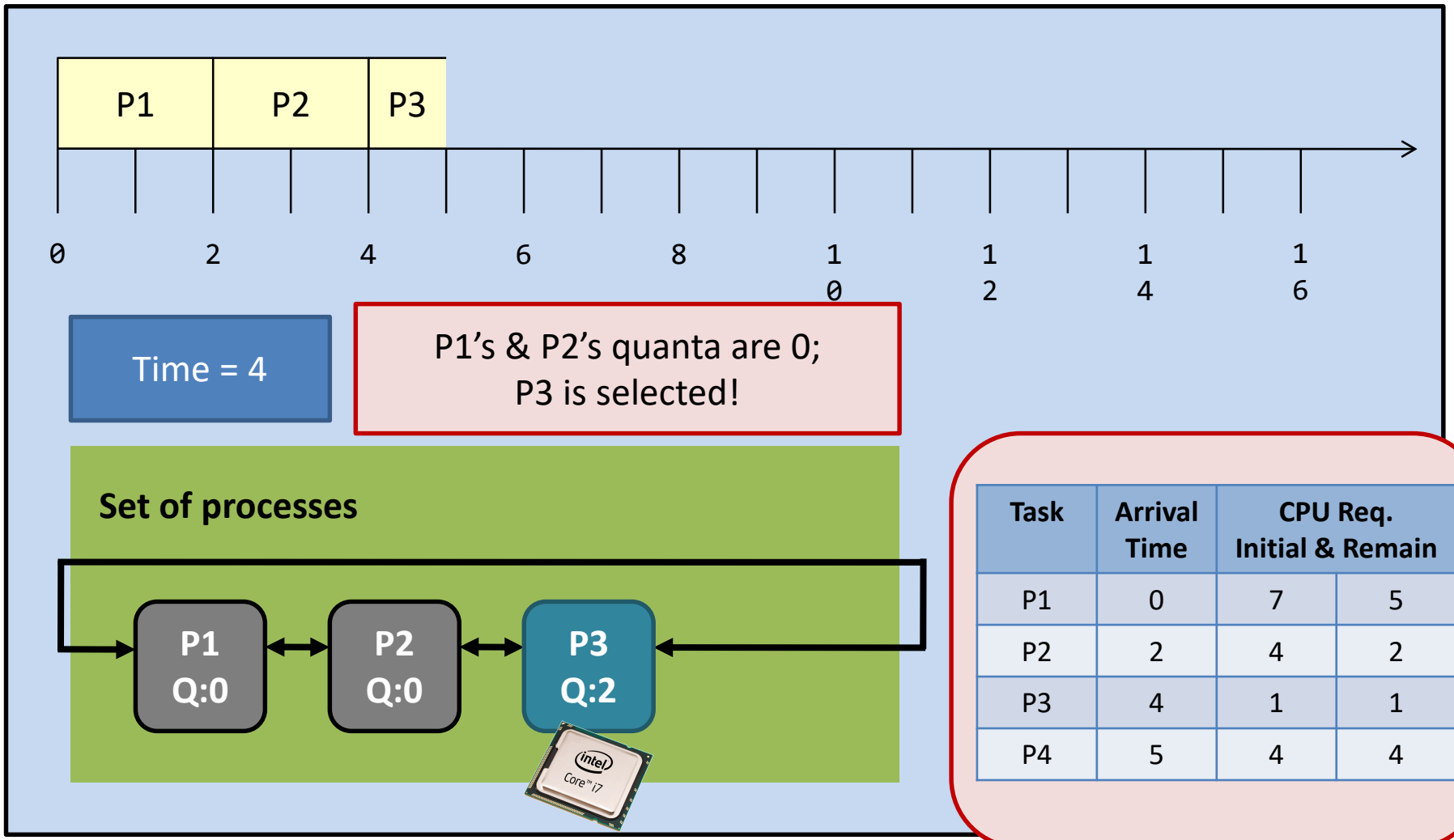
Time = 2

P1's quantum is 0;
P2 is selected!



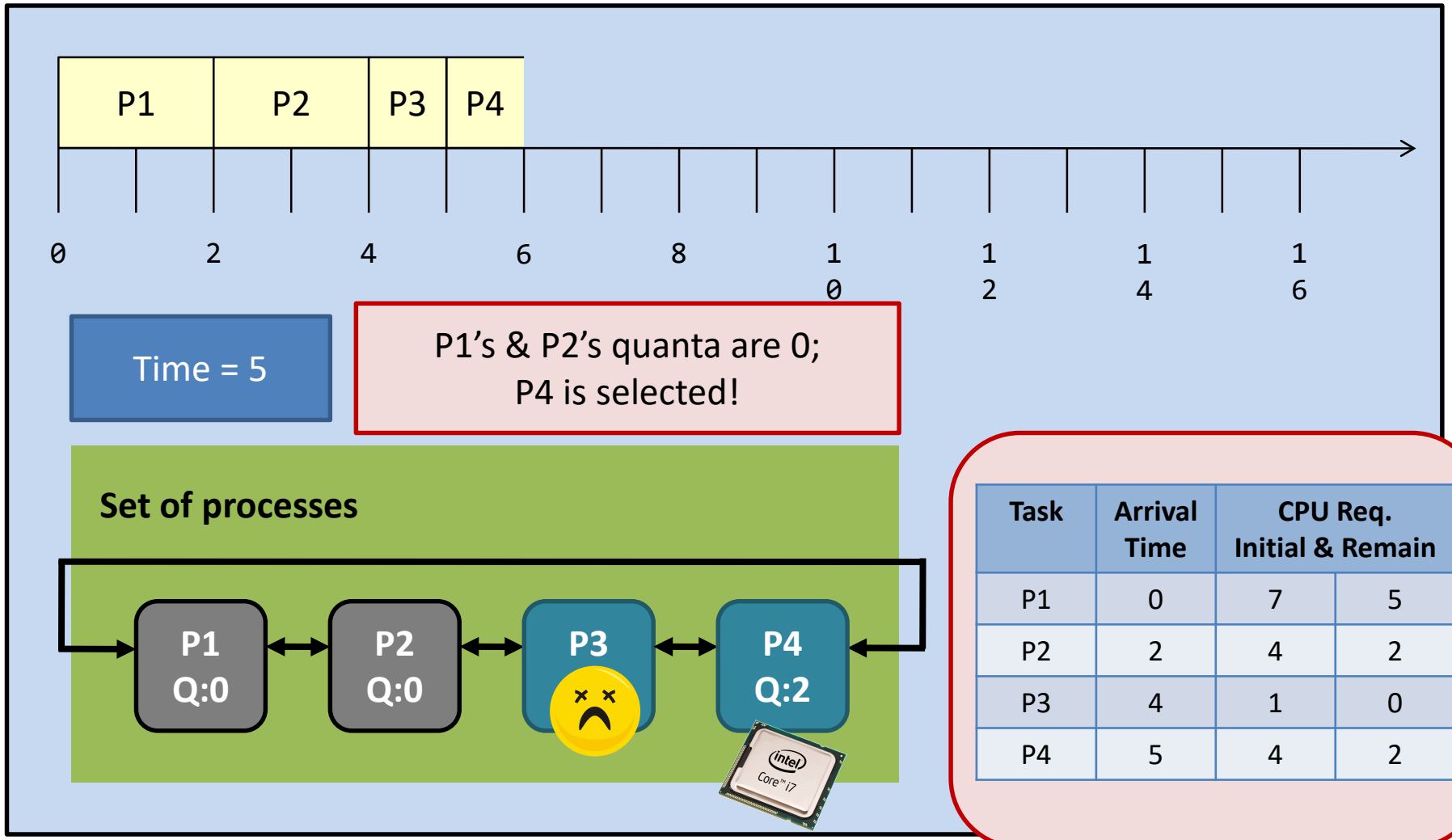
Task	Arrival Time	CPU Req. Initial & Remain	
P1	0	7	5
P2	2	4	4
P3	4	1	1
P4	5	4	4

调度算法之RR-轮转调度



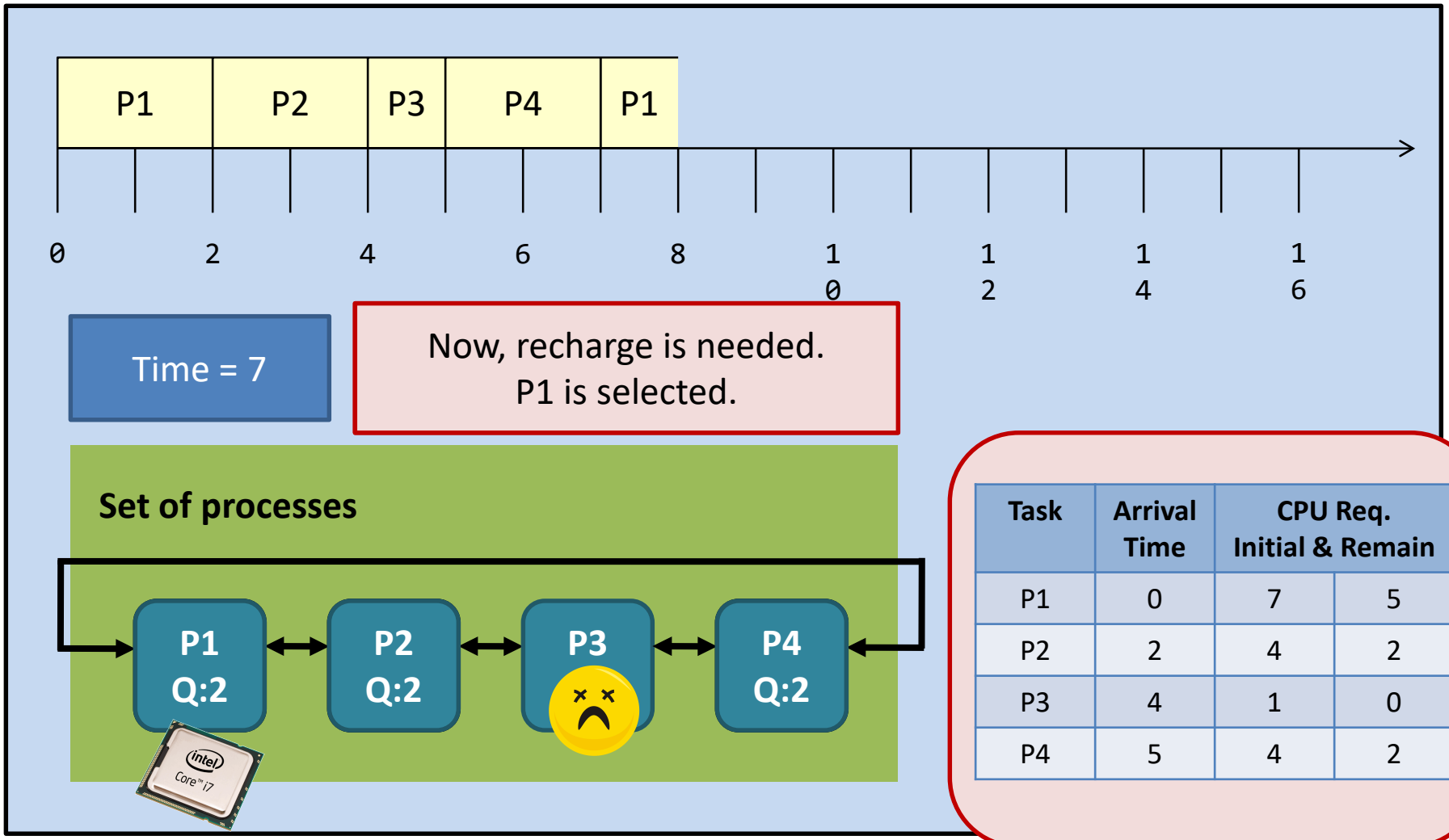
Task	Arrival Time	CPU Req. Initial & Remain	
P1	0	7	5
P2	2	4	2
P3	4	1	1
P4	5	4	4

调度算法之RR-轮转调度



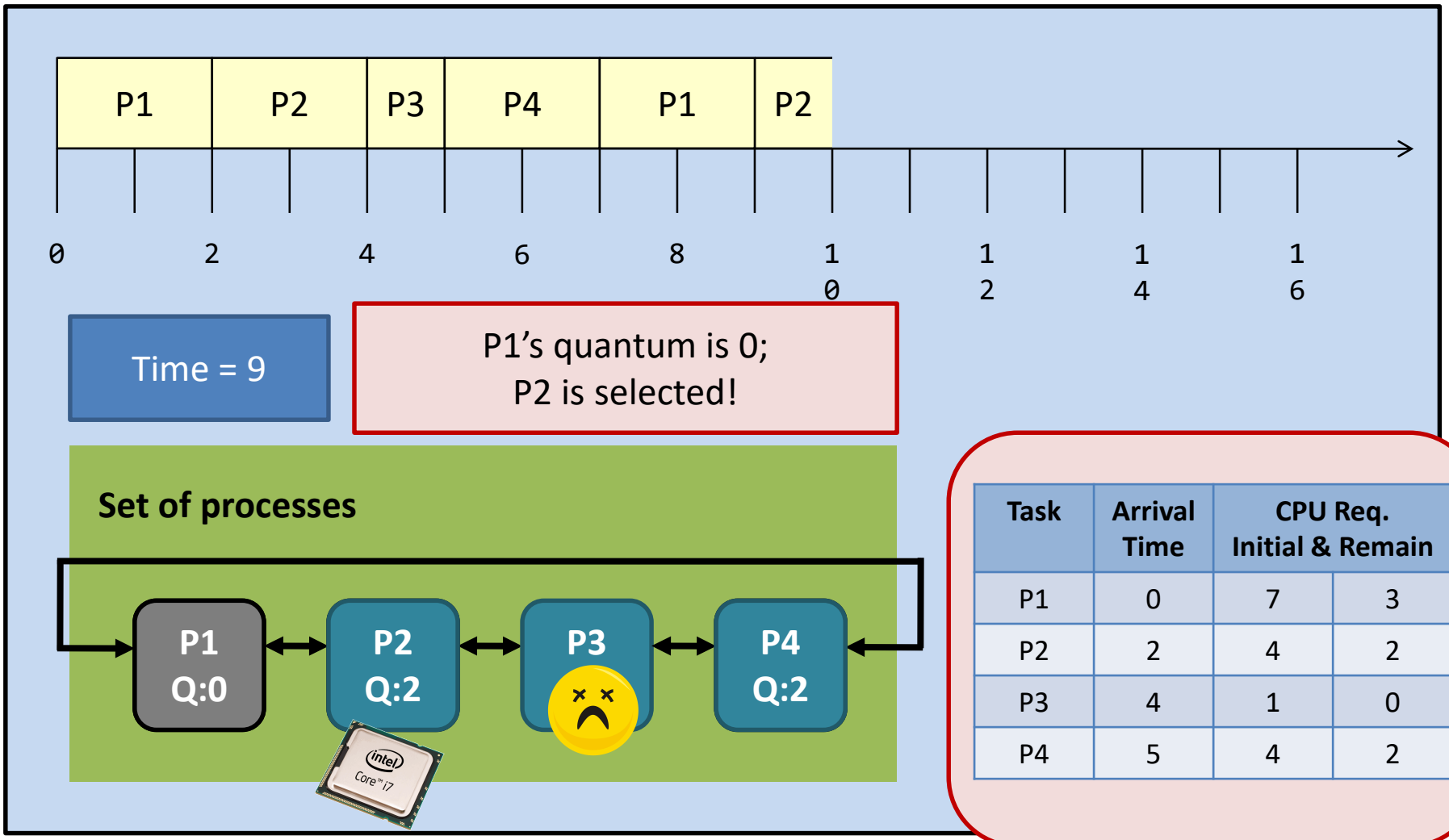
Task	Arrival Time	CPU Req. Initial & Remain	
P1	0	7	5
P2	2	4	2
P3	4	1	0
P4	5	4	2

调度算法之RR-轮转调度



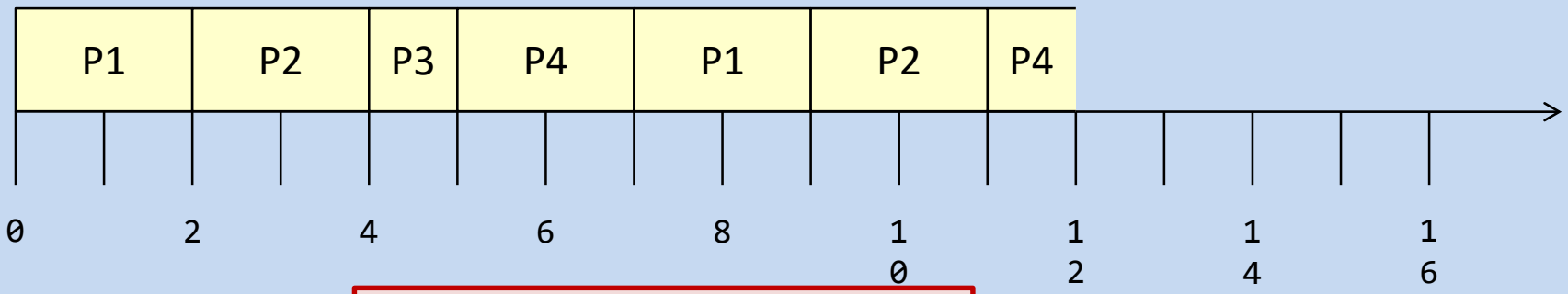
Task	Arrival Time	CPU Req. Initial & Remain	
P1	0	7	5
P2	2	4	2
P3	4	1	0
P4	5	4	2

调度算法之RR-轮转调度



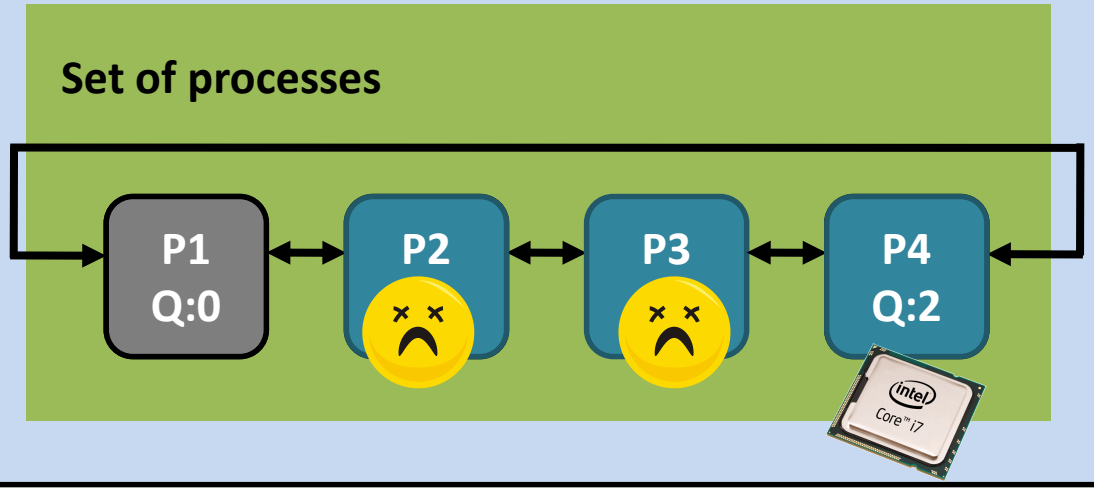
Task	Arrival Time	CPU Req. Initial & Remain	
P1	0	7	3
P2	2	4	2
P3	4	1	0
P4	5	4	2

调度算法之RR-轮转调度



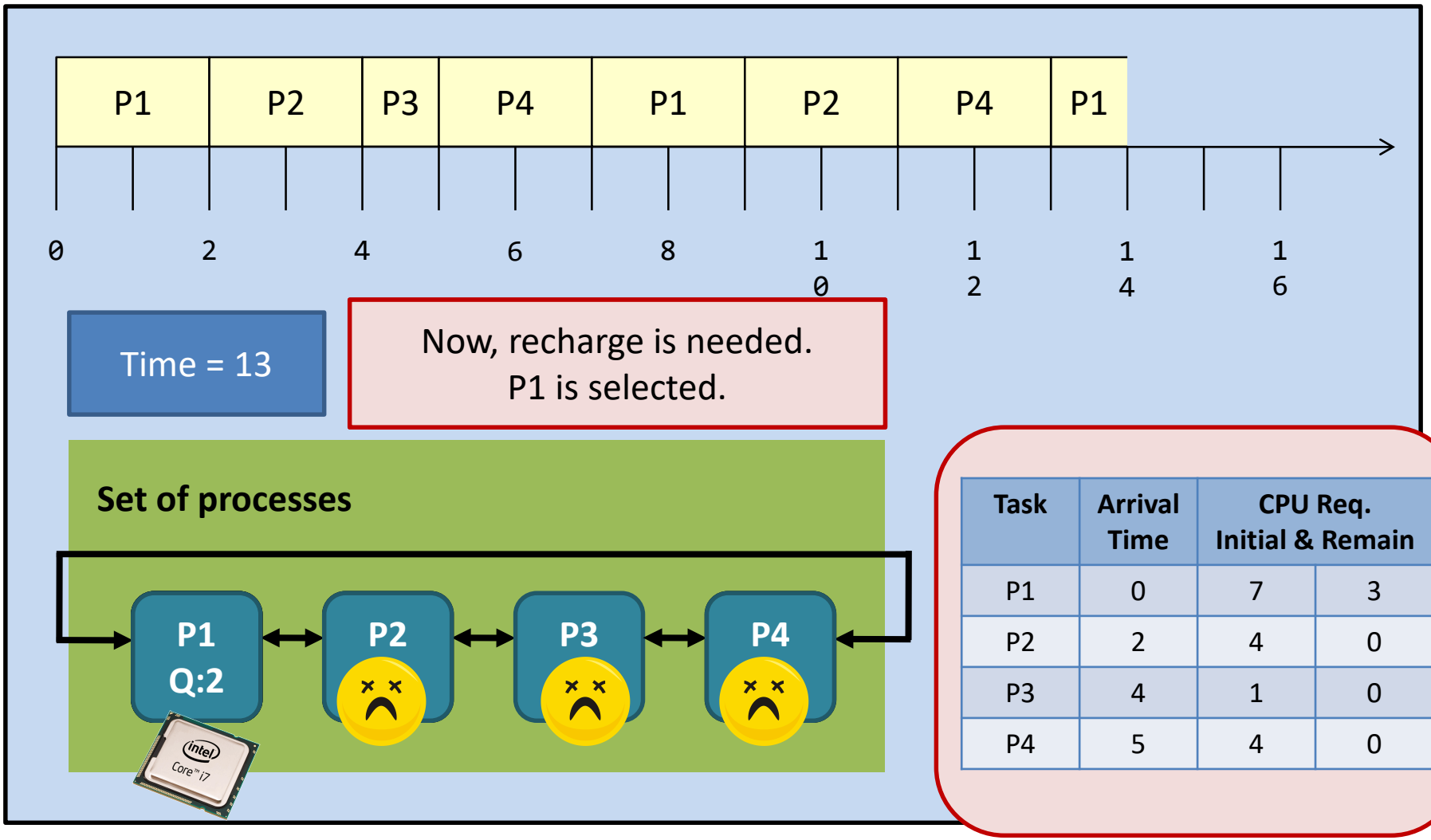
Time = 11

P1's quantum is 0;
P4 is selected!



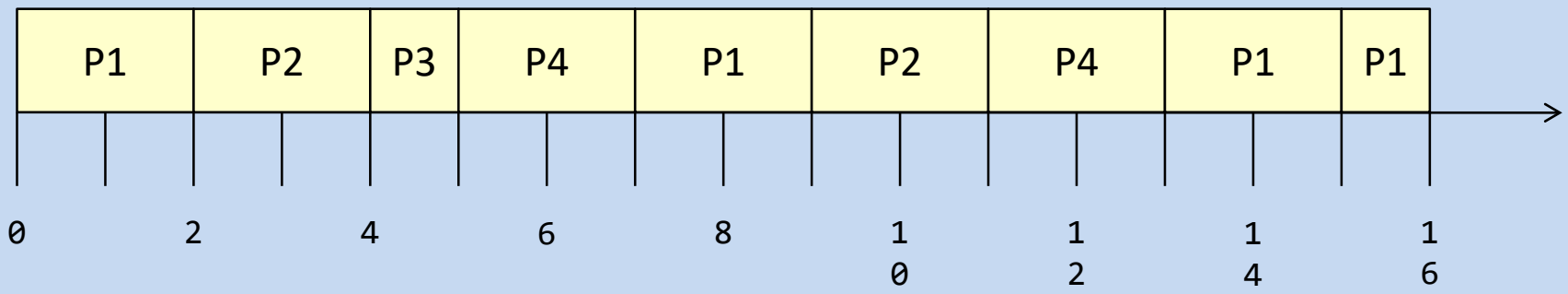
Task	Arrival Time	CPU Req. Initial & Remain	
P1	0	7	3
P2	2	4	0
P3	4	1	0
P4	5	4	2

调度算法之RR-轮转调度



Task	Arrival Time	CPU Req. Initial & Remain	
P1	0	7	3
P2	2	4	0
P3	4	1	0
P4	5	4	0

调度算法之RR-轮转调度



Waiting time:

P1 = 9; P2 = 5; P3 = 0; P4 = 4;

Average = (9 + 5 + 0 + 4) / 4 = 4.5

Turnaround time:

P1 = 16; P2 = 9; P3 = 1; P4 = 8;

Average = (16 + 9 + 1 + 8) / 4 = 8.5

Task	Arrival Time	CPU Req. Initial & Remain	
P1	0	7	0
P2	2	4	0
P3	4	1	0
P4	5	4	0

RR与SJF比较

	Non-preemptive SJF	Preemptive SJF	RR
Average waiting time	4	**3**	**4.5 (largest)**
Average turnaround time	8	**7**	**8.5 (largest)**
# of context switching	**3**	5	**8 (largest)**

So, the RR algorithm gets all the bad! Why do we still need it?

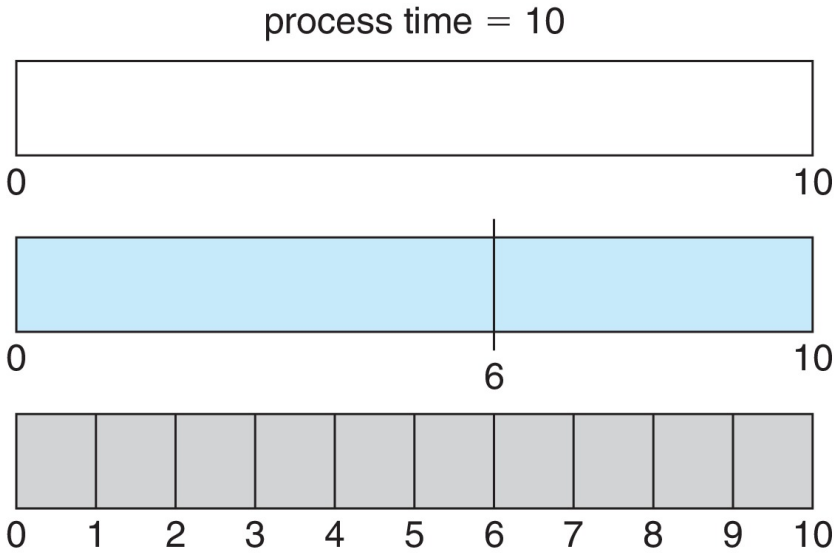
The responsiveness of the processes is great under the RR algorithm. E.g., you won't feel a job is "frozen" because every job is on the CPU from time to time!

RR的特点

RR算法的性能很大程度取决于时间片的大小：

—— 时间片太大，RR算法与FCFS算法一样；

—— 如果时间片太小，产生大量的上下文切换，反而使程序的性能降低；

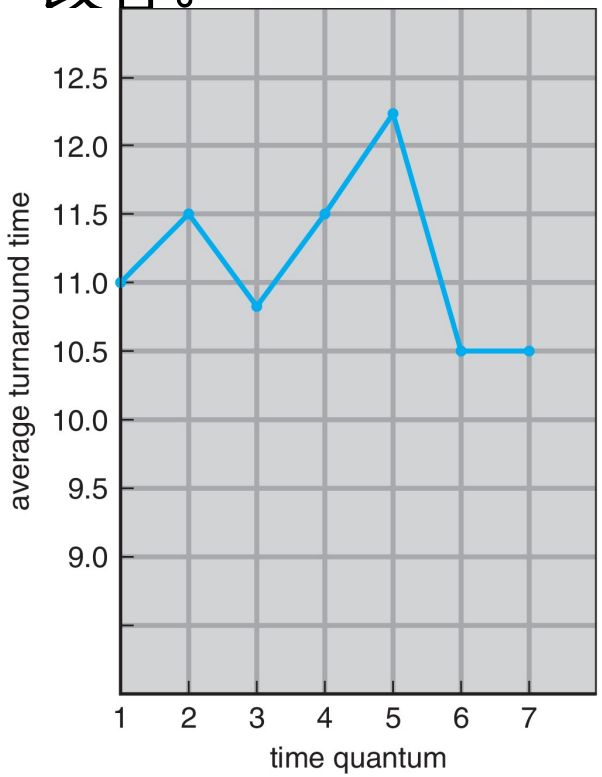


quantum	context switches
12	0
6	1
1	9

时间片大小不同时上下文切换情况

RR的特点

- 时间片要远大于上下文切换的时间，让上下文切换的时间占时间片的一小部分；
- 随着时间片大小的增加，一组进程的平均周转时间不一定会改善。



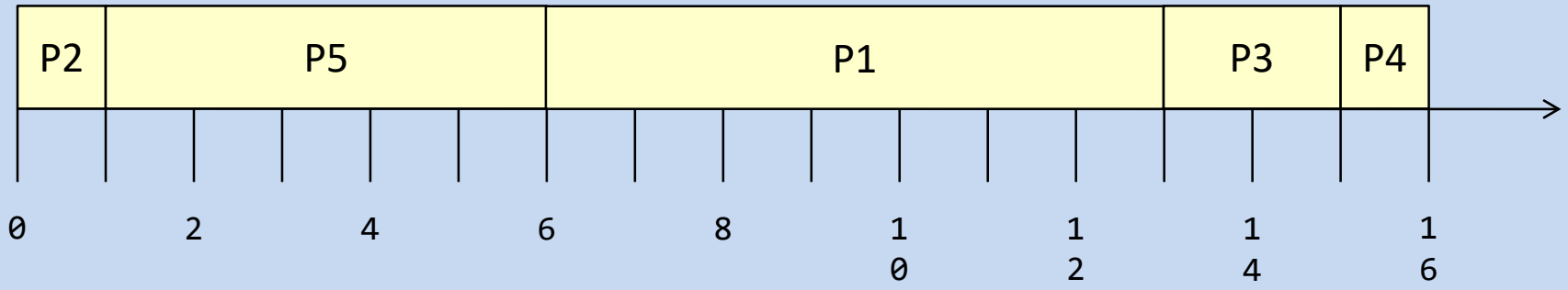
process	time
P_1	6
P_2	3
P_3	1
P_4	7

80%的CPU执行时间应小于时间片

优先级调度

- 优先级编号（整数）与每个进程相关联；
- 如何定义优先级：内部(时限、内存需求)和外部定义（重要性、所有者等）；
- CPU分配给具有最高优先级（最小整数）的进程（最高优先级）
  - 抢占式的
  - 非抢占式
- SJF是优先级调度，其中优先级是预测的下一个CPU执行时间的倒数；
- 问题：饥饿－低优先级进程可能永远不会执行；
- 解决方案:老化－随着时间的推移，进程的优先级会增加；

优先级调度



Assumption:

-All arrive at time 0
-Low numbers represent high priority

Problem: Indefinite blocking or starvation

Solution: Aging (gradually increase the priority of waiting processes)

Task	CPU Burst	Priority
P1	7	3
P2	1	1
P3	2	4
P4	1	5
P5	5	2

多级队列调度

- 仍然是优先级调度器；
- 就绪作业队列分成多个单独的队列，每个队列具有不同的调度算法；
- 分成前台进程（RR）和后台进程（FCFS）；

Priority class 5	Non-preemptive, FIFO
Priority class 4	Non-preemptive, SJF
Priority class 3	RR with quantum = 10 units.
Priority class 2	RR with quantum = 20 units.
Priority class 1	RR with quantum = 40 units.

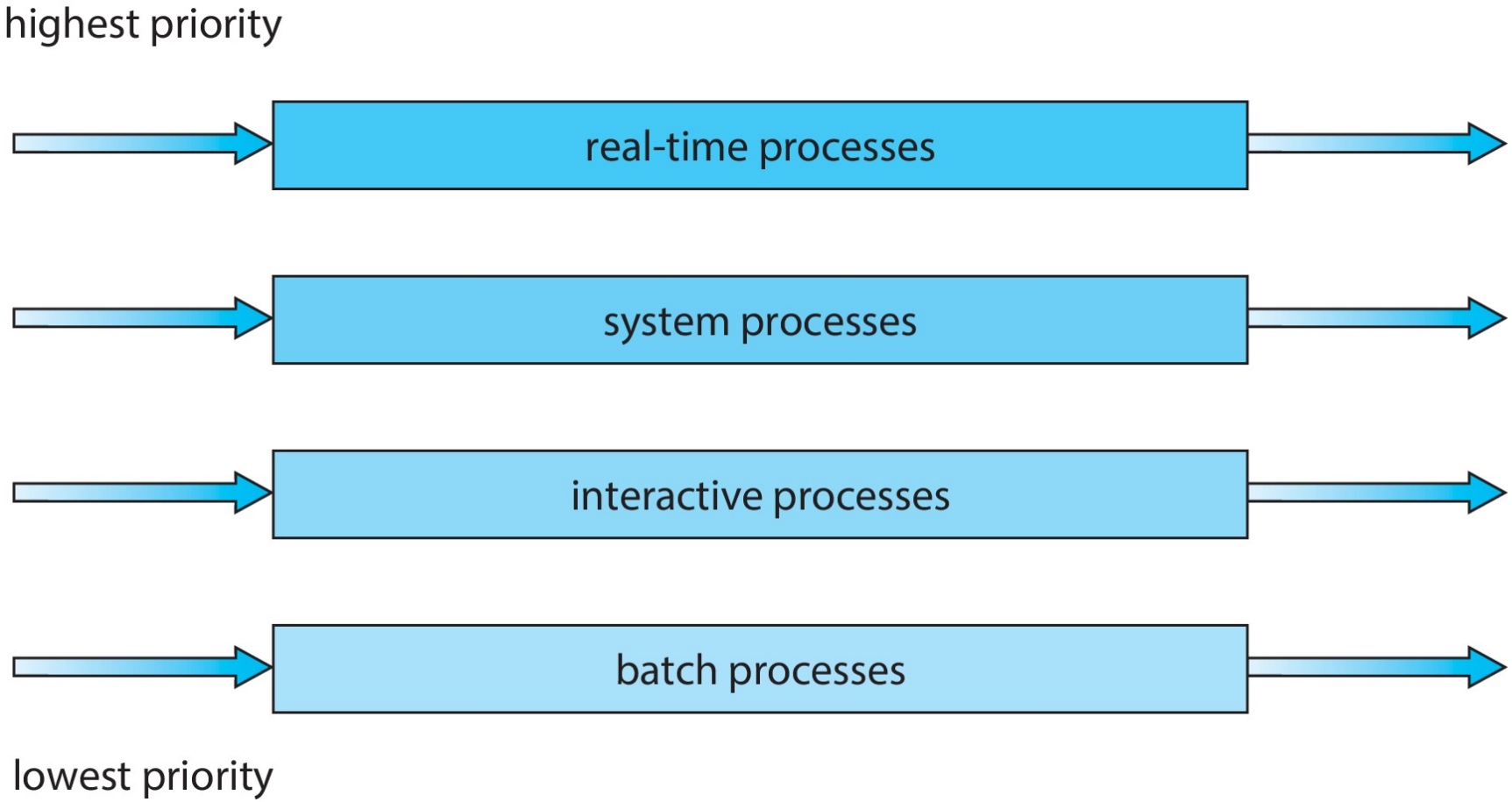
Just an example.

The processes are **permanently assigned** to one queue

Fixed-priority preemptive scheduling among queues

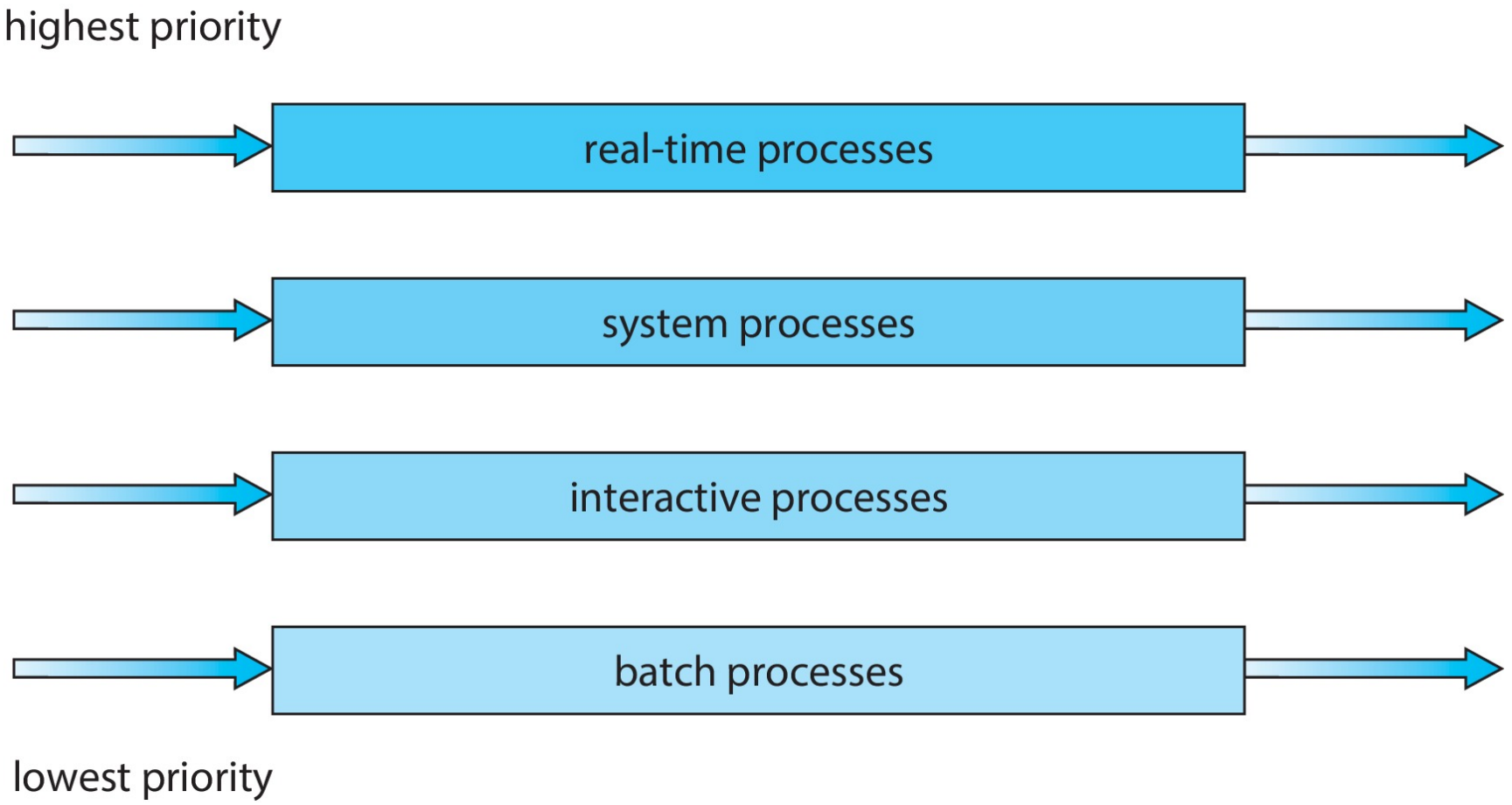
多级队列调度

不同的队列之间应用调度，通常采用固定优先级抢占调度，如前台队列可以比后台队列具有绝对的优先；



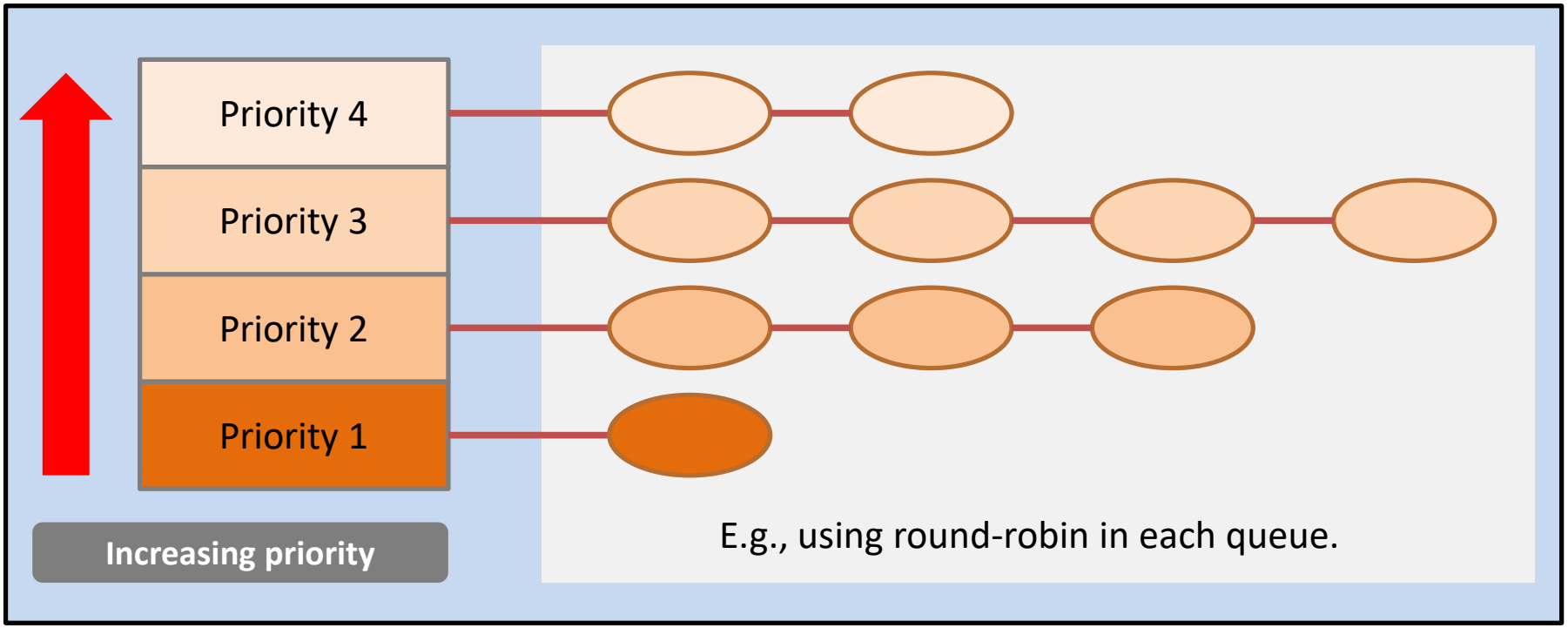
多级队列调度

另一种方式是队列之间划分时间片，每个队列都有一定比例的CPU时间，可用于调度队列内的进程；



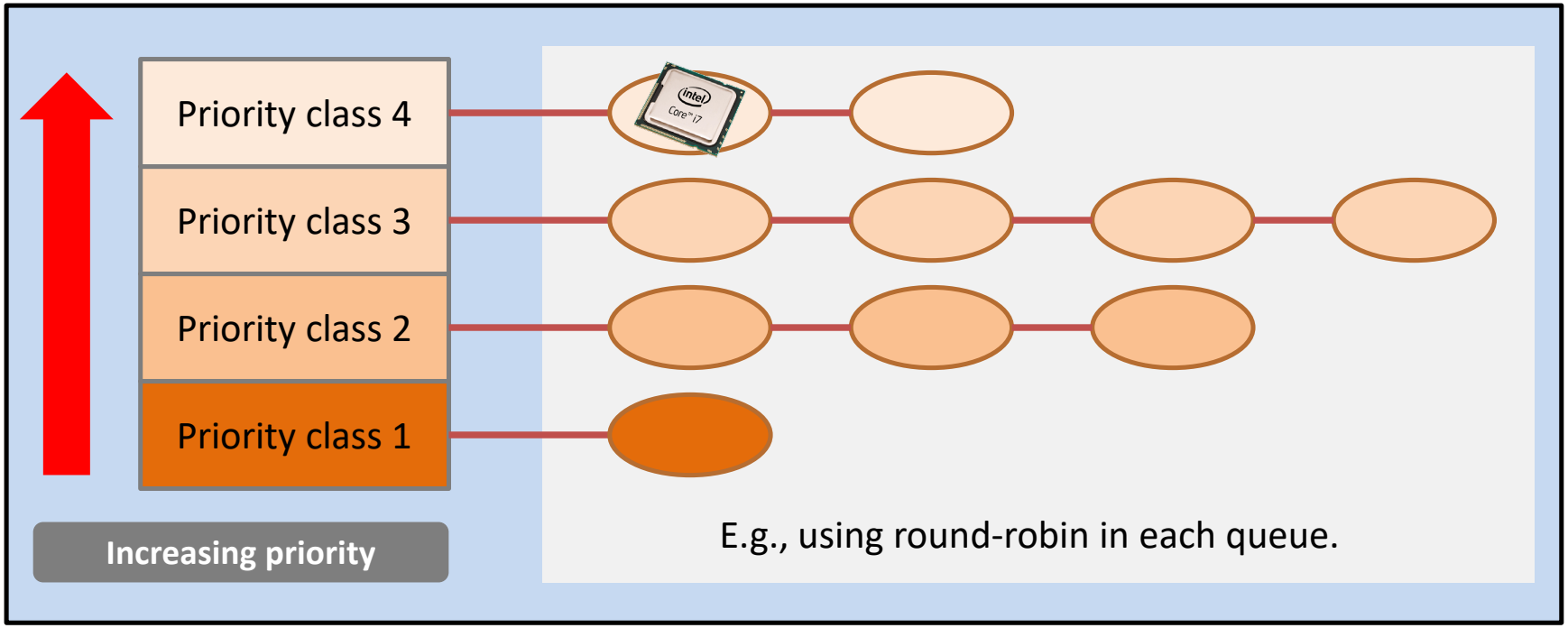
多级队列调度

- **Properties**: process is assigned a fix priority when they are submitted to the system.



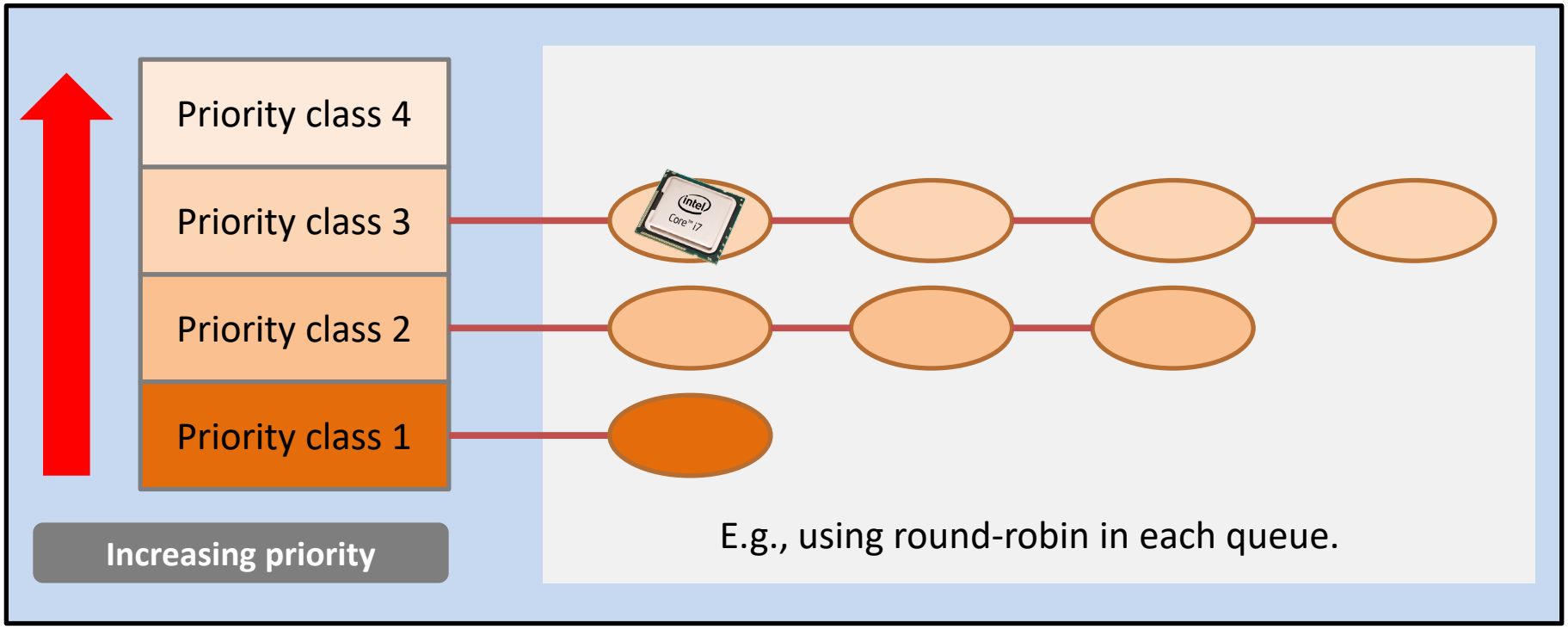
多级队列调度

- The highest priority class will be selected.
  - To prevent high-priority tasks from running indefinitely.
  - The tasks with a higher priority should be short-lived, but important;



多级队列调度

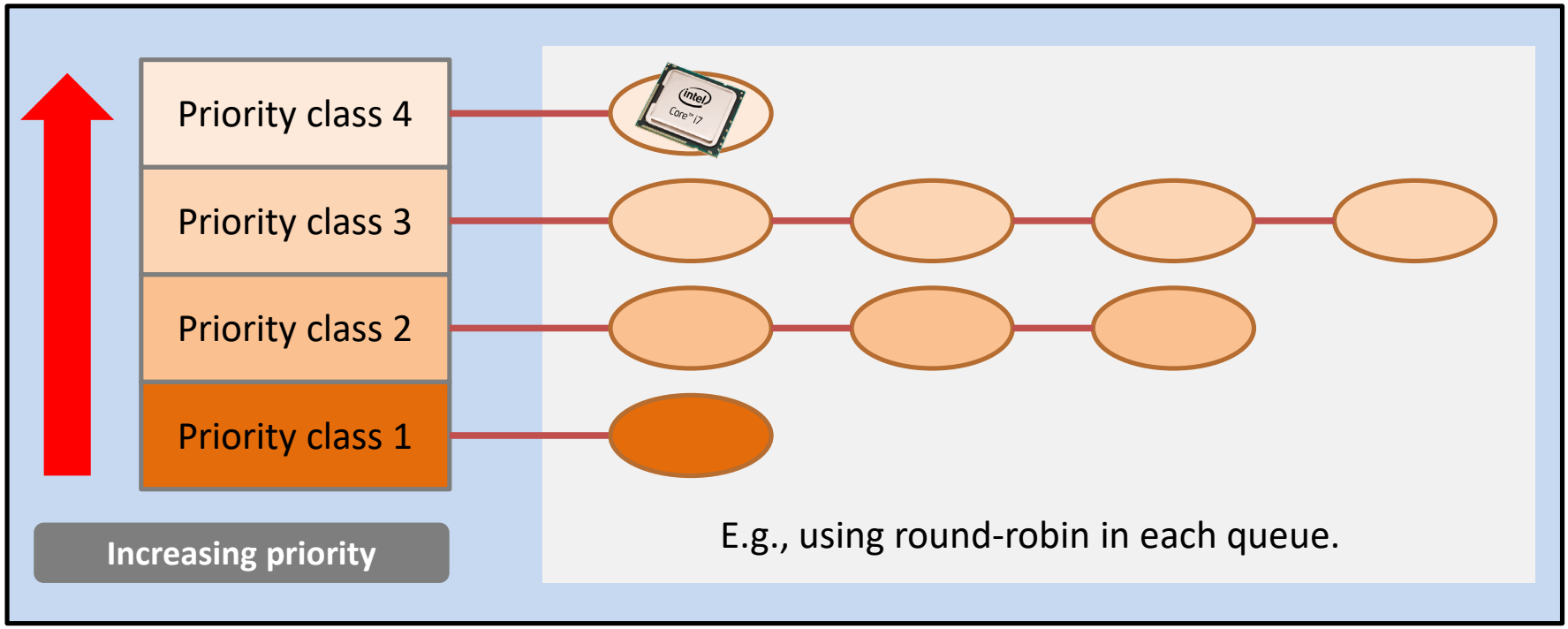
- Lower priority classes will be scheduled only when the upper priority classes has no tasks.



多级队列调度

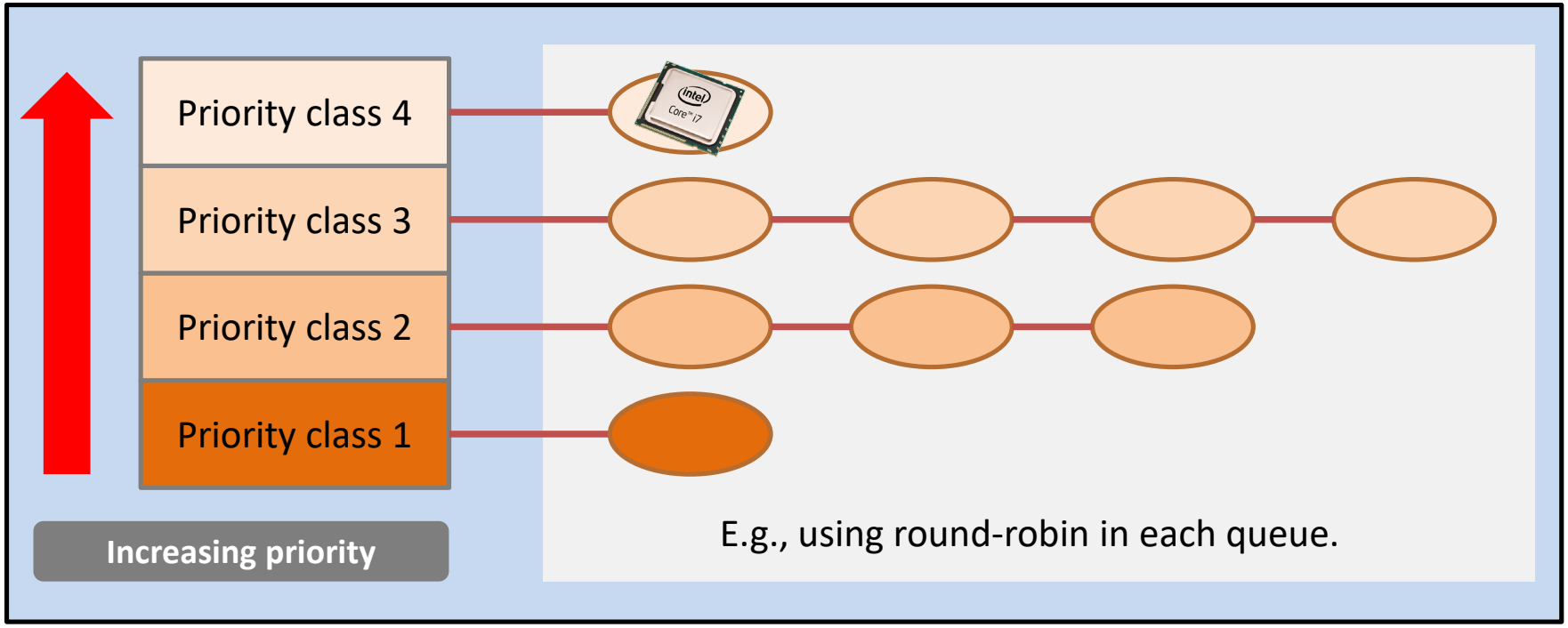
- Of course, it is a good design to have a high-priority task preempting a low-priority task.

  (conditioned that the high-priority task is short-lived.)



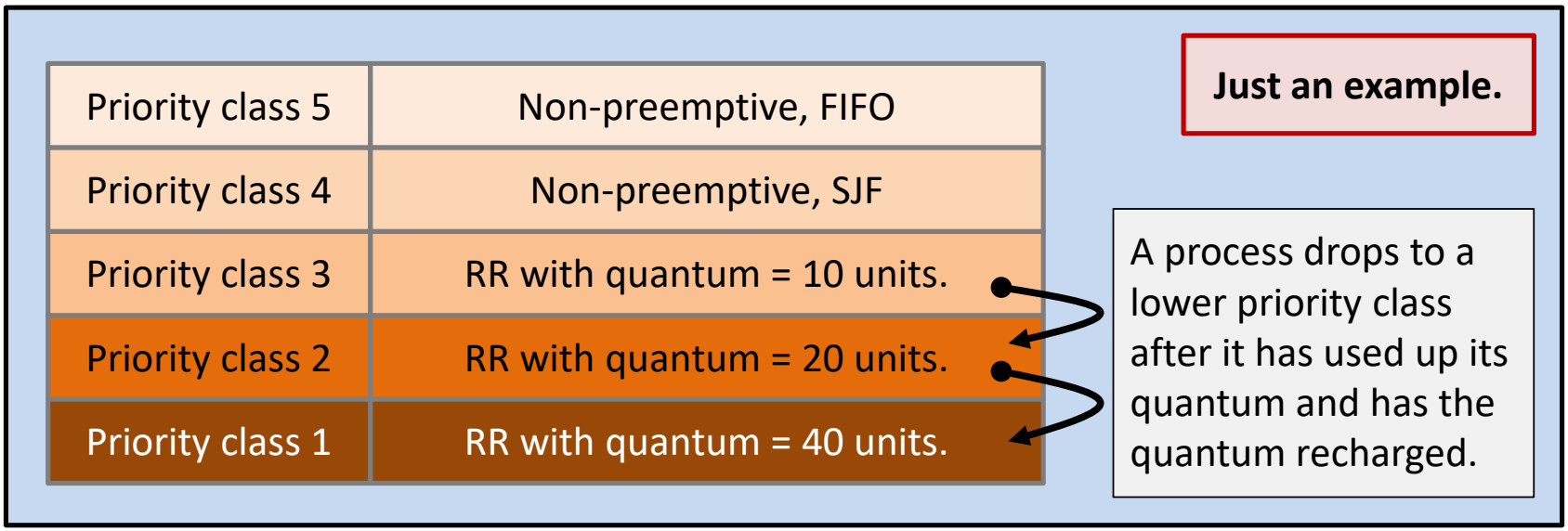
多级队列调度

- Any problem?
  - Fixed priority
  - Indefinite blocking or starvation

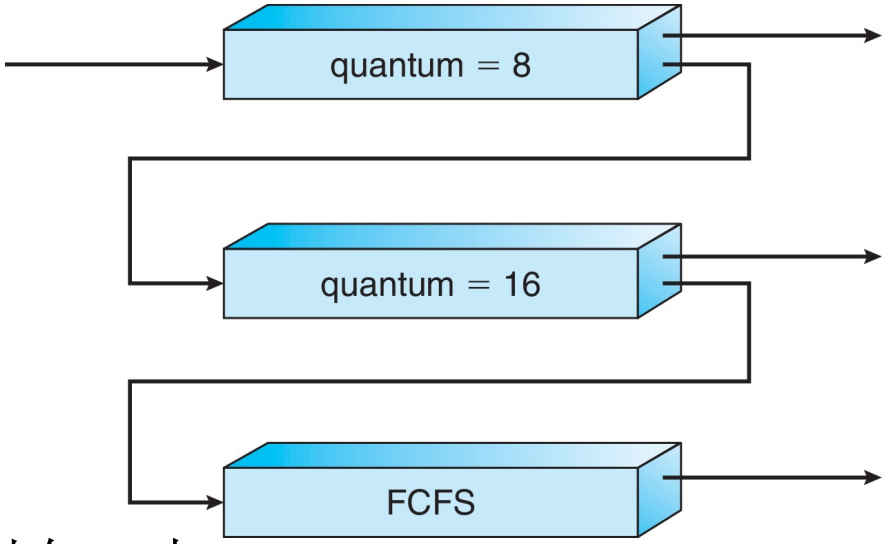


多级反馈调度队列

- **How to improve the previous scheme?**
  - Allows a process to move between queues (dynamic priority).
  - Why needed?
    - Eg.: Separate processes according to their CPU bursts.



多级反馈调度队列示例



- 三个队列：
  - Q0 – RR时间片为8毫秒
  - Q1 – RR时间片16毫秒
  - Q2 – FCFS
- 进程调度
  - 一个新进程进入队列Q0，该队列在RR中提供服务
    - 当它获得CPU时，进程接收8毫秒；
    - 如果进程未在8毫秒内完成，则进程将移动到队列Q1；
  - 在Q1，作业再次在RR中服务，并接收额外的16毫秒；
    - 如果仍然没有完成，它将被抢占并移动到队列Q2

- 短时间的进程优先级最高；
- 长时间的进程自动降低优先级；

多级反馈调度队列

- 通常，多级反馈队列调度程序可由以下参数定义：
  - 队列数量；
  - 每个队列的调度算法；
  - 用于确定何时升级到高优先级队列的方法；
  - 用于确定何时降级到更低优先级队列的方法；
  - 用于确定当某个进程需要服务时该进程将进入哪个队列的方法；
- 成为最通用的调度算法，通过配置适应特定的系统，也是最复杂的调度方法；
- 老化可以使用多级反馈队列来实现；

线程调度

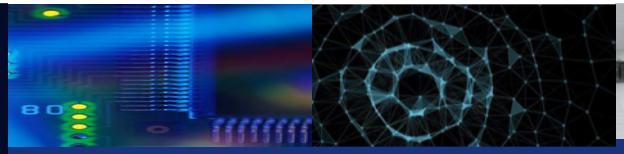
- 用户级线程和内核级线程之间的区别；
- 如果支持线程，则调度线程，而不是进程；
- 多对一和多对多模型，线程库安排用户级线程在LWP上运行
  - 称为进程级争用范围（PCS），因为调度竞争在进程内
  - 通常通过程序员设置的优先级来完成；
- 调度到可用CPU上的内核线程是系统级争用范围（SCS）——系统中所有线程之间的竞争；

- 对于一对一模型的系统，如Windows、Linux、Solaris等只采用SCS调度；
- 通常情况下，PCS采用优先级调度；

Pthreads调度

- API允许在创建线程期间指定PCS或SCS：
  - PTHREAD_SCOPE_PROCESS：使用PCS调度来调度线程；
  - PTHREAD_SCOPE_SYSTEM：使用SCS调度来调度线程；

- 可受操作系统限制 - Linux和macOS仅允许PTHREAD_SCOPE_SYSTEM



Pthreads调度

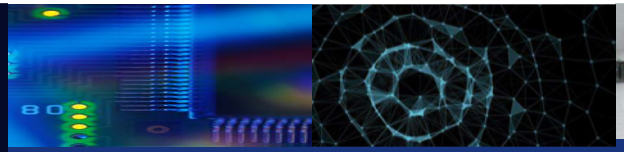
Pthreads IPC提供两个函数，用于获取或设置竞争范围策略：

- pthread_attr_setscope(pthread_attr_t *attr, int scope)
- pthread_attr_getscope(pthread_attr_t *attr, int *scope)

```
#include <pthread.h>
#include <stdio.h>
#define NUM_THREADS 5

int main(int argc, char *argv[])
{
   int i, scope;
   pthread_t tid[NUM_THREADS];
   pthread_attr_t attr;

   /* get the default attributes */
   pthread_attr_init(&attr);

   /* first inquire on the current scope */
   if (pthread_attr_getscope(&attr, &scope) != 0)
      fprintf(stderr, "Unable to get scheduling scope\n");
   else {
      if (scope == PTHREAD_SCOPE_PROCESS)
        printf("PTHREAD_SCOPE_PROCESS");
      else if (scope == PTHREAD_SCOPE_SYSTEM)
        printf("PTHREAD_SCOPE_SYSTEM");
      else
        fprintf(stderr, "Illegal scope value.\n");
   }

   /* set the scheduling algorithm to PCS or SCS */
   pthread_attr_setscope(&attr, PTHREAD_SCOPE_SYSTEM);

   /* create the threads */
   for (i = 0; i < NUM_THREADS; i++)
      pthread_create(&tid[i],&attr,runner,NULL);

   /* now join on each thread */
   for (i = 0; i < NUM_THREADS; i++)
      pthread_join(tid[i], NULL);
}

/* Each thread will begin control in this function */
void *runner(void *param)
{
   /* do some work ... */

   pthread_exit(0);
}
```

选择函数

- 选择函数决定选择哪个就绪进程下次执行；
- 这个函数可以根据优先级、资源需求或进程的执行特性来进行选择；
- 对于执行特性，可以根据以下三个重要的参数：
  - ***w*** = 目前为止在系统中的等待时间；
  - ***e*** = 目前为止花费的执行时间；
  - ***s*** = 进程所需要的总服务时间，包括 e；这个参数通常须进行估计或由用户提供；



选择调度策略

	FCFS	Round robin	SPN	SRT	HRRN	Feedback
Selection function	max[w]	constant	min[s]	min[s – e]	$\max\left(\frac{w+s}{s}\right)$	(see text)
Decision mode	Non-preemptive	Preemptive (at time quantum)	Non-preemptive	Preemptive (at arrival)	Non-preemptive	Preemptive (at time quantum)
Through-Put	Not emphasized	May be low if quantum is too small	High	High	High	Not emphasized
Response time	May be high, especially if there is a large variance in process execution times	Provides good response time for short processes	Provides good response time for short processes	Provides good response time	Provides good response time	Not emphasized
Overhead	Minimum	Minimum	Can be high	Can be high	Can be high	Can be high
Effect on processes	Penalizes short processes; penalizes I/O bound processes	Fair treatment	Penalizes long processes	Penalizes long processes	Good balance	May favor I/O bound processes
Starvation	No	No	Possible	Possible	No	Possible

Table 9.3

Characteristics of Various Scheduling Policies

(Table can be found on page 405 in textbook 操作系统精髓与设计原理)

决策模型

- 说明选择函数开始执行瞬间的处理方式；
- 分为两类：
  - 非抢占
  - 抢占

调度策略比较

表 9.4 进程调度示例

进 程	到达时间	服务时间
A	0	3
B	2	6
C	4	4
D	6	5
E	8	2

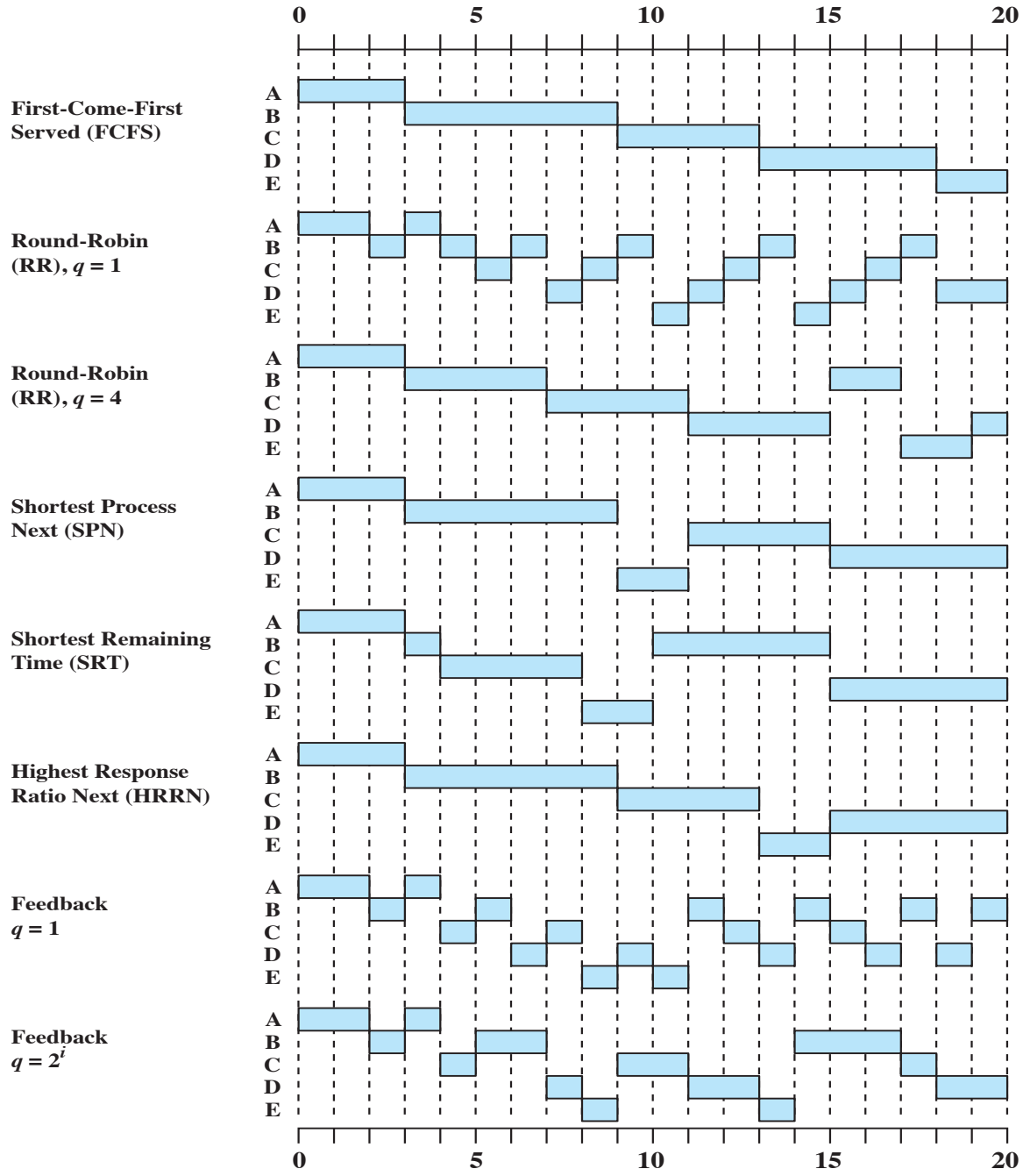


Figure 9.5 A Comparison of Scheduling Policies

性能比较

- **Any scheduling discipline that chooses the next item to be served independent of service time obeys the relationship:**

$$\frac{T_r}{T_s} = \frac{1}{1 - \rho}$$

where

T_r = turnaround time or residence time; total time in system, waiting plus execution

T_s = average service time; average time spent in Running state

ρ = processor utilization

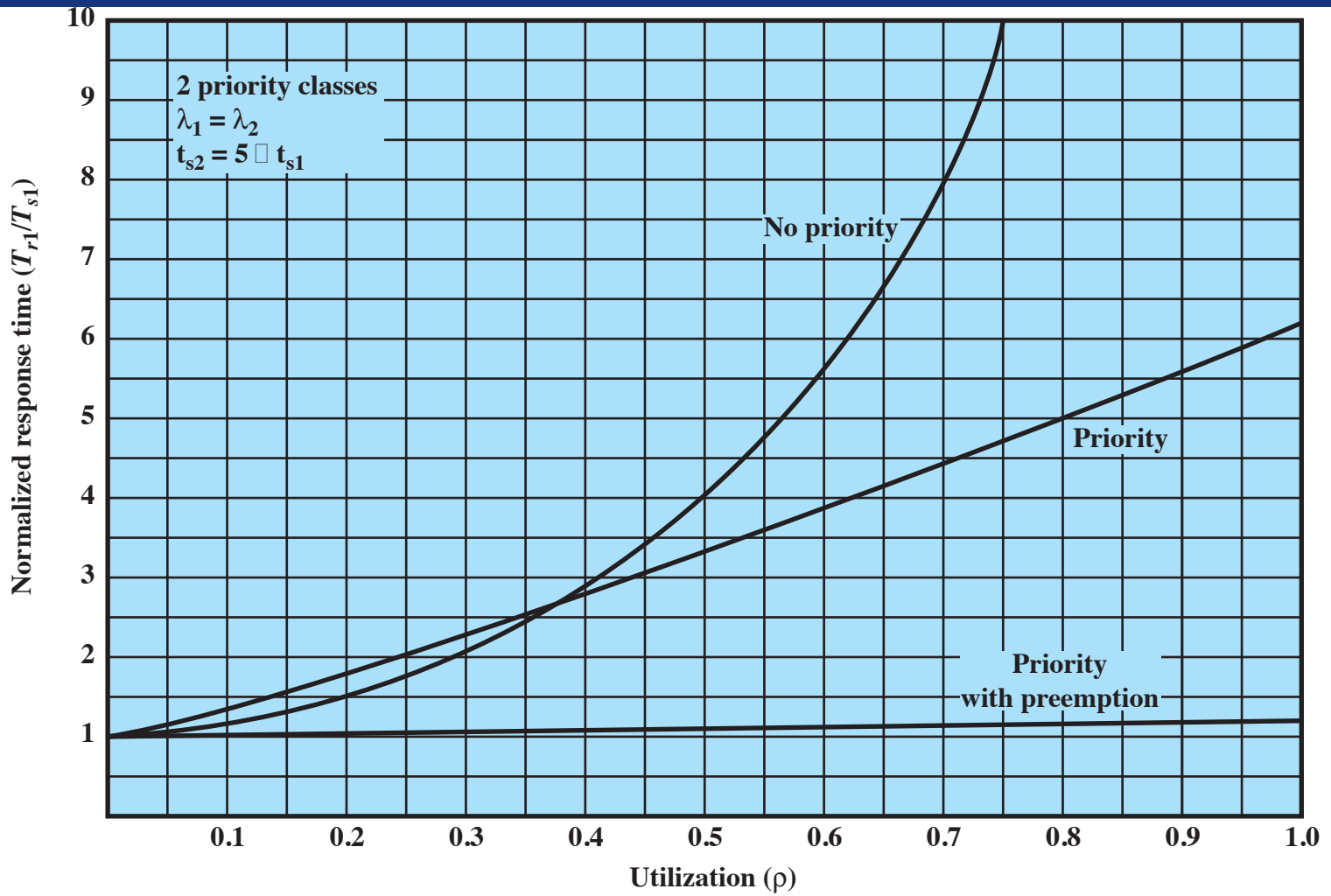
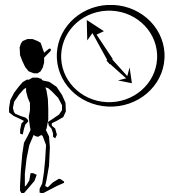
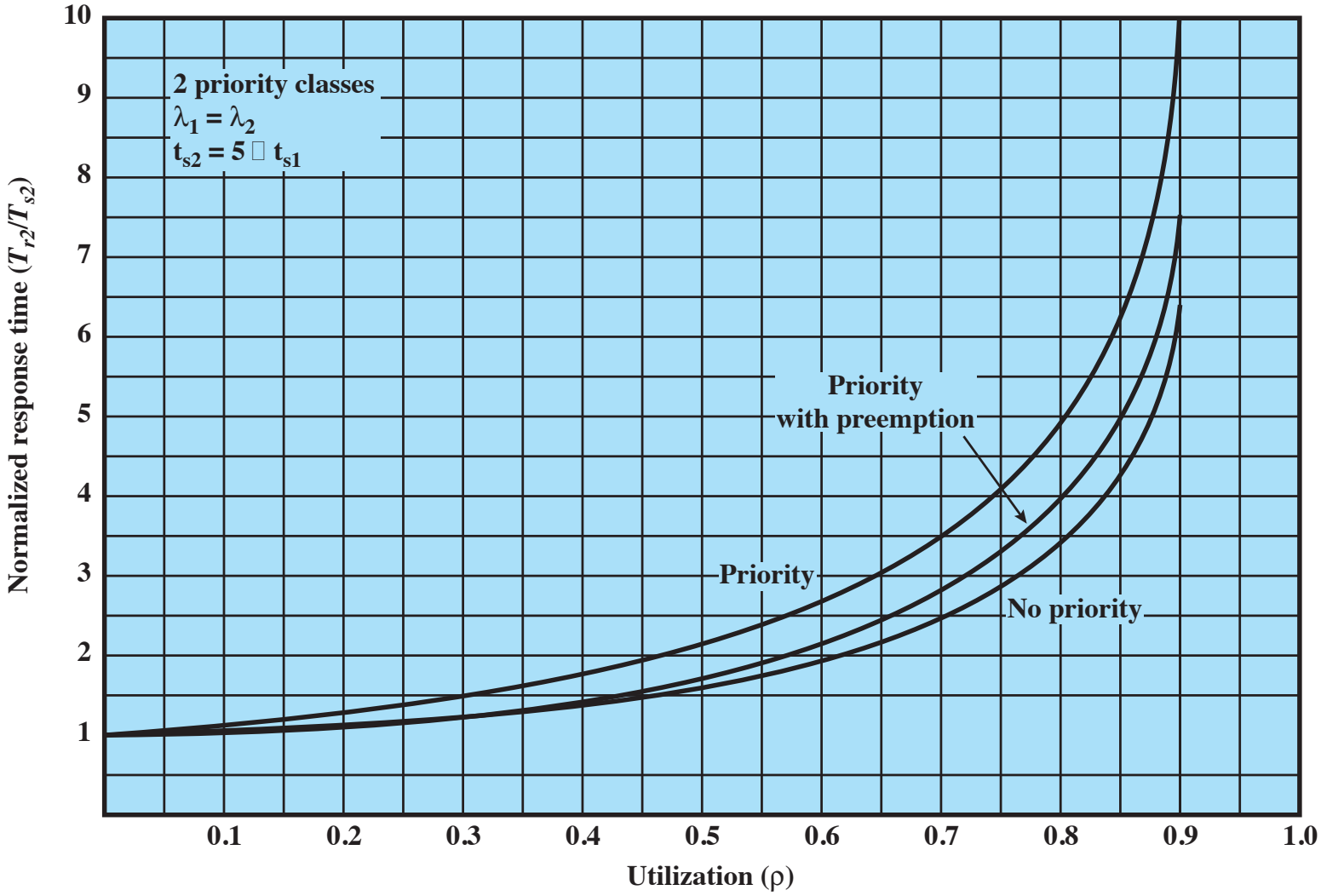


Figure 9.12 Normalized Response Time for Shorter Processes





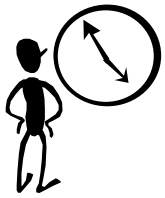


Figure 9.13 Normalized Response Time for Longer Processes

公平共享调度

- 基于进程集合的计划决策；
- 每个用户都被分配了处理器的一部分；
- 目标是监控使用情况，为拥有超过其公平份额的用户提供更少的资源，而为拥有低于其公平份额用户提供更多的资源

Time	Process A			Process B			Process C		
	Priority	Process CPU count	Group CPU count	Priority	Process CPU count	Group CPU count	Priority	Process CPU count	Group CPU count
0	60	0 1 2 • • 60	0 1 2 • • 60	60	0	0	60	0	0
1	90	30	30	60	0 1 2 • • 60	0 1 2 • • 60	60	0	0 1 2 • • 60
2	74	15 16 17 • • 75	15 16 17 • • 75	90	30	30	75	0	30
3	96	37	37	74	15	15 16 17 • • 75	67	0 1 2 • • 60	15 16 17 • • 75
4	78	18 19 20 • • 78	18 19 20 • • 78	81	7	37	93	30	37
5	98	39	39	70	3	18	76	15	18

Group 1 Group 2

Colored rectangle represents executing process

$$\mathrm{CPU}_j(i) = \frac{\mathrm{CPU}_j(i-1)}{2}$$

$$\mathrm{GCPU}_k(i) = \frac{\mathrm{GCPU}_k(i-1)}{2}$$

$$P_j(i) = \mathrm{Base}_j + \frac{\mathrm{CPU}_j(i)}{2} + \frac{\mathrm{GCPU}_k(i)}{4W_k}$$

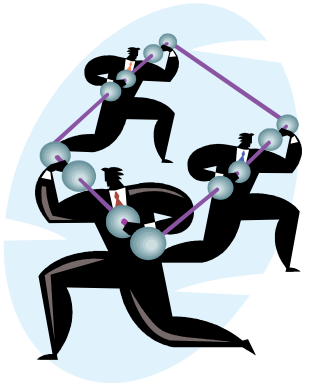
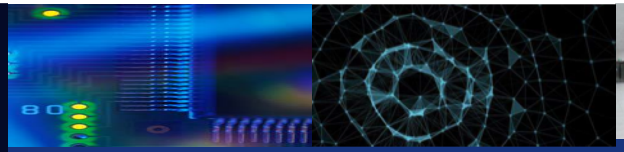


Figure 9.16 Example of Fair Share Scheduler—Three Processes, Two Groups

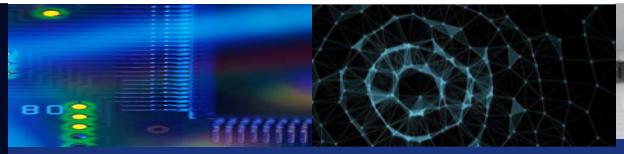


传统的Unix调度

Used in both SVR3 and 4.3 BSD UNIX

- these systems are primarily targeted at the time-sharing interactive environment

- Designed to provide good response time for interactive users while ensuring that low-priority background jobs do not starve
- Employs multilevel feedback using round robin within each of the priority queues
- Makes use of one-second preemption
- Priority is based on process type and execution history



调度形式化

$$CPU_j(i) = \frac{CPU_j(i-1)}{2}$$

$$P_j(i) = Base_j + \frac{CPU_j(i)}{2} + nice_j$$

where

$CPU_j(i)$ = measure of processor utilization by process j through interval i

$P_j(i)$ = priority of process j at beginning of interval i; lower values equal higher priorities

$Base_j$ = base priority of process j

$nice_j$ = user-controllable adjustment factor

调度形式化

Time	Process A		Process B		Process C	
	Priority	CPU count	Priority	CPU count	Priority	CPU count
0	60	0 1 2 • • 60	60	0	60	0
1	75	30	60	0 1 2 • • 60	60	0
2	67	15	75	30	60	0 1 2 • • 60
3	63	7 8 9 • • 67	67	15	75	30
4	76	33	63	7 8 9 • • 67	67	15
5	68	16	76	33	63	7

Colored rectangle represents executing process

Figure 9.17 Example of Traditional UNIX Process Scheduling

改变进程的调度策略

- Chrt 可以用于在Linux OS中查看调度相关的信息，也可以修改进程的调度策略以及优先级；

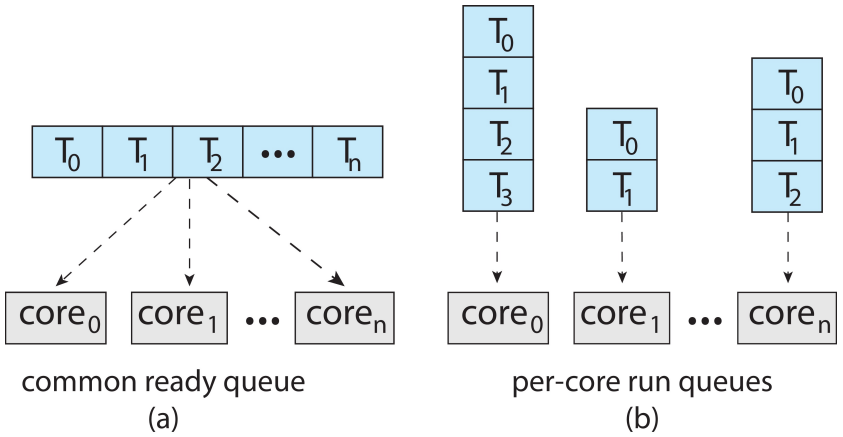
```
root@r ~# chrt -m
SCHED_OTHER min/max priority    : 0/0
SCHED_FIFO min/max priority     : 1/99
SCHED_RR min/max priority       : 1/99
SCHED_BATCH min/max priority    : 0/0
SCHED_IDLE min/max priority     : 0/0
SCHED_DEADLINE min/max priority : 0/0
root@r ~# chrt -p 32273
pid 32273's current scheduling policy: SCHED_OTHER
pid 32273's current scheduling priority: 0
root@r ~#
```

Chrt -f -p 15 （优先级）32273

```
root@r ~# chrt -f -p 15 32273
root@r ~# chrt -p 32273
pid 32273's current scheduling policy: SCHED_FIFO
pid 32273's current scheduling priority: 15
root@r ~#
```

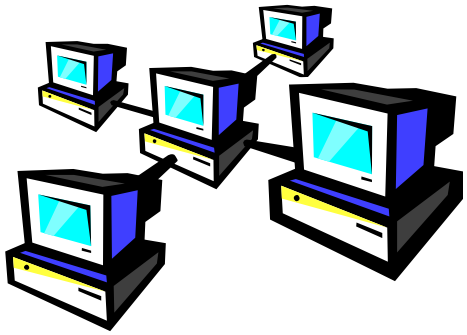
多处理器调度

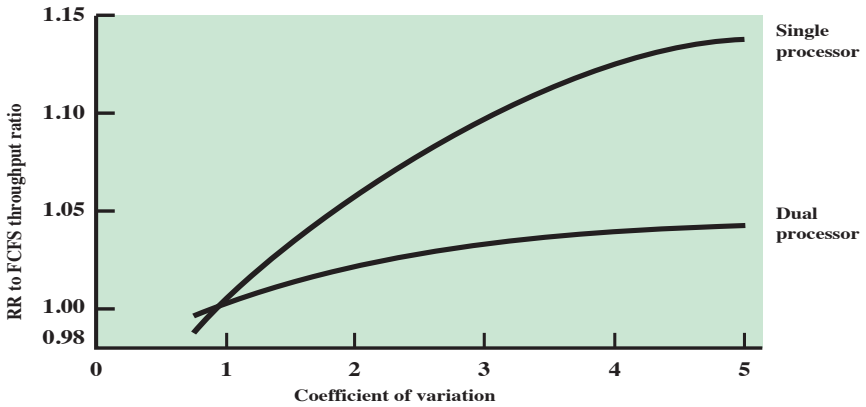
- 前面讲述的内容都是单处理器系统的调度问题，如果有多个处理器，负载分配成为可能，但是调度问题更加复杂；
- 主要关注同构处理器；
- 多处理器调度：
  - 非对称多处理器；
  - 对称多处理器（SMP）；
- **对称多处理（SMP）是指每个处理器都是自调度的。**
- **所有线程都可能位于公共就绪队列（a）中**
- **每个处理器可能有自己的线程专用队列（b）**



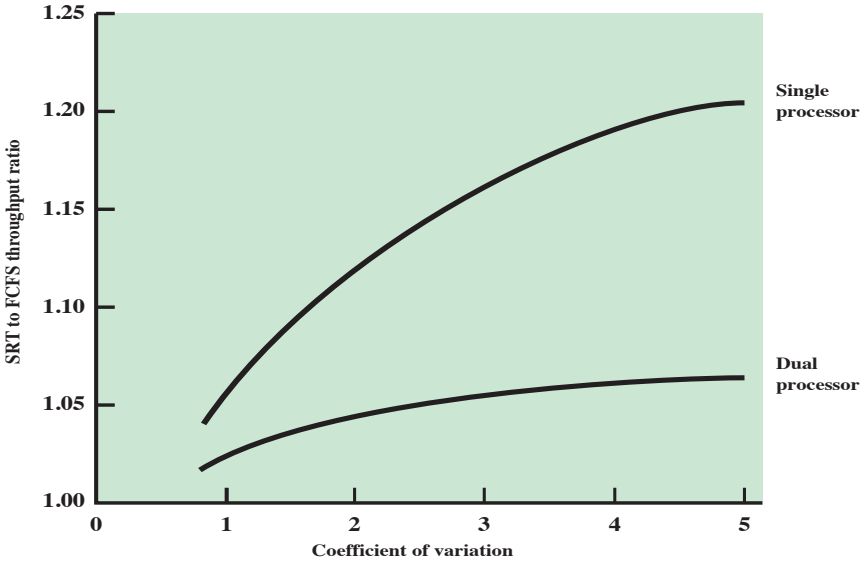
进程调度

- **Usually processes are not dedicated to processors**
- **A single queue is used for all processors**
  - if some sort of priority scheme is used, there are multiple queues based on priority
- **System is viewed as being a multi-server queuing architecture**





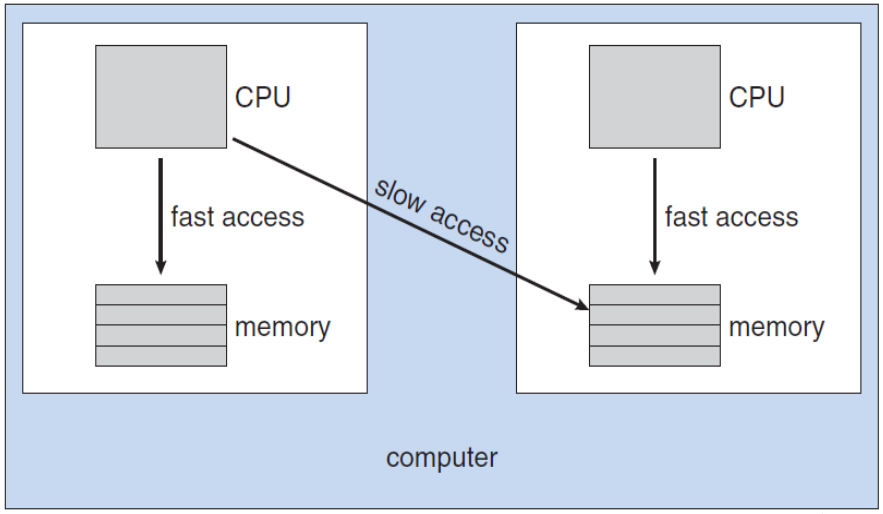
(a) Comparison of RR and FCFS



(b) Comparison of SRT and FCFS

Figure 10.1 Comparison of Scheduling Performance for One and Two Processors

SMP上的调度问题



SMP: Each processor may have its private queue of ready processes

Scheduling between processors

Process migration: Invalidating the cache of the first processor and repopulating the cache of the second processor)

Process migration is costly

处理器亲和性

Processor Affinity

Attempt to keep a process running on the same processor

Soft/hard affinity

NUMA

CPU scheduler and memory-placement algorithms work together

Load balancing

Push migration: a specific task periodically check the status & rebalance

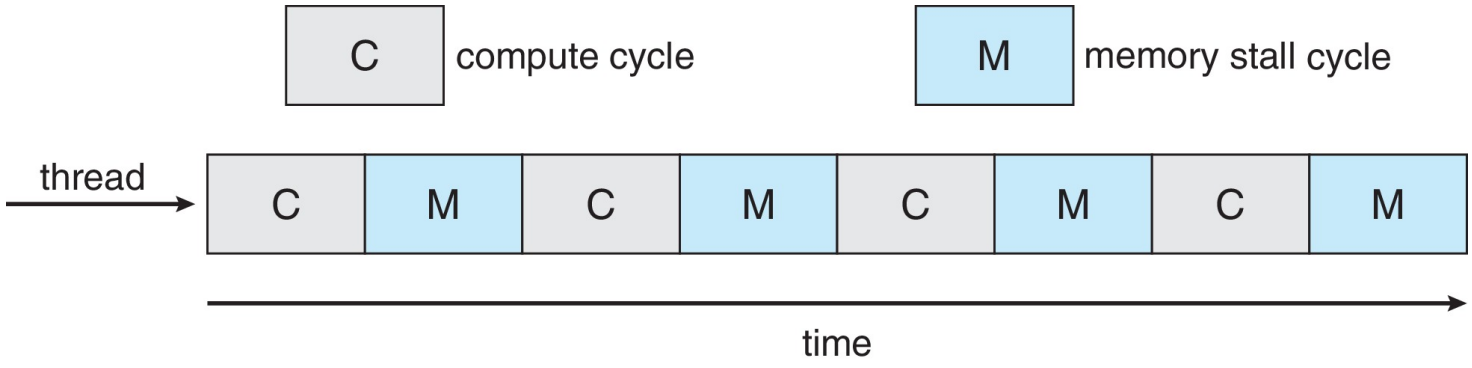
Pull migration: an idle processor pulls a waiting task from busy processor

Demo: taskset

No absolute rule concerning what policy is best

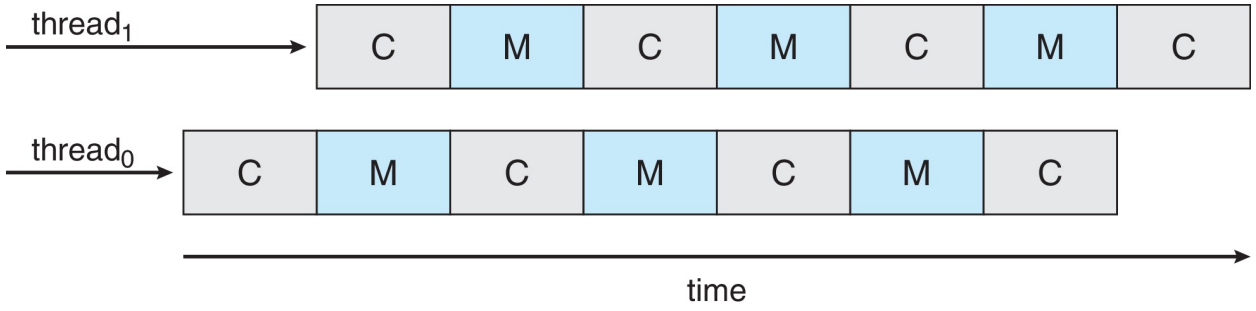
多核处理器

- 将多个处理器核心放在同一物理芯片上的最新趋势；
- 速度更快，耗电更少；
- 每个核心的线程数量也在增长；
  - 在进行内存访问时，利用内存暂停，切换到执行；
- 内存停顿，memory stall



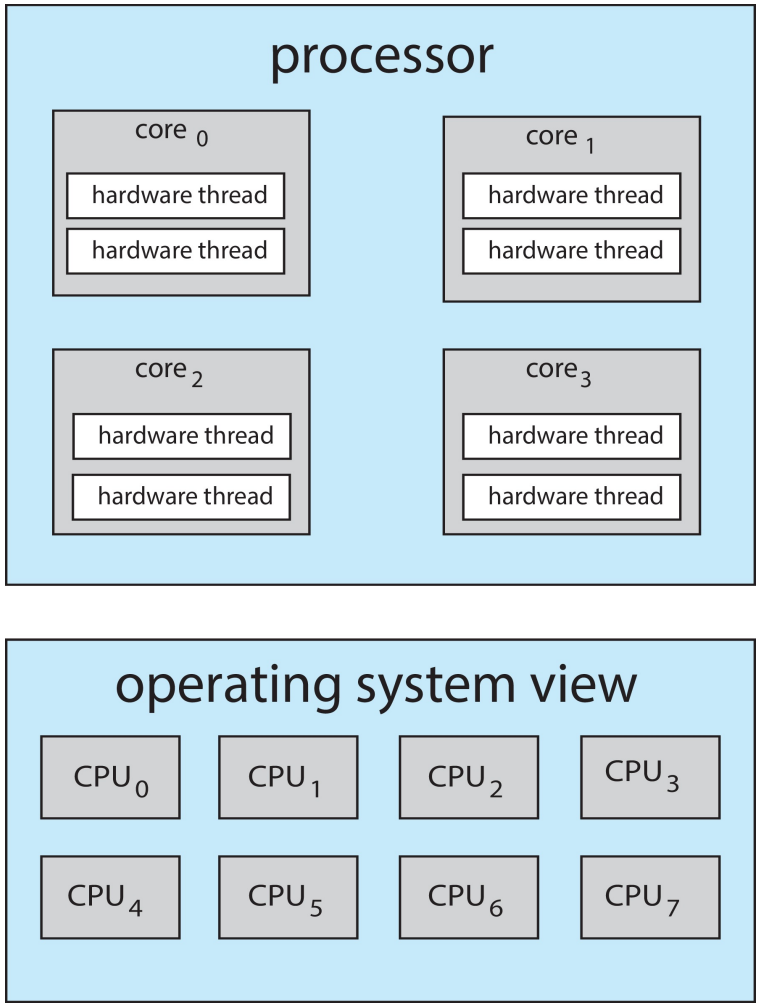
多线程多核处理器

- 每个核心都有>1个硬件线程。
- 如果一个线程发生内存暂停，切换到另一个线程！
- 如图



多线程多核处理器

- 芯片多线程（CMT）为每个核心分配多个硬件线程。（英特尔称之为超线程。）
- 在每个内核有2个硬件线程的四核系统上，操作系统可以看到8个逻辑处理器。



多线程多核处理器

- 处理器器核的多线程有两种方法：

  — 细粒度： 多线程在更细粒度级别上（通常是指令周期）切换线程，线程之间的切换时间开销很小；

  — 粗粒度：粗粒度线程，线程一直在处理器上执行，直到一个长延迟时间如内存停顿发生，线程之间的切换成本高；

- 多线程多核处理器需要两个级别的调度：一个级别由操作系统调度；另一个级别是指定每个核心如何运行哪个硬件线程；

实时系统

- 操作系统，特别是调度程序，可能是最重要的组件

Examples:

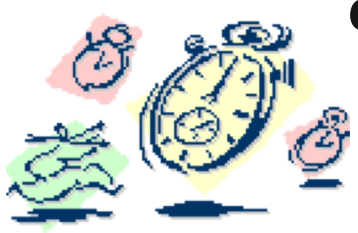
- control of laboratory experiments
- process control in industrial plants
- robotics
- air traffic control
- telecommunications
- military command and control systems

- 系统的正确性不仅取决于计算的逻辑结果，还取决于产生结果的时间；
- 任务或进程试图控制或应对外部世界发生的事件；
- 这些事件是“实时”发生的，任务必须能够跟上它们

硬实时和软实时系统

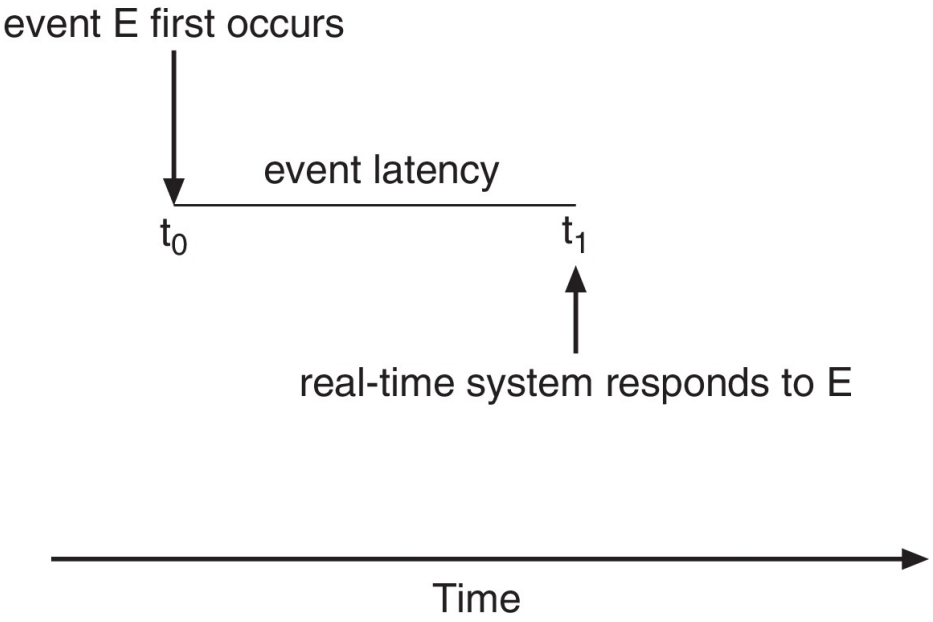
- 硬实时任务
- **one that must meet its deadline**
- **otherwise it will cause unacceptable damage or a fatal error（致命） to the system**

- 软实时任务
- **has an associated deadline that is desirable but not mandatory（强制）**
- **it still makes sense to schedule and complete the task even if it has passed its deadline**



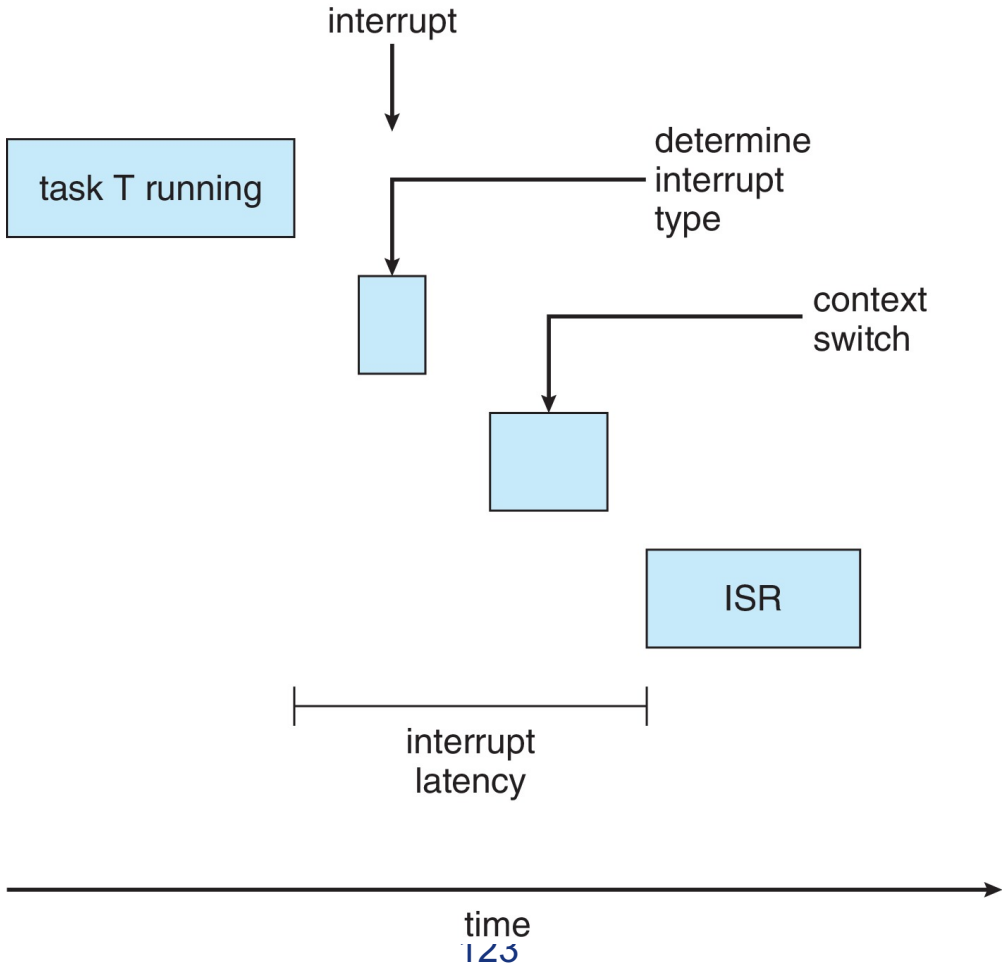
最小化延迟

- 实时系统的事件驱动性质；
- 事件发生时，系统应尽快地响应和服务它；
- Event latency(事件延迟) - 从事件发生到提供服务所经过的时间量；
- 有两种类型的延迟会影响性能
  1. 中断延迟 - 从收到中断到中断处理例程开始的时间；
  2. Dispatch latency(调度延迟)—从停止一个进程到启动另一个进程所需的时间量称为调度延迟，提供抢占式内核；



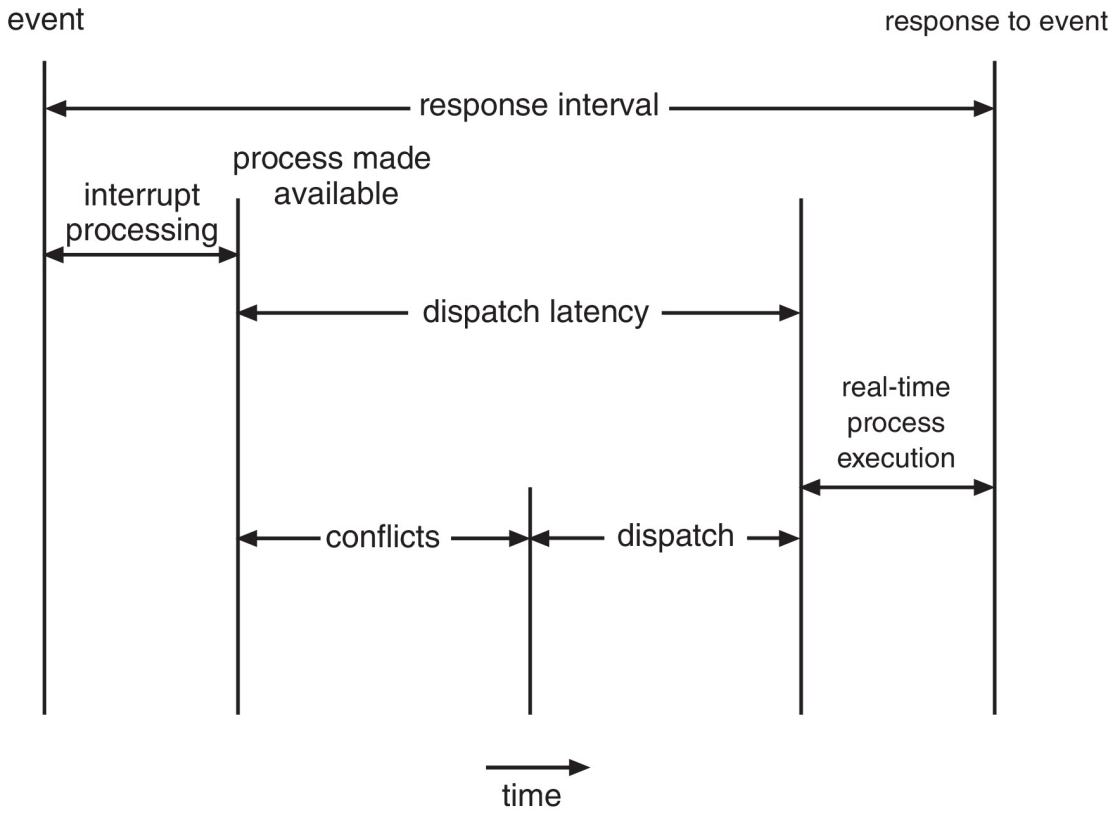
中断延迟

对实时操作新系统而言，至关重要的是，尽量减少中断延迟，以确保实时任务得到处理。



调度延迟

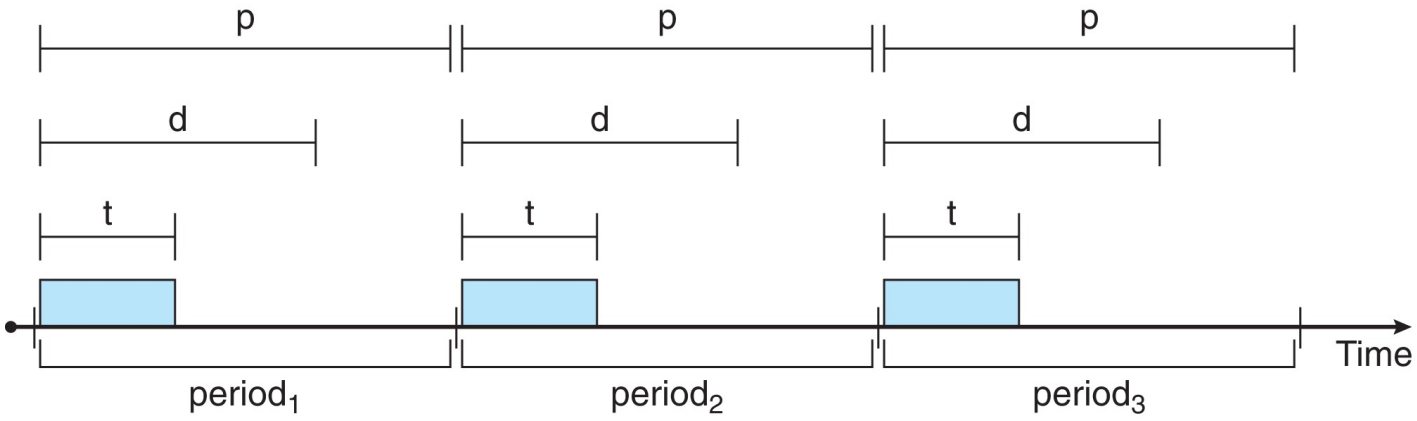
- 调度延迟的冲突阶段：
  1. 在内核模式下运行的任何进程的抢占；
  2. 低优先级进程释放高优先级进程所需的资源；



以Solaris为例，禁用抢占的调度延迟超过100ms，而启用抢占的调度延迟不到1ms，

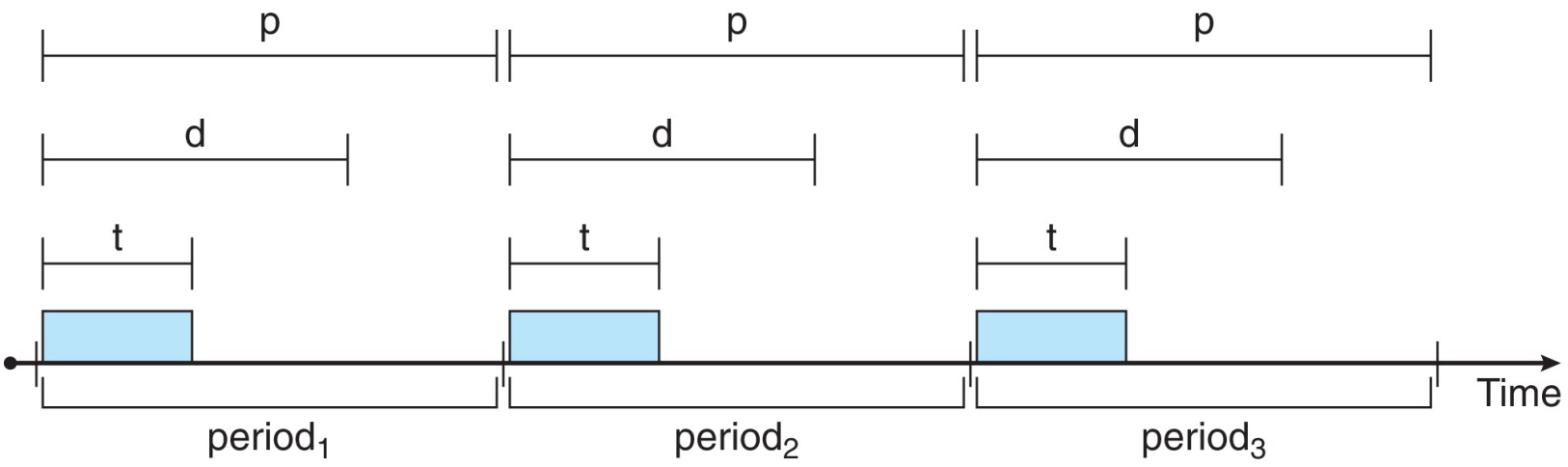
优先级调度

- 对于实时调度，调度器必须支持抢占式、基于优先级的调度
  - 但只能保证软实时性；
- 对于硬实时系统，还必须提供满足截止期限的能力，需要添加新的调度特征；
- 进程新的特点：周期性进程需要以固定的时间间隔使用CPU
  - 具有处理时间t、截止日期d、周期p
  - 0≤ T≤ D≤ P
  - 定期任务的速率为1/p



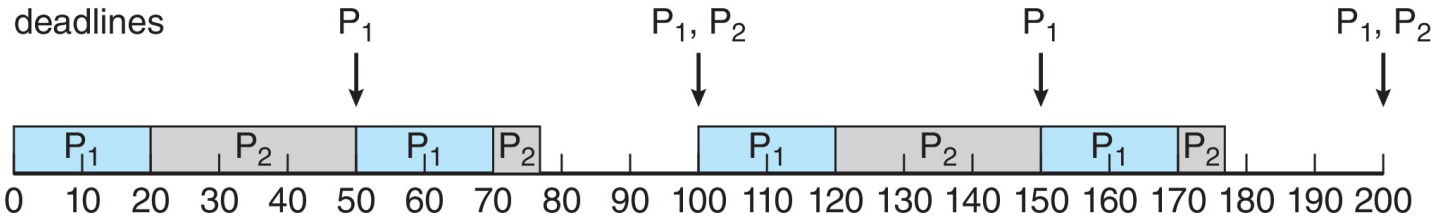
优先级调度

- 进程应向调度器公布其截止时间；
- 采用准入控制（admission-control）
  - 接受进程，保证进程完成；
  - 如果不能保证任务能够在截止期限前完成，拒绝请求；



单调速率调度

- 单调速率调度算法采用抢占的、静态优先级的策略，调度周期性任务；
- 每个周期性任务分配一个优先；
- 优先级是根据其周期的倒数来分配的；
- 周期越短->优先级越高；
- 周期越长->优先级越低
- P1（周期为50,处理时间为20)的优先级高于P2（周期为100，处理时间为35）；截止时间为下一个周期开始的时间

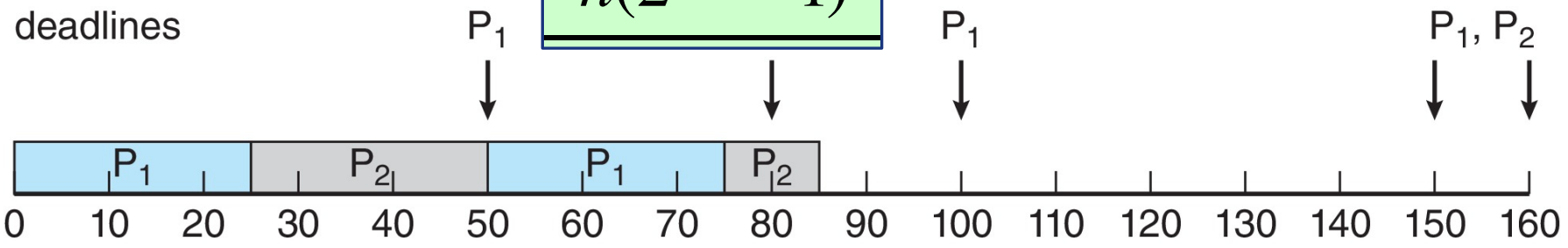


单调速率调度可认为是最优的，因为如果一组进程不能由此算法调度，它不能由任何其他分配静态优先级的算法调度。

错过截止时间的单调速率调度

- 假设进程P1 周期为50，执行时间为25；进程P2对应的值是80，35；
- 进程2并不能在截止时间80完成；
- 单调速率调度有一个限制：CPU利用率是有限的，并不能完全最大化CPU资源利用率，调度N个进程的最坏情况下的CPU利用率为

$$n(2^{1/n}-1)$$

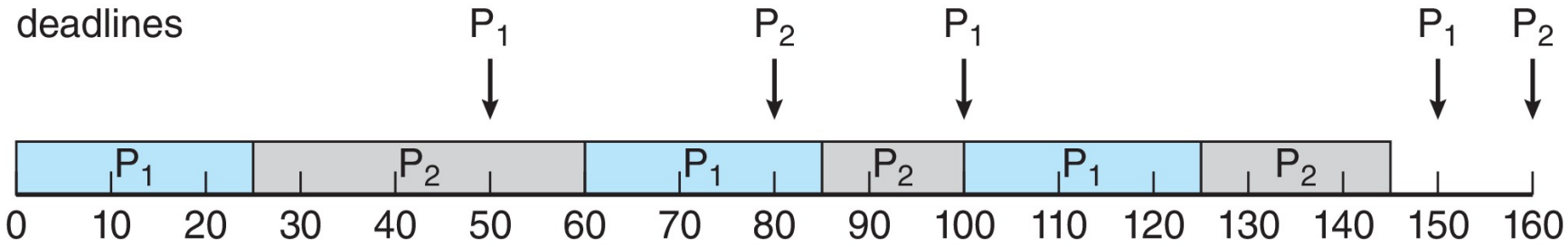


单调速率调度CPU利用率

n	$n(2^{1/n}-1)$
1	1.0
2	0.828
3	0.779
4	0.756
5	0.743
6	0.734
•	•
•	•
•	•
∞	$\ln 2 \approx 0.693$

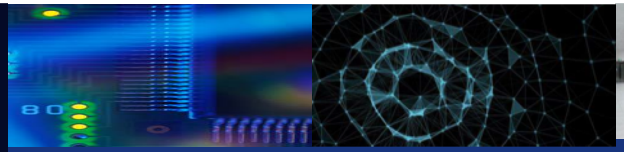
最早截止期限有限调度（EDF）

- 调度根据截止期限动态分配优先级；截止期限越早，优先级越高；截止期限越晚，优先级越低；
- 单调速率调度的优先级是固定的，而EDF是动态的；
- 此外，EDF调度不要求进程是周期执行的，也不要求进程的CPU时间是固定的，唯一的要求是：进程在变成可运行时，应给出它的截止时间。理论最佳，CPU利用率会到100%。



比例分享调度（Proportional share）

- 调度程序在所有进程之间分配T股（通证）；
- 应用程序接收N股，其中N<T；
- 这确保每个应用程序将收到总处理器时间的N/T；
- 采用准入控制策略，以确保每个进程能够得到分配时间。准入控制策略是：只有客户请求的股数小于可用的股数，才能允许客户进入。



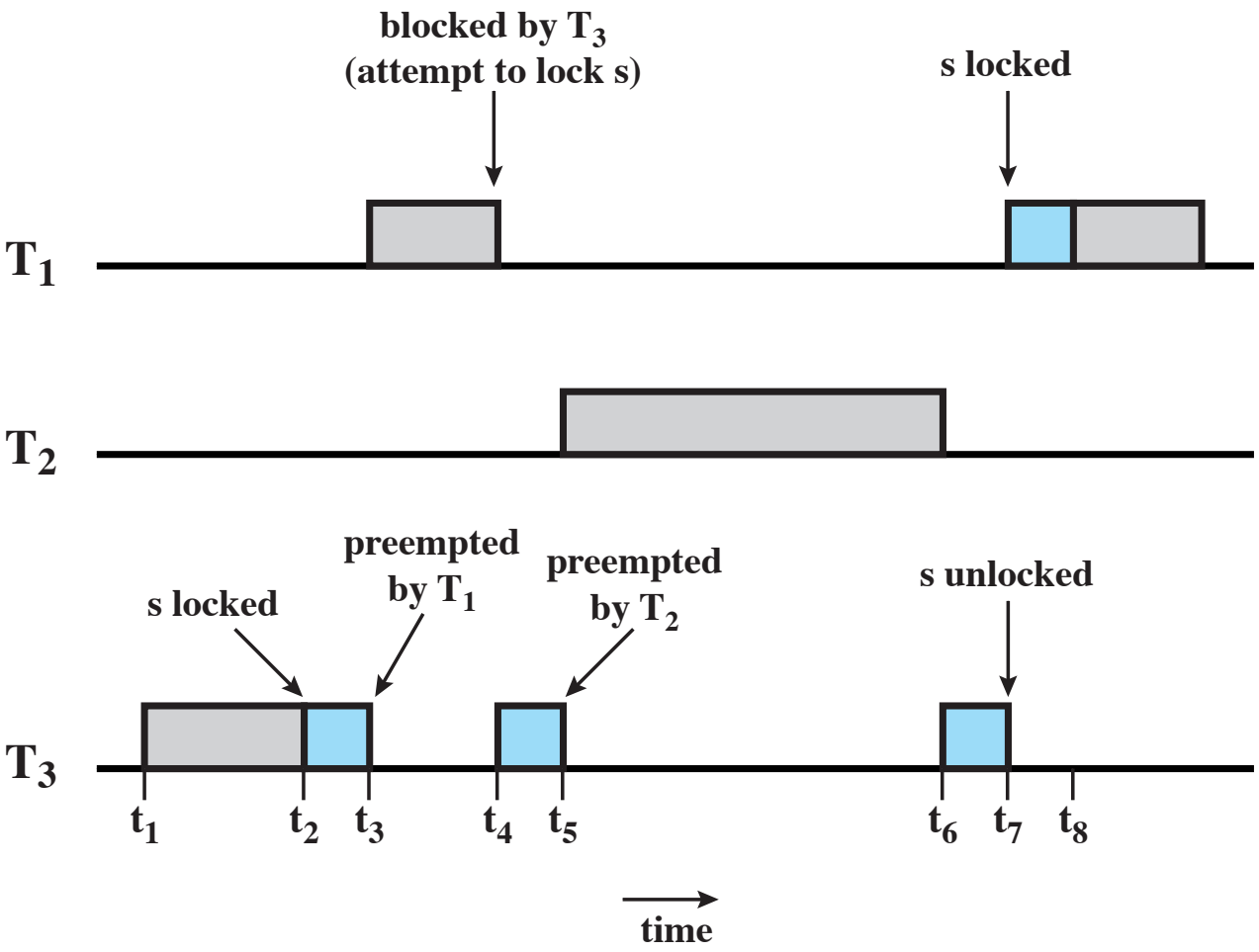
优先级翻转

- 可以在任何基于优先级的抢占式调度方案中发生;
- 在实时调度的背景下尤其重要;
- 最著名的例子是火星探路者任务;
- 当系统内的情况迫使较高优先级的任务等待较低优先级的任务时发生;

无界优先级反转

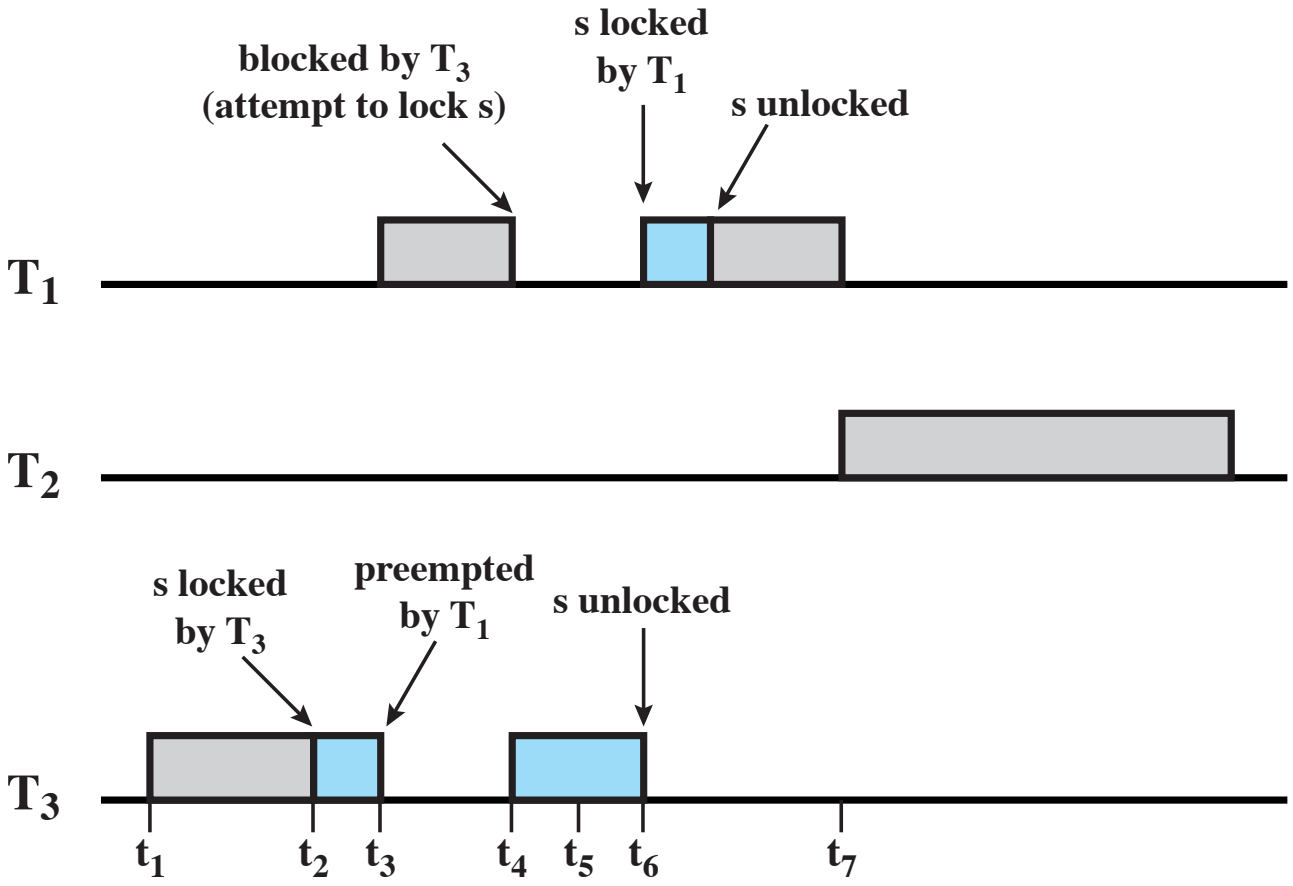
- 优先级反转的持续时间不仅取决于处理共享资源所需的时间，还取决于其他不相关任务的不可预测的操作

无边界优先级翻转



(a) Unbounded priority inversion

优先级继承



(b) Use of priority inheritance

Linux调度

- **The three classes are:**
  - SCHED_FIFO: First-in-first-out real-time threads
  - SCHED_RR: Round-robin real-time threads
  - SCHED_OTHER: Other, non-real-time threads
- **Within each class multiple priorities may be used**

Posix实时调度

- POSIX.1b标准
- API提供用于管理实时线程的函数
- 为实时线程定义两个调度类：
  1. SCHED_FIFO-线程使用FCFS策略和FIFO队列进行调度。对于优先级相同的线程，没有时间片；
  2. SCHED_RR-与SCHED_FIFO类似，不同之处在于对优先级相同的线程进行时间切片；
- 定义两个用于获取和设置计划策略的函数：
  1. pthread_attr_getsched_策略（pthread_attr_t*attr，int*policy）
  2. pthread_attr_setsched_策略（pthread_attr_t*attr，int策略）

Posix实时调度

- POSIX Real-Time Scheduling API
- #include <pthread.h>
- #include <stdio.h>
- #define NUM_THREADS 5
- int main(int argc, char *argv[])
- {
- int i, policy; pthread_t_tid[NUM_THREADS];
- pthread_attr_t attr;
- /* get the default attributes */
- pthread_attr_init(&attr);
- /* get the current scheduling policy */ if (pthread_attr_getschedpolicy(&attr, &policy) != 0)
- fprintf(stderr, "Unable to get policy.\n");
- else {
- if (policy == SCHED_OTHER) printf("SCHED_OTHER\n");
- else if (policy == SCHED_RR) printf("SCHED_RR\n");
- else if (policy == SCHED_FIFO) printf("SCHED_FIFO\n");
- }

Posix实时调度

- /* set the scheduling policy - FIFO, RR, or OTHER */ if (pthread_attr_setschedpolicy(&attr, SCHED_FIFO) != 0)
- fprintf(stderr, "Unable to set policy.\n");
- /* create the threads */ for (i = 0; i < NUM_THREADS; i++)
- pthread_create(&tid[i],&attr,runner,NULL);
- /* now join on each thread */ for (i = 0; i < NUM_THREADS; i++)
- pthread_join(tid[i], NULL);
- }
-
- /* Each thread will begin control in this function */
- void *runner(void *param){
- /* do some work ... */
- pthread_exit(0);
- }
- POSIX Real-Time Scheduling API (Cont.)

Linux调度

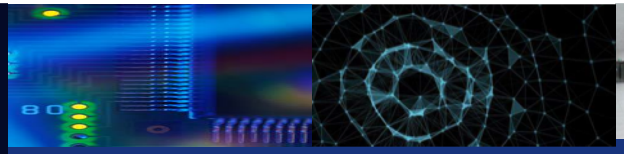
A	minimum
B	middle
C	middle
D	maximum

(a) Relative thread priorities

(b) Flow with FIFO scheduling

(c) Flow with RR scheduling

Figure 10.10 Example of Linux Real-Time Scheduling



Linux调度

- 在内核版本2.5之前，运行标准UNIX调度算法的变体
- 版本2.5移到固定顺序O（1）调度时间
  - 基于优先级的调度；
  - 两个优先级范围：分时和实时；
  - 实时任务优先级范围从0到99，普通任务优先级从100到139；
  - 映射到全局优先级，数值越小表示优先级越高；
  - 优先级越高，q值越大；
  - 只要时间片中剩余时间，任务就可以运行（活动）；
  - 如果没有剩余时间（过期），则在所有其他任务使用其时间片之前无法运行；
  - 每个CPU运行队列数据结构中跟踪的所有可运行任务
    - 两个优先级阵列（活动、过期）；
    - 按优先级索引的任务；
    - 当不再处于活动状态时，将交换阵列；
  - 工作正常，但交互过程的响应时间很短；

非实时调度0(1)

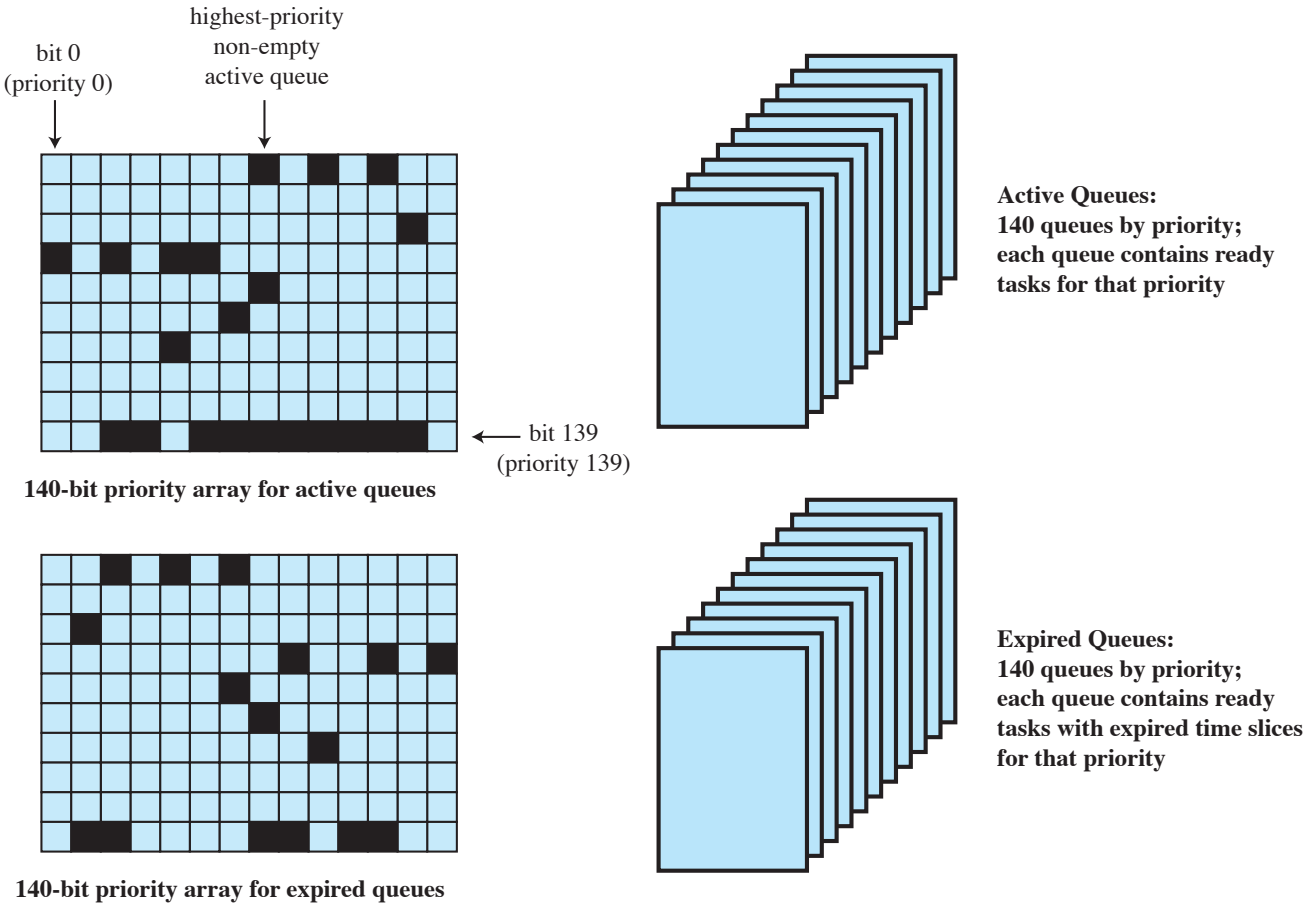


Figure 10.11 Linux Scheduling Data Structures for Each Processor

UNIX SVR4 调度

- **A complete overhaul of the scheduling algorithm used in earlier UNIX systems**

The new algorithm is designed to give:

- highest preference to real-time processes
- next-highest preference to kernel-mode processes
- lowest preference to other user-mode processes

- Major modifications:
  - addition of a preemptable static priority scheduler and the introduction of a set of 160 priority levels divided into three priority classes
  - insertion of preemption points

UNIX SVR4 优先级类型

Priority Class	Global Value	Scheduling Sequence
Real-time	159 • • • • 100	first ↓
Kernel	99 • • 60	
Time-shared	59 • • • • 0	↓ last

UNIX SVR4 优先级类型

Real time (159 – 100)

- guaranteed to be selected to run before any kernel or time-sharing process
- can preempt kernel and user processes

Kernel (99 – 60)

- guaranteed to be selected to run before any time-sharing process, but must defer to real-time processes

Time-shared (59-0)

- lowest-priority processes, intended for user applications other than real-time applications

UNIX SVR4 分发队列

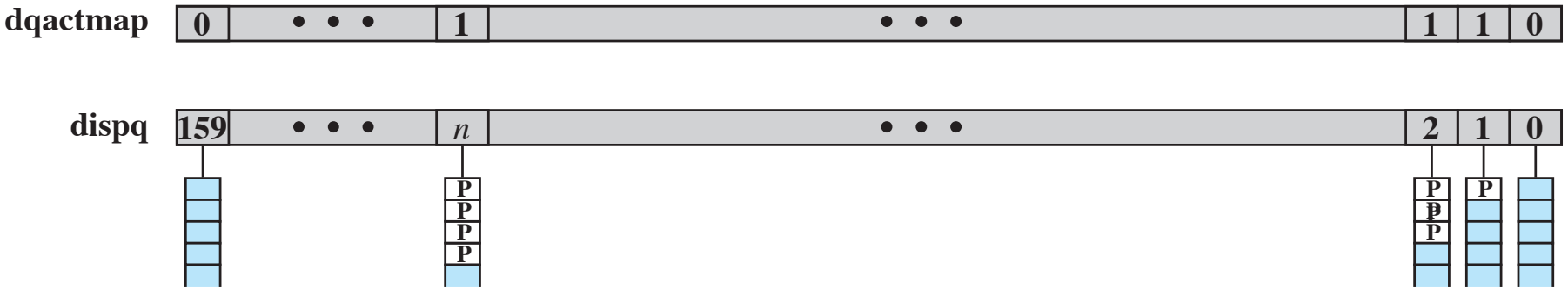
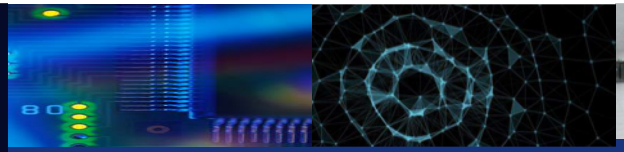


Figure 10.13 SVR4 Dispatch Queues

FreeBSD线程调度类

Priority Class	Thread Type	Description
0 - 63	Bottom-half kernel	Scheduled by interrupts. Can block to await a resource.
64 - 127	Top-half kernel	Runs until blocked or done. Can block to await a resource.
128 - 159	Real-time user	Allowed to run until blocked or until a higher priority thread becomes available. Preemptive scheduling.
160 - 223	Time-sharing user	Adjusts priorities based on processor usage.
224 - 255	Idle user	Only run when there are no time sharing or real-time threads to run.

Note: Lower number corresponds to higher priority



后续Linux版本

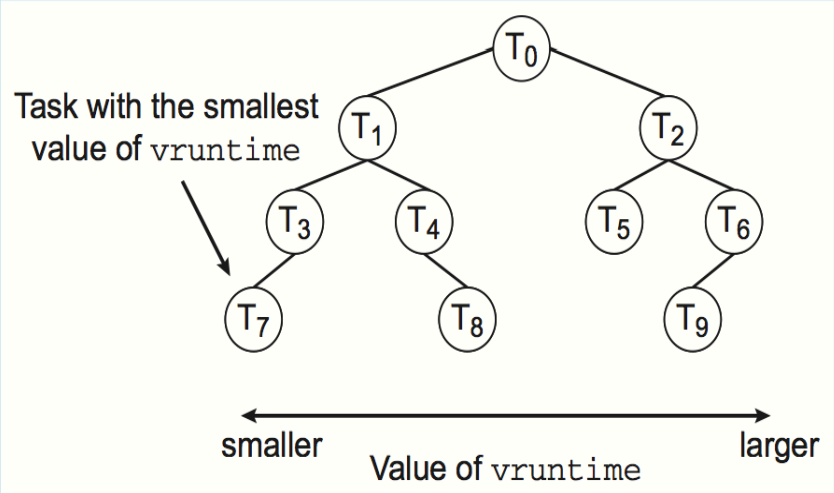
- 内核在2.6以后，完全公平调度程序（CFS）
- Linux系统的调度基于调度类；
- 内容
  - 每个调度类都有特定的优先权；
  - 调度程序在最高调度类中选择优先级最高的任务；
  - 而不是基于固定时间分配的时间片，基于CPU时间的比例；
  - 包括两个调度类，可以添加其他调度类；
    1. 采用CFS调度算法的默认调度类；
    2. 实时

后续Linux版本的调度器

- 根据从-20到+19的nice值计算时间片；
  - 值越低优先级越高；
  - 计算目标延迟 - 任务至少运行一次的时间间隔；
  - 如果活动任务的数量增加，目标延迟可能会增加；
- CFS调度程序在变量vruntime中维护每个任务的虚拟运行时间
  - 与基于任务优先级的衰减因子相关 - 优先级越低，衰减率越高；
  - 正常默认优先级产生虚拟运行时间=实际运行时间；
- 为了决定下一个要运行的任务，调度器选择虚拟运行时间最低的任务；

CFS性能

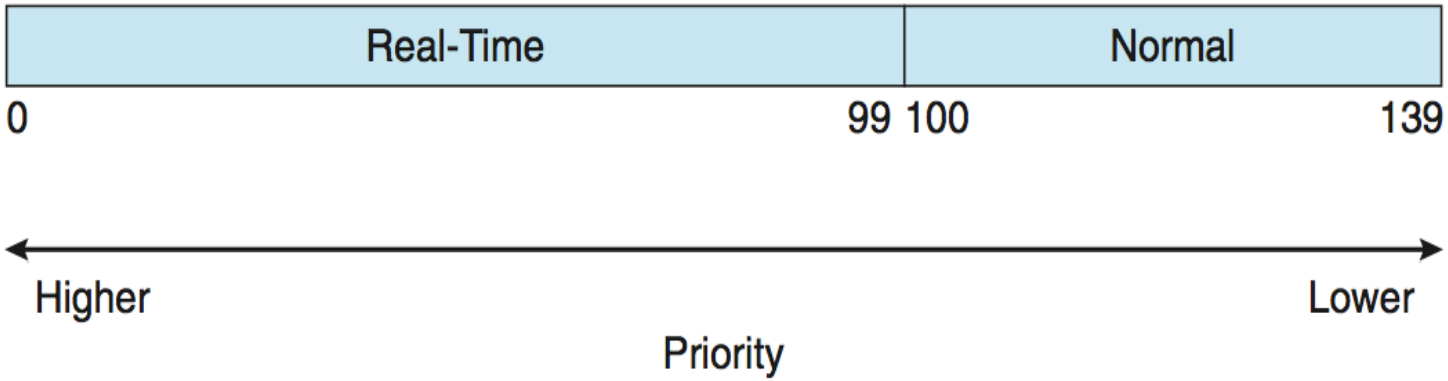
The Linux CFS scheduler provides an efficient algorithm for selecting which task to run next. Each runnable task is placed in a red-black tree—a balanced binary search tree whose key is based on the value of `vruntime`. This tree is shown below:



When a task becomes runnable, it is added to the tree. If a task on the tree is not runnable (for example, if it is blocked while waiting for I/O), it is removed. Generally speaking, tasks that have been given less processing time (smaller values of `vruntime`) are toward the left side of the tree, and tasks that have been given more processing time are on the right side. According to the properties of a binary search tree, the leftmost node has the smallest key value, which for the sake of the CFS scheduler means that it is the task with the highest priority. Because the red-black tree is balanced, navigating it to discover the leftmost node will require $O(lg\,N)$ operations (where N is the number of nodes in the tree). However, for efficiency reasons, the Linux scheduler caches this value in the variable `rb_leftmost`, and thus determining which task to run next requires only retrieving the cached value.

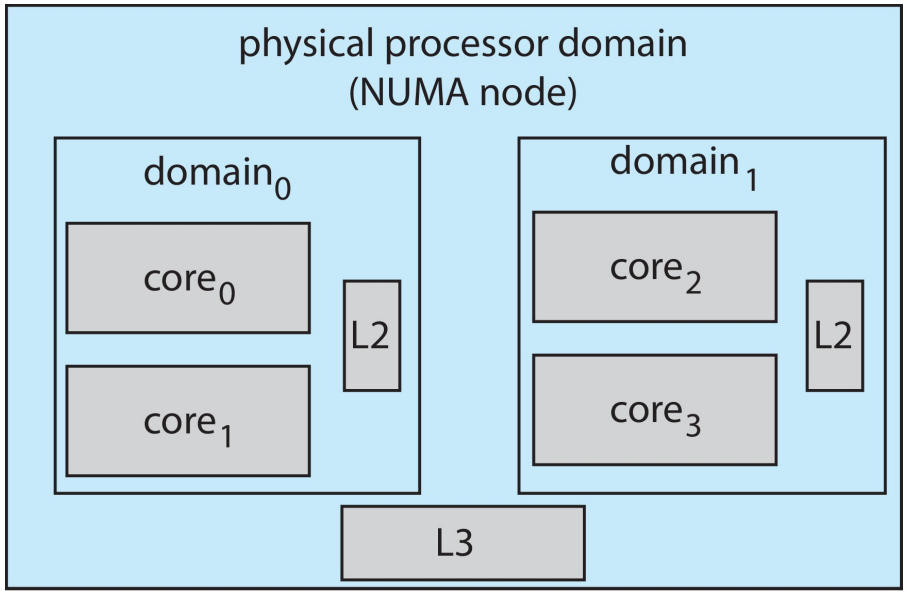
Linux 调度优先级

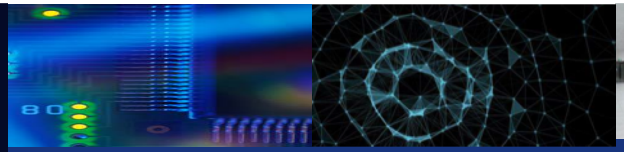
- 根据POSIX实时调度.1b
  - 实时任务具有静态优先级，0~99，普通任务：100~139
- 两个值域合并映射到全局优先级方案；
- nice的值-20映射到全局优先级100；
- +19的nice值映射到优先级139；



Linux 调度优先级

- Linux支持负载平衡，但也支持NUMA。
- 调度域是一组可以相互平衡的CPU核心。
- 域按它们共享的内容（即缓存）进行组织目标是防止线程在域之间迁移。



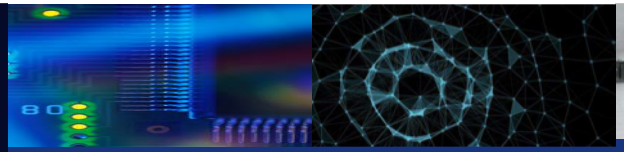


Windows调度器

- Windows使用基于优先级的抢占式调度;
- 下一个运行最高优先级的线程;
- 分派器就是调度器
- 线程运行直到（1）阻塞，（2）使用完时间片，（3）被更高优先级的线程抢占;
- 实时线程可以抢占非实时线程;
- 32级优先权方案;
- 优先级可变类为1-15，实时类线程优先级为16-31;
- 优先级0是内存管理线程;
- 为每个优先级排队;
- 如果没有可运行线程，则运行空闲线程（IDLE）;

Windows调度器

- Win32 API标识进程可以属于的几个优先级类
  - REALTIME_PRIORITY_CLASS,
  - HIGH_PRIORITY_CLASS,
    ABOVE_NORMAL_PRIORITY_CLASS,
  - NORMAL_PRIORITY_CLASS,
  - BELOW_NORMAL_PRIORITY_CLASS,
  - IDLE_PRIORITY_CLASS
  - 除实时类外，其他线程的优先级都是可变的；
- 给定优先级类中的线程具有相对优先级
  - TIME_CRITICAL, HIGHEST, ABOVE_NORMAL, NORMAL, BELOW_NORMAL, LOWEST, IDLE
  - 优先级类和相对优先级结合起来，以提供优先级数值；
- 基本优先级在类中是NORMAL；
- 如果时间片用完，优先级会降低，但决不会低于优先级基准；




Windows调度器

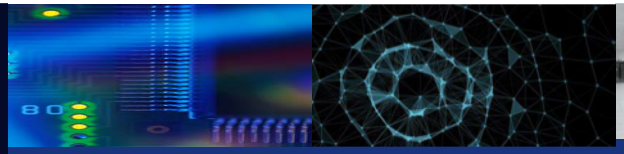
- 如果发生等待，则根据等待的内容提升优先级
- 前景窗口的优先级提高了3倍
- Windows 7添加了用户模式调度（UMS）
  - 应用程序创建和管理独立于内核的线程
  - 对于大量线程，效率更高
  - UMS调度程序来自C++等并行运行时（CORCRT）框架的编程语言库

Windows优先级

	real-time	high	above normal	normal	below normal	idle priority
time-critical	31	15	15	15	15	15
highest	26	15	12	10	8	6
above normal	25	14	11	9	7	5
normal	24	13	10	8	6	4
below normal	23	12	9	7	5	3
lowest	22	11	8	6	4	2
idle	16	1	1	1	1	1

Solaris调度

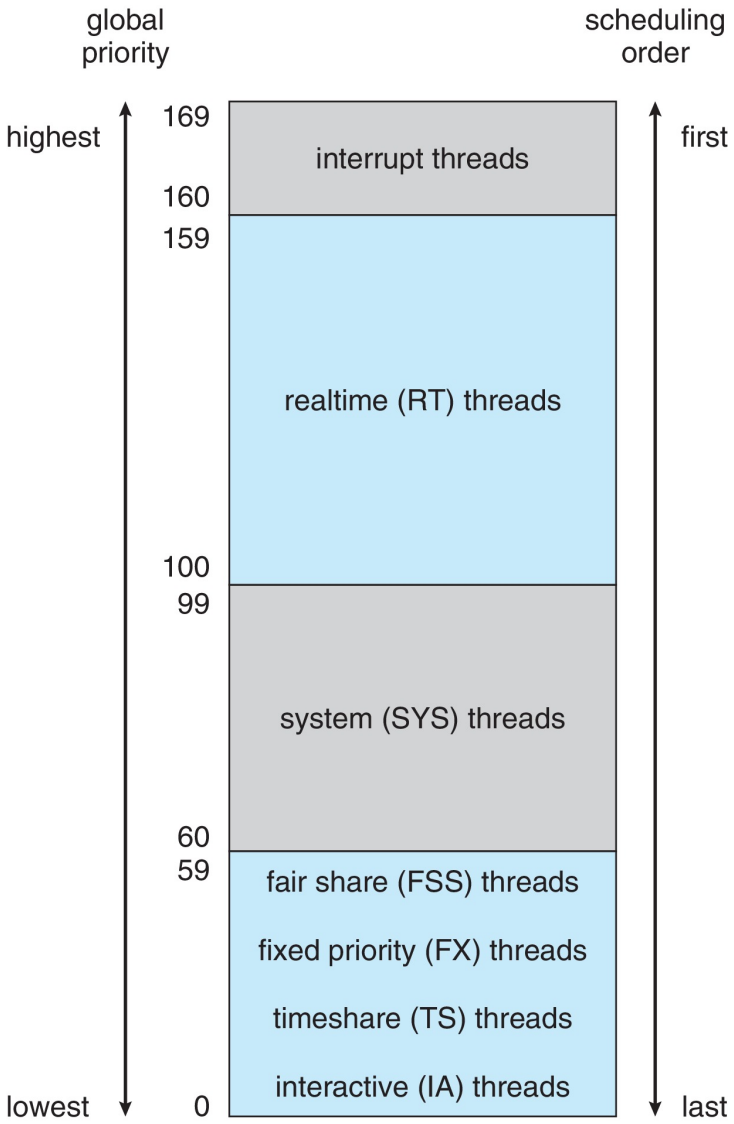
- 基于优先级的调度
- 有六个分类
  - 分时（默认）（TS）
  - 交互式（IA）
  - 实时（RT）
  - 系统（SYS）
  - 公平份额（FSS）
  - 固定优先级（FP）
- 给定的线程一次只能在一个类中
- 每个类都有自己的调度算法
- 分时是多级反馈队列
  - 可由sysadmin配置的可加载表



Solaris调度表

priority	time quantum	time quantum expired	return from sleep
0	200	0	50
5	200	0	50
10	160	0	51
15	160	5	51
20	120	10	52
25	120	15	52
30	80	20	53
35	80	25	54
40	40	30	55
45	40	35	56
50	40	40	58
55	40	45	58
59	20	49	59

Solaris调度


global priority
scheduling order
highest
lowest
first
last
169
160
159
100
99
60
59
0
interrupt threads
realtime (RT) threads
system (SYS) threads
fair share (FSS) threads
fixed priority (FX) threads
timeshare (TS) threads
interactive (IA) threads

Solaris调度

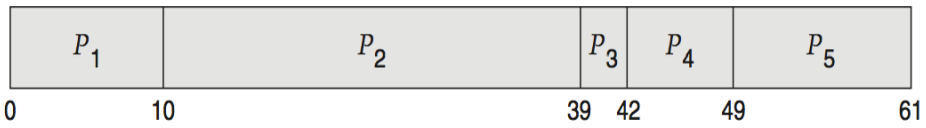
- 调度器将特定于类的优先级转换为每个线程的全局优先级
  - 下一个运行优先级最高的线程
  - 运行直到（1）阻塞，（2）使用完时间片，（3）被更高优先级的线程抢占；
  - 通过RR选择具有相同优先级的多个线程；

算法评价

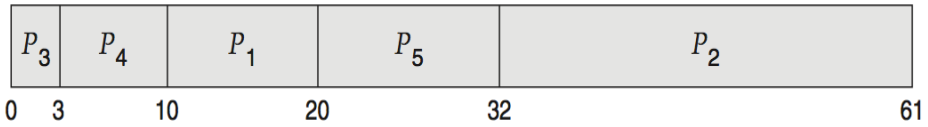
- 如何为操作系统选择CPU调度算法？
- 确定标准，然后评估算法
- 确定性建模
  - 分析评价类型；
  - 获取特定的预定工作负载，并为该工作负载计算每个算法的性能；
- 考虑在时间0到达的5个过程：

确定性评价

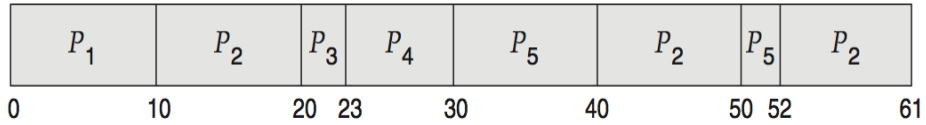
- 对于每个算法，计算最小平均等待时间
- 简单快速，但需要输入精确数字，仅适用于这些输入
  - FCS is 28ms：



  - 非抢占式SFJ为13ms：



  - RR为23ms：



排队模型

- 描述进程的到达，以及CPU和I/O执行的概率
  - 通常是指数型的，用平均值来描述
  - 计算平均吞吐量、利用率、等待时间等。
- 描述为服务器网络的计算机系统，每个服务器都有等待进程队列
  - 了解到达率和服务率
  - 计算利用率、平均队列长度、平均等待时间等。

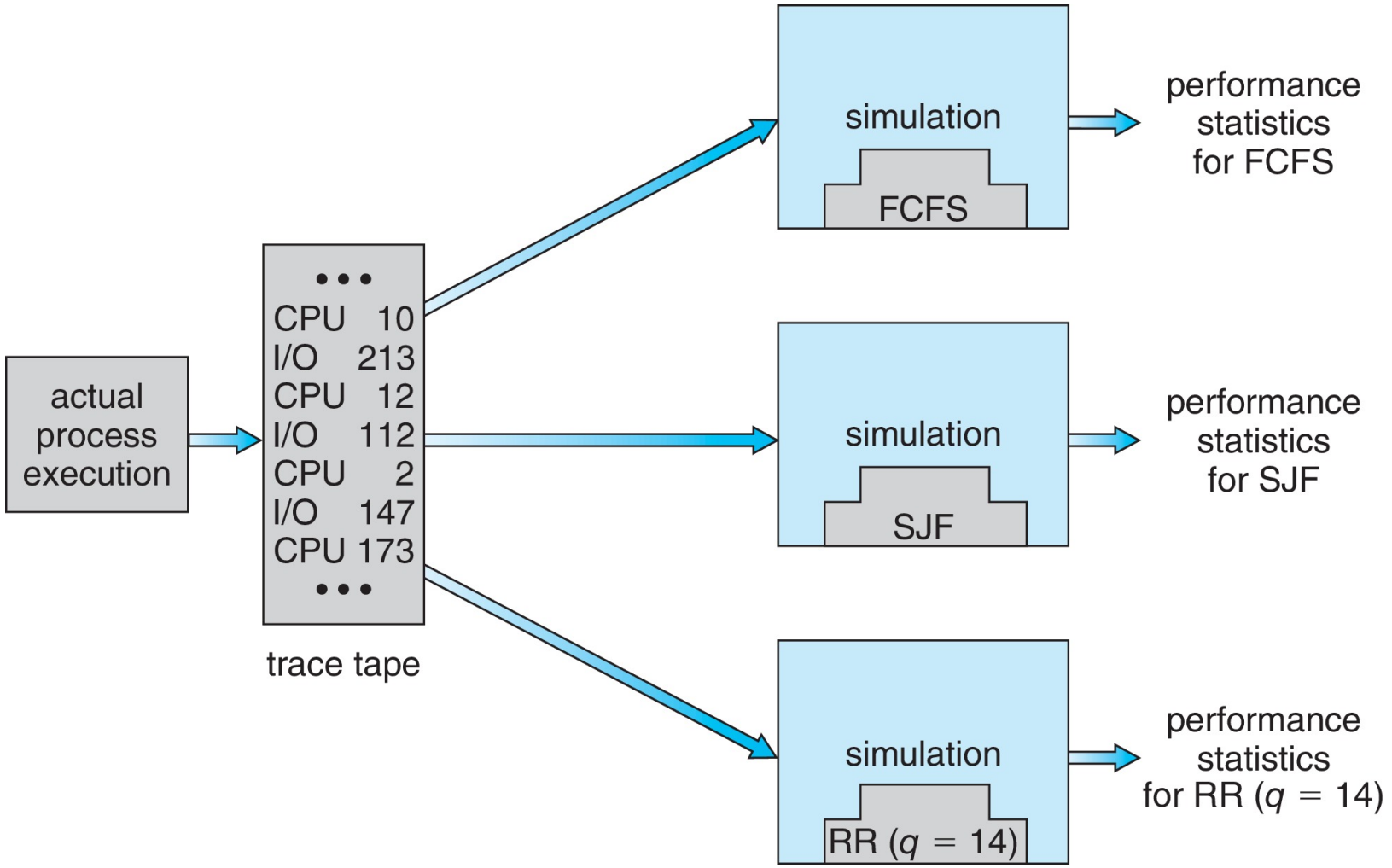
Little’s Law

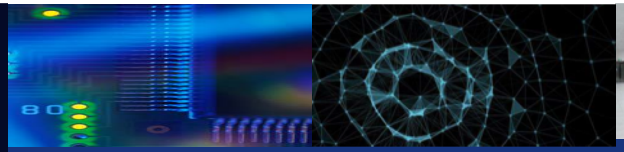
- n=平均队列长度
- W=队列中的平均等待时间
- λ=进入队列的平均到达率
- 利特尔定律(Little’s Law)——在稳定状态下，离开队列的进程必须等于到达队列的进程，因此：n=λx W
  - 适用于任何调度算法和到达分布
- 例如，如果平均每秒到达7个进程，并且队列中通常有14个进程，则每个进程的平均等待时间=2秒

Little’s Law

- 排队模型能力有限
- 更精确的模拟
  - 计算机系统的程序模型
  - 时钟是一个变量
  - 收集指示算法性能的统计信息
  - 用于驱动通过以下方式收集的模拟的数据：
    - 基于概率的随机数发生器
    - 数学上或经验上定义的分布
    - 跟踪磁带记录真实系统中真实事件的序列

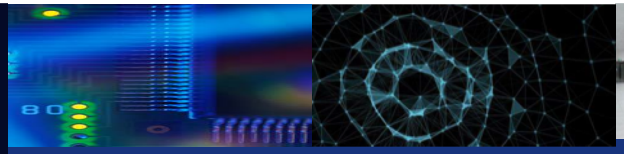
CPU调度器仿真


actual process execution
CPU 10
I/O 213
CPU 12
I/O 112
CPU 2
I/O 147
CPU 173
trace tape
simulation
FCFS
performance statistics for FCFS
simulation
SJF
performance statistics for SJF
simulation
RR (q = 14)
performance statistics for RR (q = 14)




实现

- 即使是模拟，其准确性也是有限的；
- 在实际系统中实现新的调度器和测试
  - 高成本、高风险；
  - 环境各不相同；
- 最为灵活的调度程序可以根据管理员或用户进行修改
- 利用API或者命令行来修改优先级；
- 但环境也各不相同；



谢谢